

学校代码: 10246
学 号: 23210240413

復旦大學

硕 士 学 位 论 文
(专业学位)

面向 DSP 的编译器全局指令选择的实现、优化
及评估

Implementation, Optimization and Evaluation of Global
Instruction Selection for DSP Compiler

院 系: 计算与智能创新学院
专业学位类别(领域): 计算机技术
姓 名: 周星宇
指 导 教 师: 吴俊 教授
完 成 日 期: 2026 年 3 月 20 日

指导小组成员

吴 俊 教 授

目 录

摘要	v
Abstract	vii
第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 DSP 架构简介	1
1.1.2 编译器研究背景	1
1.1.3 指令选择研究背景	3
1.1.4 研究意义	4
1.2 国内外研究现状	5
1.2.1 编译器研究现状	5
1.2.2 指令选择研究现状	7
1.3 论文研究内容和创新点	9
1.3.1 研究内容	9
1.3.2 创新点凝练	10
1.4 论文组织结构	11
第 2 章 面向 DSP 的全局指令选择实现	13
2.1 全局指令选择框架概述	14
2.1.1 传统指令选择方案及其局限性	14
2.1.2 全局指令选择原理及框架设计	16
2.2 GMIR 生成	18
2.2.1 通用机器表示	18
2.2.2 GMIR 生成方法	20
2.2.3 GMIR 生成实现	21
2.3 指令合法化	23
2.3.1 指令合法化方法	24
2.3.2 指令合法化实现	25
2.4 寄存器组选择	26

2.4.1	寄存器组	27
2.4.2	寄存器组选择方法	27
2.4.3	寄存器组选择实现	28
2.5	机器指令选择	33
2.5.1	匹配状态机	33
2.5.2	机器指令选择方法	35
2.5.3	机器指令选择实现	36
2.6	本章小结	39
第 3 章	面向 DSP 的全局指令选择优化策略	41
3.1	优化策略分析与设计	41
3.1.1	优化策略理论依据	42
3.1.2	关键优化技术分析	43
3.1.3	面向 DSP 的优化设计	45
3.2	合法化前优化策略	47
3.2.1	内存操作优化	47
3.2.2	拓展截断优化	48
3.3	合法化后优化策略	51
3.3.1	立即数装载优化	51
3.3.2	乘法优化	53
3.4	本章小结	55
第 4 章	测试评估平台设计与实现	57
4.1	DSP 编译器测试现状与改进	57
4.1.1	测试流程现状	57
4.1.2	测试体系优化需求与技术路径	59
4.1.3	随机测试生成技术与生成器选型	60
4.2	平台设计	61
4.2.1	平台整体设计	61
4.2.2	测试子系统设计	62
4.2.3	评估子系统设计	63
4.3	平台实现	64
4.3.1	面向 DSP 的 YARPGen 实现	64
4.3.2	测试子系统实现	66
4.3.3	评估子系统实现	69
4.4	本章小结	71

第 5 章 实验结果与性能分析	73
5.1 GlobalISel 代码生成正确性验证	73
5.1.1 基本指令正确性验证	73
5.1.2 控制流指令正确性验证	76
5.2 GlobalISel 优化策略有效性验证	80
5.2.1 合法化前优化策略验证	80
5.2.2 合法化后优化策略验证	83
5.3 GlobalISel 综合性能对比分析	86
5.3.1 代码体积对比	86
5.3.2 执行周期对比	88
5.3.3 编译时间对比	89
5.4 测试评估平台功能展示	91
5.5 本章小结	94
第 6 章 总结与展望	95
6.1 论文总结	95
6.2 论文展望	95
参考文献	97
致谢	103

摘要

随着 DSP (Digital Signal Processor, 数字信号处理器) 向高并行与复杂指令集演进, 编译器后端在指令选择阶段面临着更高的性能与工程化要求。传统的基于 DAG (Directed Acyclic Graph, 有向无环图) 的指令选择框架在可扩展性、跨基本块优化能力上存在局限。基于此, 本文针对自研 DSP 架构, 通过定制化实现 GlobalISel (全局指令选择) 框架、设计分阶段指令优化策略、构建全流程性能测试评估平台三个核心环节, 形成一套覆盖框架适配、性能优化与效果验证的完整技术方案。

首先, 针对 GlobalISel 在 DSP 架构上适配的复杂性, 本文提出了一种基于 LLVM (Low Level Virtual Machine, 一个开源的编译器基础设施项目) 通用框架的模块化适配方法。通过定制调用约定、合法化规则和寄存器组映射模型, 在不修改框架核心代码的前提下, 成功将 GlobalISel 部署至一款自研 VLIW (Very Long Instruction Word, 超长指令字) DSP 上, 解决了其高编译开销和扩展性差的固有问题。

其次, 针对 GlobalISel 在通用语义转换过程中产生的冗余, 本文设计了一套分阶段的指令选择优化体系。在合法化前阶段, 通过内存操作内联和冗余截断消除等策略, 削减通用语义转换引入的开销; 在合法化后阶段, 提出了立即数装载优化和常量乘法强度削减等面向 DSP 指令集的专属优化, 有效提升了代码密度和执行效率。

再次, 针对编译器后端在复杂指令映射下的稳健性验证与性能量化难题, 本文构建了一套集成随机测试与多维性能分析的自动化评估平台。平台引入随机测试生成技术, 对随机测试生成器进行针对目标架构的定制化扩展, 构建了一套覆盖指令选择及后端优化关键场景的随机测试输入集合。此外, 平台集成了自动化性能数据采集与多维数据分析模块, 实现了编译器优化策略的效能回溯与可视化评估, 为后端方案的迭代演进提供了精确的量化支撑。

最后, 实验结果表明, 本文实现的面向自研 DSP 的 GlobalISel 框架, 较传统的基于 DAG 的指令选择方案编译效率优势显著; 集成分阶段优化策略后, 有效缩减了生成代码体积并降低了程序执行周期。除此之外, 测试评估平台为成果验证提供系统化支撑, 印证了 GlobalISel 在 DSP 平台的优化潜力与工程价值。

关键词：全局指令选择；编译器优化；性能评估；LLVM；YARPGen

中图分类号：TP3

Abstract

As DSP (Digital Signal Processor) evolves towards higher parallelism and complex instruction sets, the compiler back-end faces higher performance and engineering requirements in the instruction selection stage. The traditional instruction selection framework based on DAG (Directed Acyclic Graph) has limitations in scalability and cross-block optimization capabilities. In response to this, this paper, targeting the self-developed DSP architecture, through customized implementation of the GlobalISel (global instruction selection) framework, design of phased instruction optimization strategies, and construction of a full-process performance testing and evaluation platform in three core aspects, forms a complete technical solution covering framework adaptation, performance optimization, and effect verification.

Firstly, considering the complexity of adapting GlobalISel to the DSP architecture, this paper proposes a modular adaptation method based on the general framework of LLVM (Low Level Virtual Machine, an open-source compiler infrastructure project). By customizing the calling convention, normalization rules, and register group mapping model, without modifying the core code of the framework, GlobalISel was successfully deployed onto a self-developed VLIW (Very Long Instruction Word) DSP, solving its inherent problems of high compilation cost and poor scalability.

Secondly, in response to the redundancies generated during the general semantic transformation process by GlobalISel, this paper designs a phased instruction selection optimization system. In the pre-normalization stage, through strategies such as inlining of memory operations and truncation of redundancies, the overhead introduced by the general semantic transformation is reduced; in the post-normalization stage, dedicated optimizations specific to the DSP instruction set, such as immediate number loading optimization and reduction of constant multiplication intensity, are proposed, effectively improving code density and execution efficiency.

Furthermore, to address the challenges of robustness verification and performance quantification in the compiler's back-end under complex instruction mapping, this paper has developed an automated evaluation platform that integrates random testing and multi-dimensional performance analysis. The platform incorporates random testing generation technology, customizes the random test generator for the target architec-

ture, and constructs a set of random test input combinations covering key scenarios of instruction selection and back-end optimization. Additionally, the platform integrates automated performance data collection and multi-dimensional data analysis modules, enabling the retroactive evaluation of the effectiveness of compiler optimization strategies and visual assessment, providing precise quantitative support for the iterative evolution of the back-end solution.

Finally, the experimental results show that the GlobalISel framework implemented in this paper for self-developed DSP has a significant advantage in compilation efficiency compared to the traditional instruction selection scheme based on DAG; after integrating the staged optimization strategy, the generated code size is effectively reduced and the program execution cycle is decreased. In addition, the test evaluation platform provides systematic support for the verification of the results, confirming the optimization potential and engineering value of GlobalISel on the DSP platform.

Keywords: GlobalISel; compiler optimization; performance evaluation; LLVM; YARPGen

CLC code: TP3

第 1 章 绪论

1.1 研究背景与意义

1.1.1 DSP 架构简介

DSP 是一种用于处理数字信号的微处理器，其原理是通过采样、量化与编码的技术手段，将连续变化的模拟信号转化为离散的数字信号，之后再借助相应算法对所得数字信号完成滤波、变换及压缩等一系列处理^[1]，DSP 的处理效率与精度直接影响终端设备的性能上限。自 20 世纪中叶诞生以来，DSP 已从早期单一功能的专用硬件电路，演进为具备可编程能力、高并行运算特性的专用处理器^[2]。近些年来，随着哈佛架构的引入^[3]、以及单指令多数据结构^[4]与超长指令字技术^[5-6]的结合，DSP 实现了从标量处理到大规模并行处理的跨越式发展。

随着 5G/6G 通信^[7]、边缘智能计算^[8]等新兴应用的爆发式增长，DSP 面临着前所未有的性能与能效双重挑战：一方面，数据吞吐量需求呈指数级提升，需要更宽的并行运算单元与更高效的指令调度机制；另一方面，终端设备的低功耗需求日益突出，要求 DSP 在提升性能的同时实现精细化的功耗管控。在这个背景下，国内外科研机构与企业纷纷加大高性能 DSP 的研发投入，但由于核心架构设计、指令集开发等关键技术存在一定的技术壁垒，国内高性能 DSP 领域仍存在自主化程度不足的问题，因此，构建具备完全自主知识产权的高性能 DSP 芯片及端到端技术体系，已成为当前亟待解决的重要课题。

本文所探讨的是一款由本实验室自主研发的高性能 DSP 芯片（在下文中简称为 DSP 芯片）。DSP 芯片基于哈佛架构设计，具备自主知识产权的指令集，通过四路并行向量处理单元、8 槽位 VLIW 指令发射机制及九级流水线设计，构建了高并行、高实时的运算架构；在寄存器的设计上，DSP 采用 32 个 32 位通用寄存器 GR、640 位宽向量寄存器 VR 以及专用循环访存寄存器组 MOB 等特殊寄存器的分层架构，能够适配标量与向量运算的需求。

1.1.2 编译器研究背景

编译器是一个将高级语言编写的程序转换成能在一台计算机上执行的等价目标代码或机器语言程序的软件系统^[9]。早期的计算机软件都是用汇编语言直接编写的，当人们发现为不同类型的处理器编写可重用软件的开销要明显高于编写编译器时，高级编程语言应运而生。20 世纪 50 年代末期，与机器无关的编

程语言被首次提出。随后，人们开发了几种实验性质的编译器。第一个编译器是由 Grace Hopper 于 1952 年为 A-0 系统编写的。1957 年由 John Backus 领导的 FORTRAN^[10] 则是第一个具备完整功能的编译器。1960 年，COBOL^[11] 作为一种具备跨平台移植能力的高级编程语言出现，此时的编译器无标准化编译流程，每个编译器均为特定语言与硬件定制。

1964 年 IBM 公司推出的 PL/I 语言编译器，首次实现了科学计算与商用数据处理两大领域的一体化支持，打破了此前编译器针对单一应用场景设计的局限，验证了编译器的通用性潜力。1972 年阿霍与乌尔曼提出的“词法分析-语法分析-语义分析-代码优化-代码生成”五阶段流程^[12]，如图 1-1 所示，成为全球编译器研发的标准框架；1975 年 UNIX 系统与 C 语言的结合，催生了可移植编译器的研发——1978 年贝尔实验室推出的 C 编译器^[13]，通过前后端分离的设计，首次实现同一前端解析 C 语言，不同后端适配不同硬件，为编译器的跨架构适配提供了范式。

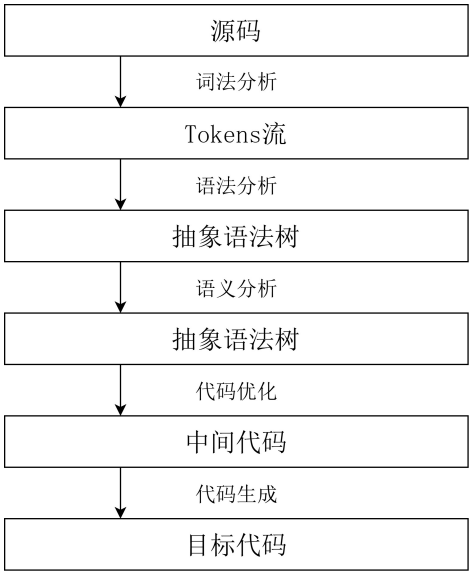


图 1-1 编译器五阶段流程图

1987 年理查德·斯托曼发起 GNU 项目^[14]，推出 GCC（GNU Compiler Collection，一个开源编译器工具集）编译器，通过开源模式吸引全球开发者参与，逐步支持 C、C++、Fortran、Java 等多语言，适配 x86、ARM 等主流架构，成为开源生态的核心基础设施；2000 年 Chris 在伊利诺伊大学创建了 LLVM^[15] 项目，其模块化、可重定向的设计理念为后续专用架构编译奠定了基础。

随着 GPU、DSP、AI 加速器等异构架构的兴起^[16]，编译器的核心目标从通用适配转向架构专属优化，追求极致性能 + 低功耗的平衡。2003 年 NVIDIA 推出 CUDA^[17] 平台及配套编译器，首次实现 C 语言到 GPU 指令的编译优化，通过单指令多线程（SIMT）指令映射策略^[18]，充分挖掘 GPU 的并行计算潜力；2009

年 LLVM 2.6 版本发布, 其模块化架构被广泛用于异构芯片编译。如苹果的 Clang 编译器、ARM 的商用编译器等, 均通过定制 LLVM 后端实现架构专属优化。

1.1.3 指令选择研究背景

指令选择是编译器后端的核心环节, 核心任务是将平台无关的中间表示转换为平台相关的机器指令, 其决策质量直接决定了生成代码的执行效率、资源占用及硬件适配性^[19], 是衔接编译器前端语义表达与后端硬件执行的关键桥梁。

早期编译器的目标是支持基础编程语言与简单的硬件架构, 其核心任务是将高级语言或中间表示映射为目标机器的可执行指令, 以保证程序的正确运行。该阶段的编译器设计侧重于功能的实现与代码生成的基本正确性, 对全局优化能力与编译过程本身的效率要求相对较低。在这样的需求背景下, 指令选择技术以局部指令选择为核心, 形成了宏拓展机制、树覆盖算法^[20]以及 DAG^[21]三类实现方案。其中, 基于 DAG 的指令选择成为主流方法。该方法将基本块内的指令序列表示为 DAG 结构, 通过预定义的树模式在图中进行子图匹配, 以寻找最优的指令组合。这种 DAG 图的优势在于能天然体现指令间的数据依赖关系, 便于实现如公共子表达式消除这样的局部优化, 不过其局限性也较为明显: DAG 图的构建与优化依赖局部子图分析, 难以感知函数级的全局依赖关系, 在 VLIW、宽位向量 DSP 等复杂架构中, 易因局部最优决策导致全局资源浪费。

随着多核心处理器、向量化扩展以及异构计算单元 (GPU、专用加速器) 等复杂硬件架构的普及, 传统基于局部基本块的指令选择方法难以充分适配其多样化的指令集特性。同时, 在高性能计算与嵌入式系统等场景中, 对代码执行效率的要求日益严苛, 局部优化所能达到的性能上限已逐渐无法满足需求, 亟需引入具备全局视角的指令选择策略。此外, 大型软件项目的编译时间长, 传统基于 DAG 等结构的指令选择方法因其较高的编译开销而成为效率瓶颈, 推动了对更轻量、更高效的指令映射方案的需求。

LLVM 作为开源编译系统的主流选择, 其生态需要一套能够统一多架构适配、兼顾编译速度与代码质量的指令选择框架。2019 年的 LLVM 全球开发者大会将 GlobalISel (全局指令选择) 列为核心主题, 正式推动其成为传统方案的替代者。相比于 SelectionDAGISel (基于 DAG 图的指令选择), GlobalISel 的优势如下:

- 更多的优化空间: 以整个函数为操作粒度进行操作, 保留完整的 IR 信息, 拥有更大的视野, 支持跨基本块的指令优化。
- 更低的编译开销: 去除 DAG 中间表示, 直接将 IR 转换为通用机器指令, 简化指令映射流程;

- 模块化与复用性：采用可配置的流水线架构，不同目标架构可复用基础流程，仅需定制架构相关规则，符合模块化的思想。

近年来，LLVM 社区持续推进 GlobalISel 的功能完善与架构适配，针对 X86、AArch64、RISC-V 以及 AMDGPU 等主流架构已完成适配。框架已从基础的指令映射阶段演进至深度优化阶段，陆续引入了基于代价模型的指令合并、全局寄存器组选择策略以及指令局部性优化等关键功能，从而逐步缩小了与传统指令选择方案在生成代码质量上的差距。

1.1.4 研究意义

在数字信号处理场景中，DSP 处理器凭借强大的并行处理能力及优异的能效表现，已经成为自动驾驶、智能传感和通信基站等对实时性与可靠性要求极高系统的核心计算单元。尽管当前 DSP 硬件在计算能力方面已取得显著进展，但编译技术的瓶颈却制约着其性能的充分释放。如何针对目标 DSP 架构的特性，在编译器层面进行深度适配与优化，以实现程序执行效率的最大化及代码存储密度的提升，已成为当前亟待解决的核心技术问题。

面向自研 DSP 架构的专用化设计需求及实时信号处理场景的高性能目标，本文围绕该核心技术难题开展系统性研究与工程实现：首先，针对传统基于 DAG 的指令选择方案在跨基本块优化能力、编译可扩展性等方面的局限，设计并实现面向 DSP 的 GlobalISel 框架，以函数为粒度进行指令映射与优化，克服传统方案的固有缺陷，为硬件潜力挖掘奠定适配基础；其次，为充分释放自研 DSP 架构的指令集优势，在 GlobalISel 框架基础上设计并实现分阶段指令合并与优化策略，在保障编译效率的同时，进一步提升生成代码的执行性能与资源使用效率；最后，为解决前两项技术创新落地过程中存在的复杂场景覆盖不足以及性能变化难以量化的验证难题，构建一套完整的测试评估平台，通过集成随机测试生成、多维度性能采集和自动化分析报告等功能，对编译生成代码的运行周期、存储占用、指令打包效率等关键指标进行系统化量化评估，为框架适配效果与优化策略有效性提供可靠的数据支撑。

本研究通过引入全局指令选择框架、实施针对性的架构定制优化以及建立系统化的评估体系，为自研 DSP 处理器构建了一套高效、稳定且可扩展的编译工具链，同时也为高实时性、高能效的嵌入式信号处理应用开发提供了坚实的技术基础，具有明确的工程应用价值与广泛的行业推广前景。

1.2 国内外研究现状

1.2.1 编译器研究现状

编译器作为连接软件与硬件的核心枢纽，用于将源代码翻译成机器可以执行的目标代码。从技术演进脉络来看，编译器架构已从早期封闭的定制化设计，逐步转向模块化、可扩展的开源框架，其中 LLVM 与 GCC 成为当前全球编译器研发的两大核心基石。

GCC 作为第一个开源编译器，打破了传统的编译器技术格局，自 1987 年诞生以来，历经三十余年迭代，从单一 C 语言编译器成长为支持多语言、多架构的跨平台编译工具，深刻影响了 UNIX-like 系统、Linux 发行版及嵌入式领域的发展^[22]。GCC 沿用了编译器领域经典的前端—优化—后端三段式架构设计^[23]，在长期迭代中整合了常量传播、死代码消除、循环展开等一系列成熟的通用编译优化技术，能够满足多数场景下的基础编译需求。然而，受限于早期架构设计的历史惯性，该架构存在难以规避的固有缺陷：一方面，核心代码模块耦合度较高，编译流程的逻辑封装不够清晰，导致代码可读性欠佳，新开发者介入二次开发或功能调试时需花费大量时间梳理逻辑关联；另一方面，架构的拓展性不足，新增语言前端、适配新硬件指令集或集成定制化优化策略时，往往需要对现有核心代码进行大幅修改，开发成本高且易引入兼容性问题；此外，不同体系架构的适配逻辑缺乏统一的抽象层支撑，导致跨架构编译时的兼容性表现不佳，难以快速适配嵌入式、异构计算等场景下的小众架构或定制化芯片，这一问题在硬件架构日趋多元化的当下尤为突出。

2000 年 LLVM 的提出影响了编译器的发展轨迹，LLVM 突破了传统编译器的紧耦合三段式架构，将编译流程拆解为独立的功能模块，各模块通过标准化接口通信，实现高度解耦，其架构如图 1-2 所示。LLVM 采用与目标平台、源语言无关的中间表示 IR (Intermediate Representation, 中间表示, LLVM 在将源代码翻译成目标机器码的过程中生成的中间语言)，作为连接前端与后端的桥梁。IR 基于 SSA (Static Single Assignment, 静态单赋值, 是编译器中间表示的一种设计形式, 核心规则是每个变量仅被赋值一次, 且每次使用变量前都有且仅有一个定义)^[24]形式设计, 兼具高层语言的抽象性与底层指令的精准性, 能够完整保留代码的语义信息与优化潜力。除此之外, LLVM 还将编译优化逻辑封装为独立的 Pass 单元, 形成可配置、可组合的优化流水线^[25]。

依托模块化的整体架构设计、跨平台统一的中间表示以及可组合的优化机制, LLVM 已发展成为跨平台编译器实现与高性能代码生成领域的重要基础设施。其良好的可扩展性与架构适配能力, 使其被多种主流及新兴编程语言的编译器所采用, 例如 Swift、TinyGo 和 Rust 等均在实现层面不同程度地依赖 LLVM 后

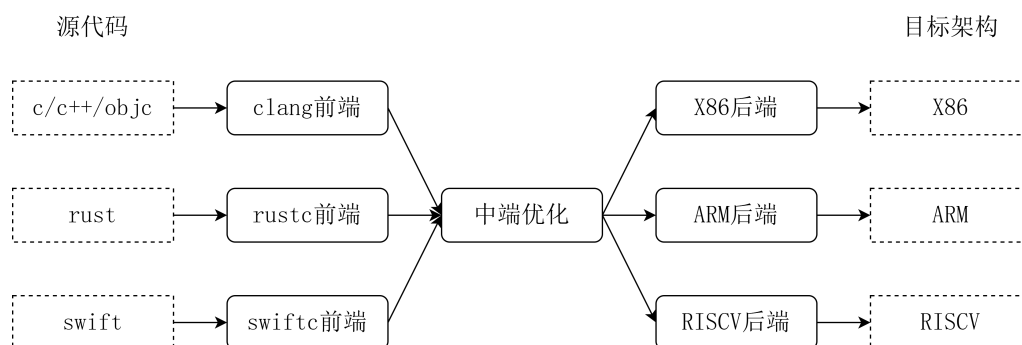


图 1-2 LLVM 架构图

端。除此之外，在国际上对 LLVM 的相关研究还有许多，如 Joseph N. Huber^[26] 等人针对 ARM A64FX 处理器（面向高性能计算的可扩展向量架构），提出基于 LLVM 工具链的 SIMD（Single Instruction Multiple Data，单指令多数据流）代码优化方法论；S Fang 等人^[27] 针对 LLVM、GCC 等主流编译器自动向量化能力不足的问题，提出一套融合专用 IR 扩展 + 灵活向量化流水线的优化方案；Kavon Farvardin 等人^[28] 对 SML/NJ 编译器^[29] 开发了 LLVM 后端，解决了 SML 的模块系统、多态类型与 IR 的语义映射问题；Paschalis Mpeis 等人^[30] 为 Android ART 开发了 LLVM 后端，解决了默认后端因优化粒度有限，难以满足交互式应用的性能需求问题。验证了 LLVM 在移动虚拟机编译器中的优化潜力；Lammich 等人^[31] 提出了一种基于 LLVM 的逐步细化方法，将并行算法的验证推进至 LLVM 代码级别，通过结合 LLVM IR 和代码生成步骤，提高程序的性能；Walter Rocchia 等人^[32] 提出利用 LLVM 编译器后端为 RISC-V 处理器设计新的 AI 指令集扩展，以提升边缘设备上神经网络的执行效率。Juan Carlos 等人^[33] 提出一种基于遗传算法和 LLVM 优化的源代码混淆技术，旨在解决云计算等开放执行环境下的软件知识产权保护与反逆向工程问题；John Regehr 等人^[34] 提出一款高吞吐量、形式化方法辅助的 LLVM 专用变异模糊测试工具，旨在高效发现 LLVM 编译器优化阶段的潜在缺陷。

随着国产芯片产业的快速发展，LLVM 由于其开源、模块化程度高等特点，逐渐被引入到多种自主研发硬件平台的工具链建设中。国内围绕 LLVM 的研究与实践，多聚焦于挖掘国产处理器架构特性，通过深度适配与优化，提升编译器对硬件算力的转化效率。刘玉等人^[35] 针对魂芯数字信号处理器的特殊架构，基于 LLVM 构建优化编译器，解决前代编译器代码质量低、硬件特性支持不足的问题。沈莉等人^[36] 针对我国自主研发的神威新一代超级计算机解决异构系统可编程性差、硬件算力释放不足的问题，基于 LLVM 定制开发优化编译器；吴忱等人^[37] 则提出了基于启发式搜索的优化序列选择算法。

1.2.2 指令选择研究现状

在 LLVM 编译器生态中，有三种指令选择的实现方式：面向快速编译的 FastISel（快速指令选择）、基于 DAG 图的 SelectionDAGISel 以及 GlobalISel。表1-1给出了三者在设计目标、IR 形态与作用域等方面的差异。

表 1-1 LLVM 不同指令选择框架对比

方案	设计目标	中间表示	作用域	编译速度	代码质量
FastISel	最大化编译速度	无	基本块	快	低
SelectionDAGISel	优化代码质量	DAG	基本块	慢	高
GlobalISel	平衡编译速度和代码质量	GMIR	函数	中	中

FastISel 通常在 O0 优化等级下启用，主要面向的是需要快速编译的场景，其核心宗旨是通过牺牲生成代码的质量来提升编译速度。与传统指令选择依赖多阶段中间表示处理不同，FastISel 在遍历 LLVM IR 过程中直接对指令进行处理。FastISel 通过对 IR 表达式结构进行递归访问，并结合目标架构中预定义的简化指令模式，完成指令的即时生成，从而避免了 DAG 构建及全局分析等开销较大的步骤。

SelectionDAGISel 是 LLVM 中应用最广泛的指令选择技术，自 LLVM 早期版本起就一直作为框架的核心实现。在 x86、ARM 以及 RISC-V 等主流架构中，尤其是在中高优化等级下仍是默认的指令选择方案。SelectionDAGISel 的核心思想是将中间表示转换为 SelectionDAG，并在有向无环图上进行模式匹配，以克服传统树匹配方法在表达共享子表达式方面的局限性。从研究角度上来看，这种以 SSA 语义为基础、在图结构上完成匹配的模型被认为是对传统局部树结构指令选择的重要补充和发展。Ebner 等人^[38]对指令选择中的匹配模型进行了理论扩展，提出了一种基于 SSA 的 DAG 匹配方法，将匹配过程从局部的树结构提升为全局的 DAG 结构，不仅增强了模式匹配的表达能力，也提高了将机器无关中间表示映射为机器相关指令时的效率与准确性。其更早的工作也曾将指令选择形式化为 SSA 图上的图文法解析和组合优化问题，探索全函数范围上的更优覆盖与代价最小化^[39]。这些研究为更大范围、更强约束的指令选择提供了理论基础，但在工程上也带来求解复杂度与编译时间的挑战。

GlobalISel 最早是在 LLVM 开发者会议上被提出，其设计初衷是在于解决 SelectionDAGISel 在编译性能、可扩展性与复用性方面的痛点：SelectionDAGISel 引入了专用中间表示并伴随较高的构建和合法化开销，而不同后端在 DAG 层的大量定制也导致维护成本上升^[40]。除此之外，随着指令选择规则规模与复杂度增长，研究工作开始关注后端选择器相关的测试覆盖与缺陷发现，例如针对

GlobalISel 匹配表/选择路径的专门化测试与模糊测试。这些方向共同推动了指令选择从局部模式向可扩展、可验证、面向多目标优化的方向演进^[41]。

2015 年, 苹果公司的 Quentin Colombet 系统剖析了传统 SelectionDAGISel 框架的若干根本性局限, 包括编译速度缓慢、仅支持基本块级别的局部作用域、架构设计过于单一化等问题。他指出, 尽管开发者通过增加 Target hooks 和优化 Pass 等方式试图规避这些局限, 但由此引发的启发式算法精度不足、需提前预测指令选择器执行行为等新问题, 进一步加剧了框架的复杂性。在此基础上, Colombet 提出推出新一代指令选择框架 GlobalISel 的构想, 该框架旨在从根源上解决上述问题, 同时为提升代码生成质量创造新的可能^[40]。次年, Colombet 等人分享了 GlobalISel 在设计与实现层面取得的阶段性成果, 并明确了后续仍需重点研发的技术方向^[42], 为框架的持续演进奠定了理论基础。

在框架初步成型后, 研究者着手开展功能完善与质量保障工作。2017 年, 苹果公司的 Justin Bogner 介绍了其团队在指令选择测试中应用模糊测试与输入生成技术的实验过程及核心成果, 深入探讨了在寻找高价值测试输入过程中的技术权衡, 以及验证生成代码有效性的核心方案^[43], 为 GlobalISel 的可靠性提供了重要保障。2024 年, Pierre Houtryve 提出为 GlobalISel 组合器基础设施新增输入/输出 MIR 模式支持, 该功能包含类 PatFrag 系统与类型推断机制, 使开发者能够直接在 TableGen 中编写大量组合器规则^[47], 显著提升了框架的可编程性与可扩展性。

随着基础功能的逐步完善, 研究重心逐渐向优化能力转移。2019 年, Daniel Sanders 指出, 早期 GlobalISel 的开发主要集中在对各类目标架构的基础支持上, 优化相关的工作投入相对有限^[44]。近期, 研发方向已转向优化能力的提升, 目标是使其优化水平达到能够全面替代 SelectionDAGISel 的程度。他详细讲解了 Combiner 的整体设计架构、支撑其运行的核心模块、与 GlobalISel 其他组件的协同运作方式, 以及该组件的测试和调试方法。2021 年, Huang Zhufeng 等人提出基于代价模型的指令合并优化、全局寄存器组选择优化以及指令局部性优化等方法^[45], 有效解决了传统指令选择的全局优化能力缺失、编译开销大等问题, 在申威平台实现了编译速度与代码质量的平衡。2024 年, Tobias Stadler 提出直接生成 GMIR 的创新方案, 跳过代码生成流水线的首个转换阶段, 在特定应用场景中使编译速度提升了约 20%^[48]。同年, Madhur Amilkanthwar 针对 AArch64 高级 SIMD 平台, 通过在 GlobalISel 各编译阶段引入补丁, 消除了其向 SelectionDAGISel 的回退现象, 并对生成的代码进行优化, 显著缩小了两者在代码质量上的差距^[50]。

架构适配是 GlobalISel 推广应用的关键环节。2022 年, Kai Nacke 等人分享了其为 PowerPC 目标架构实现 GlobalISel 框架的初步实践经验^[46], 详细阐述了

在适配过程中面临的架构特性适配挑战、与传统 SelectionDAGISel 后端的功能衔接方案，以及达成的阶段性成果（如核心指令集的无回退匹配、基础基准测试的编译通过率提升等），为其他架构的 GlobalISel 适配工作提供了可复用的实践参考。2024 年，Jiahan Xie 提出面向可扩展向量的突破性 GlobalISel 实现方案，以 RISC-V 向量扩展为目标场景，深入探讨了支持可扩展向量算术逻辑单元及加载/存储指令过程中面临的核心挑战与创新解决方案^[49]，进一步拓展了框架的应用边界。2025 年，Nvidia 的 Neil Hickey 指出，GlobalISel 在所有后端的推广应用仍受到指令选择场景覆盖不完全的限制，导致其在部分场景下需回退至 SelectionDAGISel^[51]。为解决并监控这些局限性，Neil 等人开发了一套专用的持续集成系统，通过每日自动构建最新版本 LLVM、编译基准测试程序并记录回退事件，为社区定位问题来源提供了有力支持。

综上所述，GlobalISel 研究经历了从框架设计、功能完善到优化增强、架构适配的演进路径，各方向相互促进、协同发展，推动该框架逐步走向成熟并有望最终取代传统的 SelectionDAGISel。

1.3 论文研究内容和创新点

1.3.1 研究内容

随着数字信号处理器向 VLIW 与 SIMD 架构的持续演进，编译器后端指令选择阶段面临着新的技术挑战。传统基于 DAG 的指令选择方案受限于基本块边界，难以发掘跨基本块的指令级并行机会；其复杂的中间表示转换过程也导致了较高的编译开销与维护成本。LLVM 社区近年来主推的全局指令选择框架 GlobalISel，虽然在模块化设计和编译效率上具有优势，但在面向 DSP 这类专用架构时，其通用化的指令映射策略往往引入冗余指令，且对硬件特性的挖掘深度不足，导致生成代码质量与成熟方案仍存在差距。与此同时，现有编译器测试体系对后端优化行为的覆盖能力有限，缺乏有效的性能退化分析与量化评估手段。

为系统性解决上述核心难题，本文围绕“框架适配 - 性能优化 - 验证支撑”的技术路径，聚焦以下三方面研究：

1.GlobalISel 框架在 DSP 架构上的适配与实现方法研究

针对传统指令选择方案跨基本块优化能力弱、扩展性差的问题，本文系统分析 GlobalISel 的核心流程及其模块化设计思想，结合 VLIW DSP 架构的硬件特性（异构寄存器文件、专用运算单元及自主指令集）来研究从通用中间表示到目标机器指令的映射方法。重点解决以下关键技术问题：如何通过定制调用约定实现架构相关的参数传递与返回处理；如何构建合法化规则集以确保指令

语义落入硬件能力范围；如何在多组异构寄存器文件之间建立代价驱动的寄存器组选择模型；如何融合 TableGen 自动匹配与手动匹配机制完成复杂指令的精确映射。通过上述研究，旨在为 DSP 架构构建一个兼具编译效率与代码质量的 GlobalISel 基础框架。

2. 面向 GlobalISel 流程的分阶段指令选择优化技术研究

针对 GlobalISel 通用语义转换过程中引入的指令冗余及硬件适配不充分问题，本文深入分析指令冗余的产生机理，并提出与 GlobalISel 流程深度耦合的分阶段优化策略。在合法化前阶段，聚焦通用语义层面的冗余消除，研究内存操作内联展开、冗余截断与比较指令简化、扩展指令下推规范化等优化方法，减少通用转换带来的指令开销。在合法化后阶段，面向 DSP 指令集特性开展架构专属优化，研究基于立即数位型特征的装载指令优化、基于移位与加减运算的常量乘法强度削减等策略，旨在充分挖掘硬件专用运算单元的潜力，提升生成代码的执行效率与存储密度。

3. 面向编译器后端优化的性能评估体系研究

针对现有测试体系在输入覆盖与结果分析深度上的不足，本文设计并实现一套面向 DSP 编译器的性能测试与评估平台。研究随机测试生成技术在编译器验证中的应用方法，通过对 YARPGen 随机测试生成器进行定制化扩展，引入 DSP 专用内建函数与调试机制，构建覆盖指令选择及后端优化关键场景的随机测试输入集合。同时，研究编译器性能指标的自动化采集与分析方法，建立支持代码体积、执行周期、编译时间等多维度指标的量化评估体系，并通过跨版本对比与可视化呈现，为编译器优化策略的效果验证与迭代分析提供系统化支撑。

1.3.2 创新点凝练

针对传统指令选择方案在 DSP 架构下生成代码性能不足及测试体系不完善等核心问题，本文通过系统性的研究与工程实现，形成了兼具创新性与实用性的技术方案，主要创新点凝练如下：

1. 提出了一种基于 LLVM GlobalISel 的 VLIW DSP 后端模块化适配方法

针对传统 SelectionDAGISel 跨基本块优化能力弱、架构适配侵入性强的痛点，本文提出了一套基于 GlobalISel 框架的模块化适配方法。该方法通过扩展 TableGen 指令描述模板、定制调用约定与合法化规则、构建分层寄存器组映射模型等技术手段，将架构相关的实现逻辑封装于框架的标准扩展接口之内，在

不修改 GlobalISel 核心代码的前提下，实现了从通用中间表示到目标机器指令的端到端映射。该方法不仅解决了通用框架在 DSP 平台上的适配难题，其模块化的设计思路也为其他非主流架构快速接入 LLVM 生态提供了可复用的实践参考。

2. 设计了一套与合法化流程深度耦合的分阶段指令选择优化体系

针对 GlobalISel 通用语义转换导致的指令冗余问题，本文深入分析了优化机会在指令合法化前后的分布规律，并据此设计了“合法化前通用化简”与“合法化后架构专属优化”相协同的分阶段优化体系。在合法化前阶段，通过内存操作内联、冗余扩展消除等策略削减通用语义转换引入的开销；在合法化后阶段，通过立即数装载适配、常量乘法强度削减等优化挖掘 DSP 指令集的专属优化潜力。该体系的核心创新在于将优化策略的生效时机与 GlobalISel 的合法化流程深度绑定，在保持编译器后端模块化与可维护性的同时，实现了对生成代码性能的渐进式提升。

3. 构建了一套集成随机测试与多维性能分析的自动化评估平台

针对现有测试体系对后端优化行为覆盖不足、性能分析手段薄弱的现状，本文设计并实现了一套面向编译器后端的持续性能评估平台。该平台的创新性体现在：其一，通过对 YARPGen 随机测试生成器进行架构定制化扩展，引入了 DSP 专属内建函数与调试机制，实现了对指令选择等后端优化路径的高覆盖测试；其二，建立了支持代码体积、执行周期、编译时间等多维指标的自动化采集与分析流程，并通过与持续集成流水线的深度集成，实现了优化效果的持续追踪与性能退化的早期预警。该平台为编译器优化策略的量化评估与迭代验证提供了完整的技术支撑。

1.4 论文组织结构

本文一共分为六个章节，围绕全局指令选择的实现与优化方法以及性能评估平台的设计与实现展开研究，论文整体结构安排如下。

第一章为绪论部分，介绍了论文的研究背景与意义，重点分析了 DSP 架构与编译器技术的发展现状，梳理了指令选择技术，尤其是全局指令选择相关研究的国内外研究现状与存在的问题，在此基础上明确了本文的研究内容、技术路线和主要创新点。

第二章为全局指令选择框架的实现部分。本章先对传统指令选择方案及其局限性进行了分析，并给出全局指令选择的基本概念和设计思路，之后分节详细阐述了面向 DSP 的全局指令选择的 GMIR 的生成、指令合法化、寄存器组选

择以及机器指令选择这四个核心阶段的设计与实现。

第三章为全局指令选择的优化策略的实现部分。本章先结合 DSP 架构特点从理论角度分析了全局指令选择优化的必要性和可行性，之后针对内存操作指令、乘法指令等典型应用场景，提出并实现了多种优化策略，提升了生成代码的执行效率和质量。

第四章为测试评估平台的设计与实现部分。本章根据现有 DSP 编译器测试流程中存在的问题，确定了测试体系优化的需求与技术路径，之后从系统架构层面给出了两个子系统的整体设计方案，并在此基础上实现。

第五章为实验和展示部分。实验部分从编译时间、代码体积和执行周期等多个维度来分别对全局指令选择实现的正确性及优化的有效性进行了验证，展示部分则通过运行截图来直观地展示测试评估平台的实现情况。

第六章为总结与展望部分。本章对全文的研究工作进行了回顾，总结主要成果和技术创新，并对存在的不足与未来改进方向进行讨论。

第 2 章 面向 DSP 的全局指令选择实现

编译器后端指令选择阶段的工作是将 LLVM IR 转换为目标架构 MI (Machine Instruction, 机器指令)。这一阶段所要解决的问题是在保持程序语义不变的基础上生成适用于目标架构的指令序列。这个过程对生成的目标代码质量、资源利用率以及编译器对各种复杂硬件架构的兼容性有很大影响,是连接前后端的重要纽带。

随着处理器架构的不断发展以及 LLVM 编译器基础设施的不断成熟,指令选择的实现方式也在不断变化。出于不同应用场景及性能要求的考虑,LLVM 演变出不同的指令选择实现方案,先后引入了快速指令选择、基于有向无环图的指令选择以及全局指令选择这 3 个指令选择实现技术路线。这 3 种指令选择方案在设计思路、适用性以及代码生成质量特点上各有侧重。

LLVM 编译器主要依赖基于有向无环图的指令选择方案来生成高质量代码,但其复杂的图变换流程与以基本块为限的优化范围,导致编译时间长、内存占用高,且难以维护。与此同时,面向调试的快速指令选择方案虽提升了编译速度,但其简化的匹配逻辑无法满足 DSP 应用对代码密度与指令级并行的严苛要求。这两种方案在适配指令集异构性强、硬件约束复杂的现代 DSP 处理器时,均显现出固有的局限性。

为应对上述挑战,本章结合 DSP 的硬件与指令集特点,设计并实现了面向 DSP 的全局指令选择框架,旨在为 DSP 架构构建一个兼具编译效率、代码质量与架构适配灵活性的新型指令选择流程。GlobalISel 采用一种更贴近机器指令的通用中间表示,并以函数为粒度进行全局性的指令选择与优化,这为充分挖掘 DSP 硬件特性、减少冗余指令和数据移动开销提供了新的可能。此外,本框架的设计思路与实现路径,也为其他专用处理器的 GlobalISel 适配提供了可复用的工程参考。

本章将系统阐述 GlobalISel 在 DSP 目标架构上的完整实现路径。首先,将对传统指令选择方案的原理与局限进行对比分析,以明晰技术演进脉络与选型依据。随后,将依据 GlobalISel 的核心流程,分阶段深入剖析其关键实现:GMIR 生成阶段负责完成语义转换与初步架构适配;指令合法化阶段确保所有操作均

落在硬件能力范围内；寄存器组选择阶段负责优化寄存器文件映射以降低数据搬移成本；机器指令选择阶段最终完成到目标指令的精确映射。

2.1 全局指令选择框架概述

2.1.1 传统指令选择方案及其局限性

在 LLVM 的早期发展阶段，指令选择的目标是保证 LLVM IR 能够正确、稳定地映射为目标架构支持的机器指令，并在开启优化选项时能生成质量较高的代码。为此 LLVM 采用了基于 DAG 的指令选择方案。该方案是 LLVM 长期以来的主流指令选择方案，也是高优化等级下的默认指令选择方案。该方案支持复杂架构指令集，能够处理 X86、ARM 以及 RISC-V 等各种架构复杂的指令选择需求。

基于 DAG 的指令选择方案通过将基本块内的 LLVM IR 转换为 SelectionDAG，对 SelectionDAG 进行指令合法化、节点合并以及指令匹配替换等操作，从而实现中间表示转成对应机器指令的映射过程。方案的整体处理流程如图 2-1 所示。

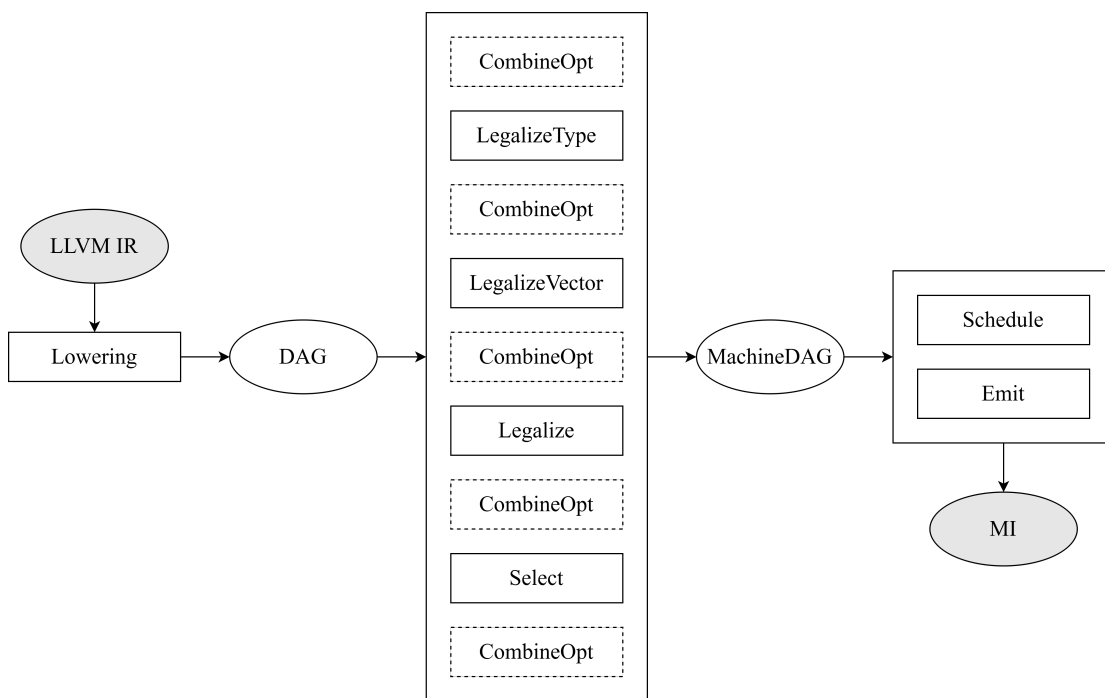


图 2-1 SelectionDAGISel 流程图

SelectionDAGISel 的设计思路是将 LLVM IR 转换成 DAG 来表示指令间的数据流依赖关系，然后再通过模式匹配和树覆盖的方法来将 DAG 内的节点转换为目标架构的机器指令。SelectionDAGISel 的执行过程分为三个阶段：第一阶段，框架将 LLVM IR 指令序列转为 SelectionDAG。在 DAG 图中每个节点表示一种操作，边表示数据依赖，能直观地看到指令之间的数据流依赖关系，不再需要对指

令进行线性的扫描；第二阶段，为了简化后续的匹配流程，框架对 SelectionDAG 进行标准化操作及优化；第三阶段，框架采用基于 TableGen 自动生成的模式匹配器，对优化后的 SelectionDAG 进行拓扑排序，通过自底向上遍历节点来匹配目标架构在 .td 文件中定义的 DAG 模式，之后将匹配成功的节点替换为对应的机器指令，最终生成目标架构的机器代码。图 2-2 展示了一个简单自增运算函数在指令选择阶段生成、调度之前的部分 SelectionDAG 图。

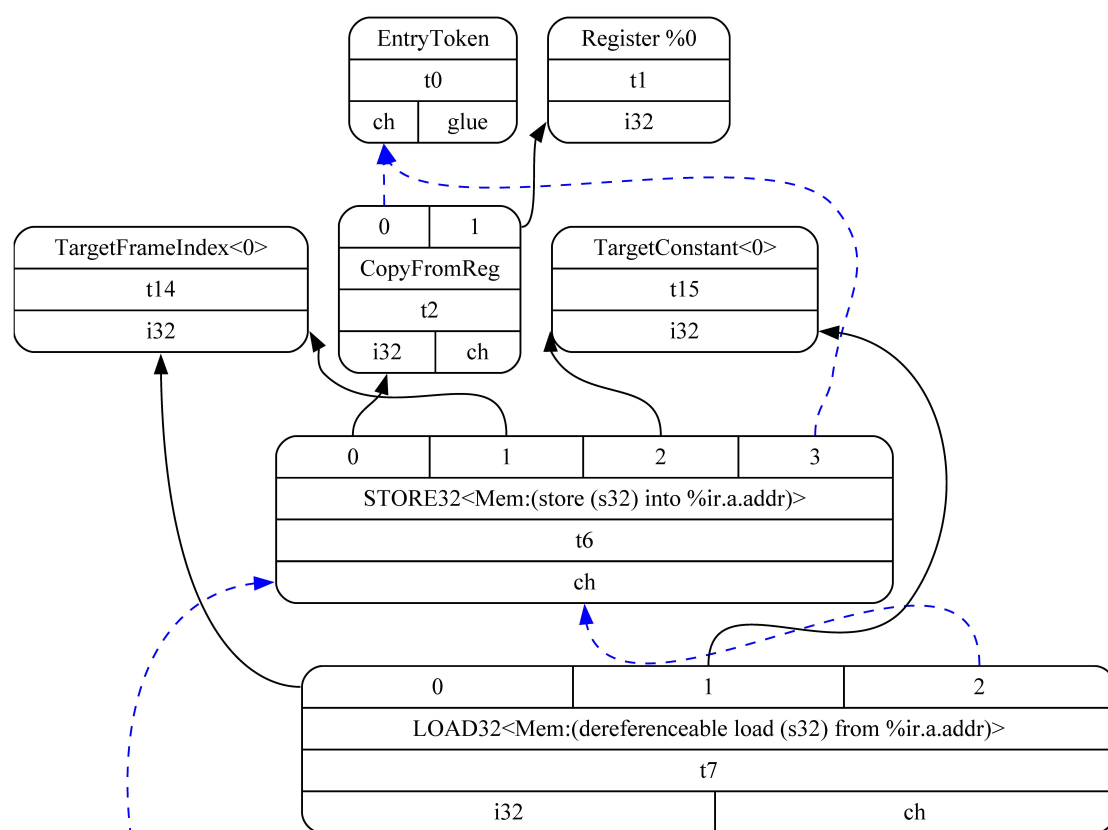


图 2-2 自增运算函数部分 SelectionDAG 图示例

利用 SelectionDAG 结构，SelectionDAGISel 可以实现对全局数据流进行详细分析，能够对指令进行比较复杂的优化、合成，并能提高生成代码的质量。此外，开发者仅需在描述文件中定义指令模式就可以自动生成匹配代码，大大降低新架构的适应开发成本。但是这种指令选择方案的设计比较复杂，这是因为 SelectionDAG 的构造、优化、拓扑排序等都需要比较复杂的逻辑，并且 DAG 与 LLVM IR 及机器指令之间的转换也需要额外的转化层，增加了设计的复杂度。

而随着 LLVM 编译器生态的发展，LLVM 因其模块化的特征、良好的跨平台兼容性等特点，在各种大型工业软件开发、复杂代码调试和维护等场景中得到了广泛应用。在这样的背景下，低优化等级下快速地编译代码成为了关键诉求。为了满足项目调试构建过程快速编译的这一需求，LLVM 在原来的 SelectionDAGISel 框架之外引入了 FastISel 框架。FastISel 是 LLVM 为平衡编译速度与代码质量设

计的轻量级指令选择方案，FastISel 的核心定位是优先考虑编译速度，同时兼顾基础指令选择需求，其主要用于 O0 优化等级下的代码生成。

从设计原理来看，快速指令选择采用线性扫描 + 模式匹配的策略，以单个指令为处理单位，不构建复杂的中间表示结构。其核心流程为：遍历 LLVM IR 指令序列，对每条指令直接匹配目标架构的简单指令模式，匹配成功则直接生成机器指令，匹配失败则回退到基于 DAG 的指令选择。方案的整体流程如图2-3所示。

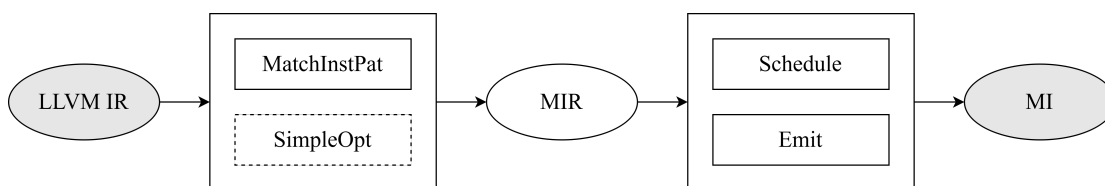


图 2-3 FastISel 流程图

为了提高编译速度，快速指令选择采用手写 C++ 代码的方式来实现匹配规则。这种方式能够有效地避免 TableGen 工具的解析、代码产生所带来的额外成本，也避免了指令间的依赖分析以及复杂的优化逻辑。使用快速指令选择的编译效率非常高，在调试模式下能大幅度减少编译的时间，但 FastISel 也有局限性。因为只有简单指令模式的匹配，无法匹配比较复杂的指令模式（如向量运算、指令融合等等），导致生成的代码质量较低。

除此之外，FastISel 缺乏全局视角，无法做跨指令的数据流和控制流优化，在性能敏感的场合无法满足性能要求。所以快速指令选择只能作为一种调试的默认模式，仅适合用来调试和快速编译，在对性能敏感的优化阶段仍然需要更强的指令选择机制。

近些年来，编译器的应用领域不断拓展，这些传统指令选择方案无论是在编译速度，还是在架构独立性上都难以适应新的应用需求。具体而言，SelectionDAGISel 的复杂中间表示及其相应优化机制增加了编译流程的复杂性和维护的负担，以及由于 SelectionDAGISel 的局部作用域的特性，SelectionDAGISel 在处理基本块间复杂优化等复杂情况下存在着明显的不足。这些问题推动了新一代指令选择机制的产生和发展。

2.1.2 全局指令选择原理及框架设计

全局指令选择是 LLVM 为突破基于 DAG 的指令选择框架的局限性而推出的全新指令选择方案。全局指令选择的核心目标在于平衡编译速度、代码质量以及架构适配的灵活性，并且通过统一的流程实现从 LLVM IR 到机器指令的转换。

从设计理念来看，全局指令选择摒弃了基于 DAG 的专用中间表示，转而使用 GMIR 作为其核心载体。GMIR 本质上是对 LLVM IR 的轻量级扩展，在保留了 LLVM IR 的语义特征的同时引入了目标架构的基础信息，这使得在指令选择的过程无需在多种中间表示间进行频繁转换，从而简化了整体流程。

GlobalISel 的核心优势是全局性和灵活性。全局性体现在 GlobalISel 是以函数为粒度的，结合全局的控制流和数据流，用全局视角进行指令选择，所以 GlobalISel 可以对一个基本块内部的指令序列进行优化；而灵活性是指 GlobalISel 具备手动和自动两种模式进行匹配，在自动模式下除了可以根据 TableGen 进行匹配外，还可以重用基于 DAG 的指令选择的那部分对指令集的描述信息，这样前期的设计工作量会大大降低。

全局指令选择的执行流程分为两个核心阶段：第一阶段为 GMIR 预处理，包括将 LLVM IR 转换为原始 GMIR、指令合法化、寄存器类型选择，最终生成附着目标架构信息的合法 GMIR；第二阶段为机器指令选择，以函数为粒度，采用基于树覆盖的指令选择算法，通过自动匹配模块和手动匹配模块，将 GMIR 指令树映射为 MIR，同时将通用虚拟寄存器限制到具体的寄存器类中，并传递 COPY 指令至后续的寄存器分配阶段。

全局指令选择将两个阶段涉及的每个功能都拆分成独立的 Pass，这样既能使整体的架构更加清晰，也能够方便利用 LLVM Pass 的相关基础设施进行代码维护和问题分析定位。LLVM 实现中将全局指令选择所必需的基本功能划分为 4 个 Pass，其中第一阶段包含 3 个 Pass，而第二阶段为 1 个 Pass。还有窥孔类的优化也会以独立 Pass 进行实现，并可以放置在 4 个基础 Pass 之间的任意位置，不同的架构可以根据需要配置一到多个这种优化 Pass。该流程如图 2-4 所示。

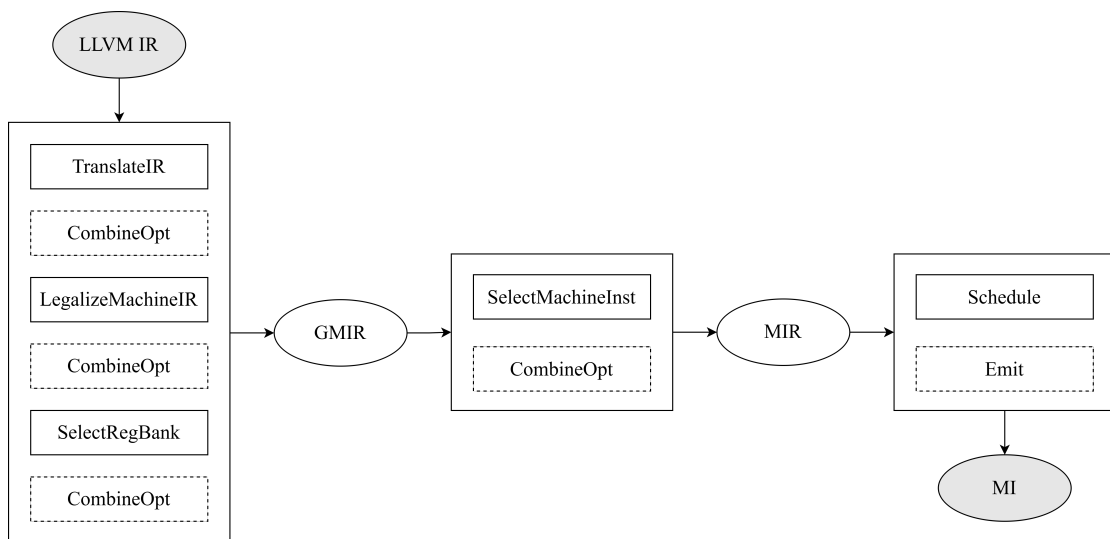


图 2-4 GlobalISel 流程图

相比于前两种指令选择方案，GlobalISel 实现了在编译效率、代码质量和架

构造适配性之间的多维度平衡。从编译速度的角度来看，GlobalISel 摒弃了 SelectionDAG 中复杂的处理流程，编译效率接近快速指令选择；从代码质量的角度来看，凭借其全局视角和复杂模式的匹配能力，GlobalISel 能够生成与基于 DAG 的指令选择相当的高质量代码；从架构适配性的角度来看，GlobalISel 的模块化设计使其能更方便地扩展和维护，由于 GlobalISel 采用通用的 GMIR 表示和灵活的模式匹配方式，使其更容易地适配新架构，尤其是包含特殊寄存器组或独特指令集的架构。

2.2 GMIR 生成

GMIR 的生成是 GlobalISel 第一阶段的第一个 Pass，也称为 IRTranslator。IRTranslator 阶段的任务是完成 LLVM IR 到 GMIR 的语义下沉与架构适配，核心是要完成寄存器处理、栈管理以及调用约定处理等操作。IRTranslator 的实现以目标平台的 ABI（Application Binary Interface，应用二进制接口）约定、指令集特性及数据类型支持为核心依据，通过模块化的调用处理机制，确保函数调用、参数传递、返回值处理的正确性与高效性。

2.2.1 通用机器表示

GMIR 是 GlobalISel 的中间层表示，在 LLVM IR 与目标机器强相关的 MIR 之间起到承上启下的作用。GMIR 的设计目标之一就是在保留通用机器语义的同时延迟引入具体架构细节，这样的好处是可以为跨平台指令选择、合法化以及后继的优化提供统一而稳定的基础。通过这一设计，GlobalISel 能够在更大作用域内进行分析与决策，避免过早绑定目标架构而减少优化空间。GMIR 由四个要素组成：

1. 通用虚拟寄存器（Generic Virtual Register）：不绑定具体物理寄存器，由 MachineRegisterInfo 统一管理生命周期，是 GMIR 中数据传递的核心载体，如函数参数、运算结果均存储于虚拟寄存器。
2. 通用指令集（Generic Instructions）：覆盖算术运算、逻辑运算、内存访问、分支跳转、函数调用与返回等基础操作，指令格式标准化，避免架构专属指令的碎片化。
3. LLT（Low-Level Type，低级别类型）：基于位宽和类型类别定义（如标量、指针、向量），摒弃了 LLVM IR 的高级类型（如结构体、类），更贴近硬件数据处理逻辑。

4. 架构约束元数据：通过 MachineMemOperand（内存访问属性）、RegMask（寄存器掩码）、CCValAssign（调用约定分配信息）等属性来系统化地表示目标架构的各种约束。

在 GlobalISel 整体流程内，GMIR 存在于指令选择前后的多个关键阶段。IR-Translator 以函数为单位去遍历 LLVM IR，把它直接映射成 GMIR 的通用指令和虚拟寄存器表示。在这一阶段，编译器可结合目标架构展开必要的特化处理，比如说 DSP 架构会对 64 位浮点运算进行拆分，而后进入到合法化阶段，框架要对 GMIR 里目标架构不直接支持的指令及参数类型修正，保证其完全在硬件能力范围之内。在寄存器组选择阶段，框架把通用虚拟寄存器映射到目标架构定义的寄存器组当中，为后续的物理寄存器分配奠定基础。在指令选择阶段，框架会把 GMIR 的通用指令替换成目标架构的机器指令，最终生成与目标硬件紧密绑定的 MIR 表示。为进一步明确 GMIR 在 LLVM 编译流程中的抽象层级及其过渡性定位，表 2-1 对 LLVM IR、GMIR 与 MIR 在抽象层级、核心元素及架构相关性等方面进行了对比。

表 2-1 LLVM 不同中间表示层次对比

类型	抽象层级	核心元素	架构相关性	核心作用
IR	高级语言无关抽象 (函数、模块级)	函数、基本块、 Value、LLVM IR 指令	完全无关	跨平台程序分析与 中端优化
GMIR	通用机器语义抽象 (指令、寄存器级)	通用虚拟寄存器、通用指令、LLT 类型	部分相关	衔接 LLVM IR 与架构相关 MIR，统一指令选择流程
MIR	目标架构专属抽象 (硬件指令、物理寄存器级)	物理寄存器、目标架构指令、硬件约束	完全相关（绑定具体硬件）	目标架构指令调度与寄存器分配

与 SelectionDAGISel 中 LLVM IR 到 SelectionDAG 的映射流程相比，GlobalISel 的 LLVM IR 到 GMIR 的映射过程更加简单，语义更加完整，优化机会也更多。这个过程省掉了 SelectionDAG 流程中 LLVM IR、SelectionDAG 以及 MachineDAG 多次转换过程所带来的繁琐和开销，编译过程非常简单。同时，GMIR 以函数粒度生成，能够保留 LLVM IR 中不同基本块之间的控制流和数据相关依赖信息的完整性，为函数级优化创造机会。另外，通用虚拟寄存器会跨基本块存在，LLT 又对异构类型系统比较友好，这使得 GMIR 在更大范围内起作用，为跨基本块的指令合并、全局寄存器分配等任务提供支撑。

2.2.2 GMIR 生成方法

GMIR 的指令分为两类：架构无关指令和架构相关指令，这样设计的好处在于既保留了通用优化的灵活性，又能通过架构特化逻辑适配不同硬件的底层约束。其中，架构无关指令包括算术指令、逻辑指令等，因为这类指令只表达通用机器的语义，不依赖于特定的硬件，故可以用 GlobalISel 统一生成；而架构相关指令主要是针对于硬件约束较强的指令操作，如函数调用、形式参数处理等操作。架构相关指令所依赖的 ABI 调用约定（即相应的寄存器约定、栈帧约定、参数传递约定等）是特定于目标机器的，必须有针对性地实现各种指令。

IRTranslator 是 LLVM IR 到 GMIR 的翻译器，负责对所有指令翻译过程进行协调和调度。在翻译的执行过程中，IRTranslator 以函数为单位对 LLVM IR 中的指令进行逐条遍历，将这些指令转换成对应的通用机器指令表示，最终完成从高级中间表示到通用机器语义的译码转换，为后续阶段提供标准化的输入。

CallLowering 是 IRTranslator 的核心，包括了参数传递、返回值处理等调用约定的细节处理。IRTranslator 在处理函数的入口、调用指令以及返回指令时，通过统一的接口调用目标后端的 CallLowering 来处理函数的常规翻译，从而实现通用翻译框架与目标架构定制的业务解耦。

基于这种设计，GlobalISel 能够在保持 IRTranslator 通用性的同时，还能实现对不同目标架构 ABI 差异的良好适配，从而实现通用指令翻译逻辑与目标架构定制逻辑之间的有效解耦。IRTranslator 与 CallLowering 之间的关系及协作方式如图2-5所示。

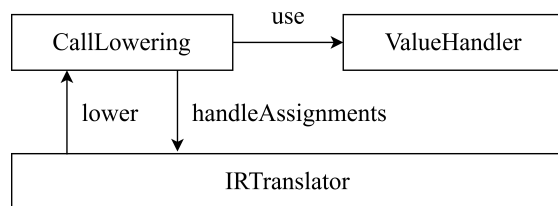


图 2-5 IRTranslator 调用降级流程及组件交互示意图

以函数调用约定处理为例，在函数调用的过程中需要将 LLVM IR 中抽象的函数调用、参数传递语义，转换为符合目标架构规范的 GMIR。由于不同架构的寄存器布局、栈对齐要求、参数存储优先级（寄存器优先或栈优先）存在显著差异，因此需要由各目标架构单独实现专属的 CallLowering 模块，模块通过解析自身的 ABI 约定来完成 LLVM IR 到 GMIR 的精准映射，从而确保后续指令选择与硬件执行的兼容性。

GMIR 生成的执行过程是以函数为粒度进行的，与 SelectionDAGISel 是以基本块为粒度不同，由于函数可以分为包含形参信息的函数头与函数体，函数体

又有基本块等表示，所以按处理的函数信息不同，将执行过程分为 4 个主要的阶段，如图2-6所示。

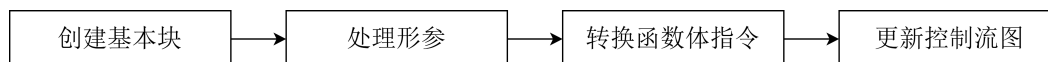


图 2-6 GMIR 生成流程图

1. 创建基本块：这个阶段会遍历函数的 LLVM IR 基本块，依次为每个基本块创建对应的 GMIR 的基本块，并且会保留相关的控制流信息，形成初始的控制流图。还会为每个函数添加一个额外的基本块（EntryBB），作为函数入口。
2. 处理形参：根据目标架构的调用约定规则处理函数的入参，为每个入参生成一条从传参寄存器复制到虚拟寄存器的 GMIR 复制指令，或者为入参生成一条从栈上加载到虚拟寄存器的 GMIR 加载指令，并将生成的指令放入 EntryBB 中。
3. 转换函数体指令：按 RPOT（Reverse Post-Order Traversal，逆后序遍历）的方式遍历函数的控制流图，即对于每个基本块，以自顶向下的顺序将基本块里的每条指令转换成一组 GMIR 指令。
4. 更新控制流图：在第 3 阶段的指令转换过程中，有些原本不是跳转的指令会被翻译成跳转指令（例如，跳转指令的条件码是由多条连续的逻辑运算指令生成的，此时就有可能拆分为逻辑运算生成多个跳转指令），导致原有基本块被拆分为多个新基本块，破坏原有的控制流。因此在基本块指令转换好后，需要维护好这些新基本块与原有基本块的控制流边，形成新的控制流图。此外，转换过程还可能会将 EntryBB 和函数体中的第一个基本块合并，此时控制流信息也要跟着更新。

2.2.3 GMIR 生成实现

根据 2.2.2 节设计可知，GMIR 的指令生成逻辑分为架构无关和架构相关两类。算术运算、逻辑运算等架构无关指令的处理由框架统一实现。与 DSP 结构强绑定的函数调用、函数参数、结果值等架构相关指令的处理需要根据 DSP 的 ABI、寄存器布局以及数据传输等特性来对 CallLowering 进行定制化开发。对于 DSP 结构来说，以上这些场景之间都会在寄存器宽度、类型支持、端序等方面有较大差异，因此也是 IRTranslator 实现的重点和难点。

为适配 DSP 的特性，本文基于 ValueHandler 接口实现了 DSPIncomingValueHandler 和 DSPOutgoingValueHandler 两个辅助类，这两个辅助类分别负责函数

入参数出参数的传递、函数返回值的返回，类的结构如图 2-7。这一设计将参数与函数返回值的相关处理从 IRTranslator 主流程中分离出来，可以让调用相关的架构特化代码能够集中被管理，有效地提高了可维护性和可扩展性。

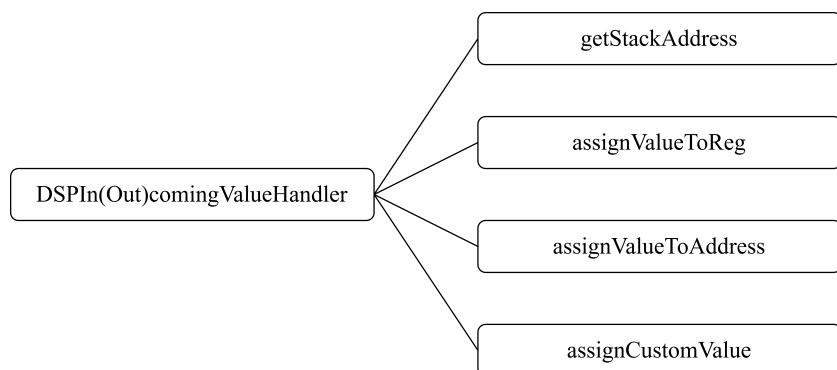


图 2-7 DSPValueHandler 核心函数

1. `getStackAddress` 用于生成 DSP 架构下栈帧空间的目标地址虚拟寄存器，根据上下文选择基于帧索引还是直接操作栈指针，为出参的栈上存储提供地址支持。
2. `assignValueToReg` 用于将目标值分配至物理寄存器，核心完成 COPY 指令构建，并按需标记虚拟寄存器的状态属性。
3. `assignValueToAddress` 用于完成栈上数据的加载和存储，将其加载或存储在合适的位置并在需要进行扩展。
4. `assignCustomValue` 是针对 DSP 架构的定制化实现，采用数据的合并、拆分算法解决非法类型跨函数传入的情况。例如，函数根据 DSP 的端序特性调整高低位数据的存储顺序，保证了跨函数的属性一致性。除此之外，函数会利用 `G_COPY` 指令将拆分的数据寻址映射到目标物理寄存器上。

在具体的实现中，IRTranslator 需要分别完成函数调用、形式参数以及返回值的降级处理等操作。其中，以 `lowerCall` 为例，其主要流程可概括为以下几个阶段：

1. 调用合法性校验：约束调用约定（如 DSP 仅支持 C 语言调用约定），确保跨函数调用的二进制兼容性；校验入参和返回值类型是否为架构支持的类型，提前拦截非法调用以避免后续流程异常。
2. 栈帧预处理与调用指令构建：`lowerCall` 会先生成栈帧调整的起始指令，为调用过程预留所需的栈空间；随后根据被调用者的类型动态选择合适的调

用指令，并将被调用者标识附加到调用指令中，从而为后续指令选择阶段提供明确的目标地址信息。

3. 出参传递与栈空间校准：lowerCall 对参数按 DSP 数据类型规则进行拆分，生成适配 DSP 结构的 ArgInfo 列表，再结合 DSPCCState 和参数调用规则 ABI，解析参数的寄存器/栈分配规则，由 DSPOutgoingValueAssigner 给出分配值的具体计算和参数传递。最后获取 DSP 结构栈对齐的要求，将栈空间的大小按对齐要求校准，使栈访问符合硬件约束。
4. 调用指令提交与间接调用约束：lowerCall 将构建完成的调用指令（CALL/-CALLR）插入当前基本块，若为 CALLR 指令，通过 constrainAllUses 结合 DSP 的 RegisterBank 信息及目标指令信息，约束指令操作数的合法性，避免跨银行访问或非法寄存器使用。
5. 返回值处理与栈帧恢复：若返回值不为空，lowerCall 则利用 DSPIncomingValueAssigner 解析出返回值分配策略，将物理返回值寄存器的值映射到通用虚拟寄存器，将栈指针恢复并释放调用过程占用的栈空间，从而完成整个调用流程。

上述 IRTranslator 的实现在 DSP 架构下实现了完整的函数调用、参数传递和返回等机制，解决了通用 CallLowering 逻辑在寄存器宽度和非法类型处理上的不足的问题，保证了语义正确性，也为后续指令合法化和指令选择阶段提供了清晰明确的中间表示基础。

2.3 指令合法化

在 IRTranslator 完成转换后，函数的表示方式由 LLVM IR 转变成了 GMIR。这一转换并非简单的指令映射，而是伴随通用机器语义适配的深度处理。为了衔接后续环节，IRTranslator 会在转换过程中嵌入部分目标架构的如寄存器组初步约束、调用约定相关标记等基础信息。但从设计本质来看，GMIR 作为通用机器中间表示的核心属性并未改变，转换生成的绝大多数 GMIR 指令仍保持架构无关性，这种设计虽保障了跨架构复用性，但这也必然导致一个核心问题：部分 GMIR 指令可能超出目标架构的硬件能力范围，成为非法指令，即有部分 GMIR 指令会存在目标架构不支持的情况。

非法 GMIR 指令的产生源于架构能力的固有差异，最典型的场景便是数据位宽不匹配。例如，针对 16 位字长的嵌入式架构，其硬件指令集仅支持 16 位或 32 位运算，若 IRTranslator 因 LLVM IR 中原有的 64 位数据运算生成了 64 位的 G_ADD 指令，该指令便会因超出硬件处理能力而成为非法指令。类似地，部分

架构不支持向量运算或特定类型的内存访问，对应的 GMIR 向量指令、特殊寻址模式指令也会被判定为非法。

为解决非法 GMIR 指令的问题，GlobalISel 框架实现了一个合法化 Pass，这个 Pass 是衔接 GMIR 与目标架构特性的关键枢纽。该 Pass 的工作原理是：首先加载目标架构的完整指令集描述信息，涵盖其支持的数据类型、运算操作以及寄存器约束等关键元数据，随后以函数为基本单位遍历 GMIR 代码，将其中所有非法指令逐一替换为语义等价的合法 GMIR 指令序列，从而使生成的指令组合能够完全适配目标架构的硬件能力。

2.3.1 指令合法化方法

合法化过程以函数为基本单位，采用 RPOT 算法自函数入口开始依次扫描所有基本块，该遍历顺序能够保证父基本块中的指令优先于子基本块被处理，从而有效避免由控制流分支依赖引发的转换冲突。在进入单个基本块之后，指令按照自顶向下的顺序逐条处理，对每一条 GMIR 指令进行合法性检查，一旦发现非法指令，立即触发相应的转换逻辑。待当前基本块内所有指令均完成合法化后，再继续处理下一个基本块，直至整个函数范围内的 GMIR 指令全部完成合法化。指令合法化的处理过程实际上包含了以下两个关键子问题的处理，二者共同构成了指令合法化的完整技术链路：

1. 非法指令识别：通过查询目标架构提供的 LegalizerInfo 接口来进行判断。一方面需要判断校验指令操作码与数据类型的组合是否支持，如 G_ADD 指令是否支持 64 位数据。另一方面需要检查指令的操作数约束是否符合架构的要求。例如，针对 ARM 架构的 LegalizerInfo 会明确标记 G_FADD（浮点加法）仅支持 32 位和 64 位，若 GMIR 中出现 16 位浮点加法指令，则会被 Pass 直接判定为非法。此外，部分架构对指令的隐含约束也会纳入识别逻辑，用于确保识别结果的精准性。
2. 非法指令转换：当检测到非法指令时，需要通过指令重构来生成一组语义不变且实现架构兼容合法指令序列。由于非法指令产生原因不同，转换策略通常可分为两类。针对数据位宽不匹配的情形，采用拆分与重组相结合的方法，例如将 64 位的 G_ADD 运算拆解为两个 32 位 G_ADD 指令，并通过进位标志的传递实现语义等价的计算。对于目标架构不支持的操作类型，如在缺乏硬件除法单元的体系结构上遇到 G_SDIV 指令，则需要通过算法模拟的方式加以替代。除此之外，在整个转换过程中还必须同步完成虚拟寄存器类型的调整以及指令间依赖关系的维护，从而确保重构后的指令序列在数据流和执行效果上与原始指令保持一致。

在以上两个子问题得到解决后，合法化 Pass 就能实现 GMIR 从通用到架构适配的关键转换。这为后续的寄存器组选择以及目标指令生成等流程提供了合法且可靠的输入基础。

2.3.2 指令合法化实现

合法化阶段的主要任务是完成非法指令的识别与非法指令的转化。在具体实现上，首先需要明确目标架构所支持的 LLT 类型集合，该集合不仅包括各类数据类型，还包括地址类型与内存访问相关的类型信息。上述类型集合构成了指令合法性判定的基础边界，用以描述目标架构在位宽支持、对齐约束以及寻址能力等方面的硬件限制。

在明确架构支持的 LLT 类型集合之后，构建一组用于对 GMIR 指令进行合法性判定与处理策略选择的合法化规则集。规则集的核心载体是 LegalityQuery 对象，对象采用结构化封装的方式，将合法性判定所需的关键信息统一组织为标准数据结构。具体包括指令操作码、各操作数索引对应的 LLT 类型、Machine-MemOperand 中描述的内存访问字节大小，以及针对内存类指令的原子性与顺序约束等信息。DSP 架构的部分合法化规则集如下所示。

```

1  DSPLegalizerInfo::DSPLegalizerInfo(const DSPSubtarget &ST)
2      : STI(ST), XLen(STI.getXLen()), SXLen(LLT::scalar(XLen)) {
3      const LLT S1 = LLT::scalar(1);
4      const LLT S8 = LLT::scalar(8);
5      const LLT S16 = LLT::scalar(16);
6      const LLT S32 = LLT::scalar(32);
7      const LLT S64 = LLT::scalar(64);
8      const LLT P0 = LLT::pointer(0, 32);
9      const LLT SDoubleXLen = LLT::scalar(2 * XLen);
10     .....
11     auto &ShiftActions = getActionDefinitionsBuilder({G_ASHR,G_LSHR,G_SHL});
12     if (ST.is64Bit())
13         ShiftActions.customFor({{S32, S32}});
14         ShiftActions.legalFor({{S32, S32}, {S32, SXLen}, {SXLen, SXLen}})
15         .widenScalarToNextPow2(0)
16         .clampScalar(1, S32, SXLen)
17         .clampScalar(0, S32, SXLen)
18         .minScalarSameAs(1, 0);
19     .....
20 
```

这种设计在工程上具有明显的优势。一方面，其他编译 Pass 可直接基于 LegalityQuery 对象查询指令合法性，而无需重复解析指令内部结构，从而有效地降低实现复杂度。另一方面，合法性判定过程不依赖于实际指令实例的构造，

这样做的好处是能够避免额外的对象创建与析构开销，在提升编译效率的同时，能支撑更为复杂和精细的合法性判断逻辑。

合法化规则集的生成与执行遵循统一的标准流程。通过调用 `getActionDefinitionsBuilder` 接口，可为指定操作码构建专属的合法化规则集。当输入多个操作码时，该接口会自动将其绑定至同一规则集中，适用于操作语义相近、合法化策略一致的指令类型。规则集中的各条规则按照自上而下的优先级顺序依次匹配，当某条指令成功匹配并完成合法性判定后，其后续处理立即终止，下一条指令将重新从规则集起始位置开始匹配，从而保证每条指令都能获得唯一且确定的合法化结果。

对于一些合法化逻辑复杂的指令，框架提供的标准化的规则集往往不足以无法完成目标架构适配，因此需要实现自定义合法化函数 `legalizeCustom` 来承载专有的合法化逻辑。与此同时，对于架构强相关的内建函数，还需要额外实现 `legalizeIntrinsic` 的处理入口，以完成对应内建函数的合法化。以 `G_VASTART` 指令为例，该指令用于在可变参数函数中标记可变参数的起始位置，由于规则集不足以完成 DSP 架构的适配，于是需要通过 `legalizeCustom` 实现该指令的合法化，将通用的 `G_VASTART` 语义映射为 DSP 架构可执行的内存存储序列，并显式体现与栈帧布局相关的约束关系。实现代码如下所示。

```

1  bool DSPLegalizerInfo::legalizeVASTart(MachineInstr &MI,
2      MachineIRBuilder &MIRBuilder) const {
3      assert(MI.getOpcode() == TargetOpcode::G_VASTART);
4      MachineFunction *MF = MI.getParent()->getParent();
5      DSPMachineFunctionInfo *FuncInfo = MF->getInfo<DSPMachineFunctionInfo>();
6      ;
7      int FI = FuncInfo->getVarArgsFrameIndex();
8      LLT AddrTy = MIRBuilder.getMIRI()->getType(MI.getOperand(0).getReg());
9      auto FINAddr = MIRBuilder.buildFrameIndex(AddrTy, FI);
10     assert(MI.hasOneMemOperand());
11     MIRBuilder.buildStore(FINAddr, MI.getOperand(0).getReg(), *MI.
12         memoperands()[0]);
13     MI.eraseFromParent();
14     return true;
15 }

```

2.4 寄存器组选择

虽然指令合法化阶段引入了目标架构的指令信息，并将 GMIR 生成阶段生成的 GMIR 转换为符合目标架构约束的合法 GMIR，但此时的 GMIR 指令仍然没有具体的寄存器分配信息。指令里的虚拟寄存器操作数仅携带用于表示其类型和位数大小的数据类型描述。因此，全局指令选择模块需要进一步为 GMIR 中

的虚拟寄存器确定其可映射的寄存器组类型，即完成寄存器组选择过程。

寄存器组选择是全局指令选择模块的第三个基本 Pass，它会利用目标架构的寄存器信息，自顶向下的为合法化后的 GMIR 指令中的虚拟寄存器操作数分配合适的寄存器组类型，并且它还可以利用 GMIR 指令之间的关系选取一个较优的寄存器组类型。这也是不能直接在 GMIR 生成处理中为直接分配寄存器类型的原因之一：在 GMIR 生成阶段生成指令时，LLVM IR 还没有全部转换成 GMIR，无法利用指令之间的关系进行寄存器类型择优。

2.4.1 寄存器组

在 GlobalISel 框架中，Register Bank（寄存器组）是面向目标架构的重要抽象组件，其设计目标是在满足硬件指令访问约束的前提下，通过弱约束分组机制管理寄存器组资源，实现寄存器分配灵活性与跨寄存器组数据交互开销的平衡，从而兼顾编译效率与生成代码性能。与传统的 Register Class（寄存器类）不同，Register Bank 仅关注寄存器的最大数据宽度及支持的操作集合，而不对具体物理编号和精确位宽做强约束，这使得编译器在分配阶段拥有更大的选择空间。

Register Bank 的提出源于硬件架构的实际限制。许多嵌入式处理器及异构计算平台将物理寄存器划分为多个相互独立的寄存器文件，不同文件之间的寄存器无法被同一条指令同时访问。若操作数分布在不同寄存器文件中，必须通过额外的数据复制指令完成中转，从而引入不必要的性能开销。寄存器组通过将物理寄存器按文件属性进行逻辑分组，使编译器能够在寄存器分配阶段优先将关联变量分配到同一寄存器组中，从源头减少跨文件数据复制。

在实际架构中，寄存器组的划分通常与运算单元类型相关，例如通用寄存器组（GPR）对应整数运算单元、浮点寄存器组（FPR）对应浮点运算单元。GMIR 指令在寄存器组的使用上遵循以下原则：一方面，特定指令必须使用指定寄存器组（如浮点运算只能使用 FPR）；另一方面，部分指令允许在多个寄存器组间灵活分配，其最终选择由后续运算需求决定，从而避免不必要的跨组数据复制。该弱约束设计不仅降低了寄存器分配复杂度，也显著减少了跨寄存器文件的数据搬运开销，尤其适合嵌入式与 DSP 等资源受限架构。同时，寄存器组的抽象方式具有良好的扩展性，新架构仅需在目标描述中定义寄存器组属性即可完成适配，无需修改编译器核心逻辑。表2-2列出了 DSP 架构下使用的 Register Bank 与 Register Class 划分情况。

2.4.2 寄存器组选择方法

寄存器组选择阶段的核心任务是在 GMIR 指令完成合法化处理后，为指令中所有虚拟寄存器操作数匹配并分配适配的寄存器组，从而确保后续指令选择

表 2-2 DSP 架构下寄存器抽象类型与分类

寄存器抽象层级	具体实例
Register Bank	GPRBRegBank、CRBRegBank、VPRBRegBank
Register Class	GPR32EVEN、GPR32、GPR64、VTRegs、 OBMRegs、VPR4Out、VPR8Out、VPR16Out、 VPRSf、VPRHf

与物理寄存器分配过程能够满足目标架构的硬件访问约束。在 DSP 架构下，寄存器组选择模块的整体设计如图 2-8 所示。该模块主要由 DSPGenRegisterBankInfo 与 DSPRegisterBankInfo 两部分构成：前者由 TableGen 根据 DSPRegisterBanks.td 文件自动生成，负责描述目标架构中可用的寄存器组及其基础属性；后者则作为目标后端的核心实现类，封装了寄存器组选择所需的全部信息与决策逻辑，包括指令到寄存器组的映射规则、操作数属性以及不同映射方案之间的代价评估。

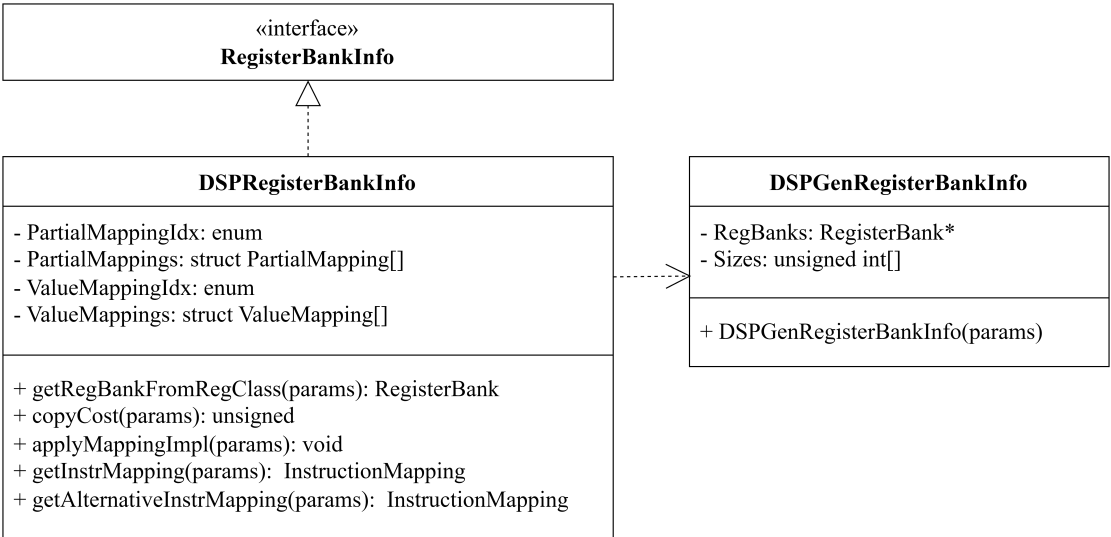


图 2-8 DSP 架构下寄存器组选择模块结构图

2.4.3 寄存器组选择实现

基于上一节对寄存器组选择核心机制的设计分析，本节将从寄存器组定义、映射结构定义及映射逻辑实现三个核心维度展开，详细阐述面向 DSP 架构的编译器后端中，寄存器组选择模块的具体实施方案。

1. 寄存器组的定义

DSP 处理器内部包含了多种不同类型的寄存器，不同寄存器在位宽、用途及访问方式上存在显著差异。为保证寄存器分配与硬件执行语义的一致性，需要

首先对目标架构中可参与寄存器分配的寄存器资源进行建模。表2-3给出了 DSP 架构中主要寄存器类的数量、位宽及功能特性，为后续寄存器组划分与寄存器组选择策略的设计提供硬件基础。表中列出的仅为 DSP 架构下部分不透明寄存器类及其功能描述，其中不透明寄存器类是指对编译器后端可见、并参与寄存器分配与调度的寄存器集合；而对开发人员透明的寄存器类（如中断返回地址寄存器、基址寄存器、偏移寄存器及取模寄存器等）由于其使用方式固定，不参与寄存器分配过程，故不在此列出。

表 2-3 DSP 架构主要寄存器类特性

寄存器类	数量	宽度 / bit	功能描述
通用寄存器 (GR)	32	32	使用频率最高的寄存器，用于存放标量指令中的运算数据及地址信息，支撑整型运算与内存访问等基础操作
矢量寄存器 (VR)	4 × 16	640	用于存放矢量运算数据，采用四路并行设计，支持 SIMD 模式下多数据元素的并行加载、存储与计算
控制寄存器 (CR)	1	32	用于保存处理器状态信息，包括进位标志、溢出标志以及比较指令的结果标志等，参与指令执行控制与条件判断

在 DSP 架构编译器后端的 RegisterInfo 类中，通用寄存器被划归至 CPURegs 寄存器类，矢量寄存器被划归至 CPUVecRegs 寄存器类，控制寄存器被划归至 CCR 寄存器类。上述寄存器类的划分构成了寄存器组定义的基础，基于该分类体系完成了 DSP 架构寄存器组的定义如下所示。

```

1 def GPRBRegBank : RegisterBank<"GPRB", [GPR32, GPR64]>;
2 def VPRBRegBank : RegisterBank<"GPRB", [VPR4Out, VPR8Out, VPR16Out]>;
3 def CRBRegBank : RegisterBank<"CRB", [CCR]>;

```

2. 映射结构的定义

寄存器组选择阶段采用三级分层映射结构，其整体架构及层级关联关系如图2-9所示。该映射体系自底向上由 PartialMapping、ValueMapping 和 Instruction-Mapping 三个层次逐级构成，用于描述指令操作数到寄存器组的映射关系及其代价评估过程。

作为三级分层映射结构的最底层原子映射单元，PartialMapping 的核心功能是精准描述操作数中某一连续比特区间与特定寄存器组的单向绑定关系，其设计天然支持大整数位宽拆分、向量数据分片映射以及非对齐数据布局等复杂场景，可灵活适配不同指令类型的操作数映射需求，为上层映射结构提供细粒度的硬件约束支撑。以图2-9中指令为例，64 位整型加法指令 G_ADD 的通用操作数

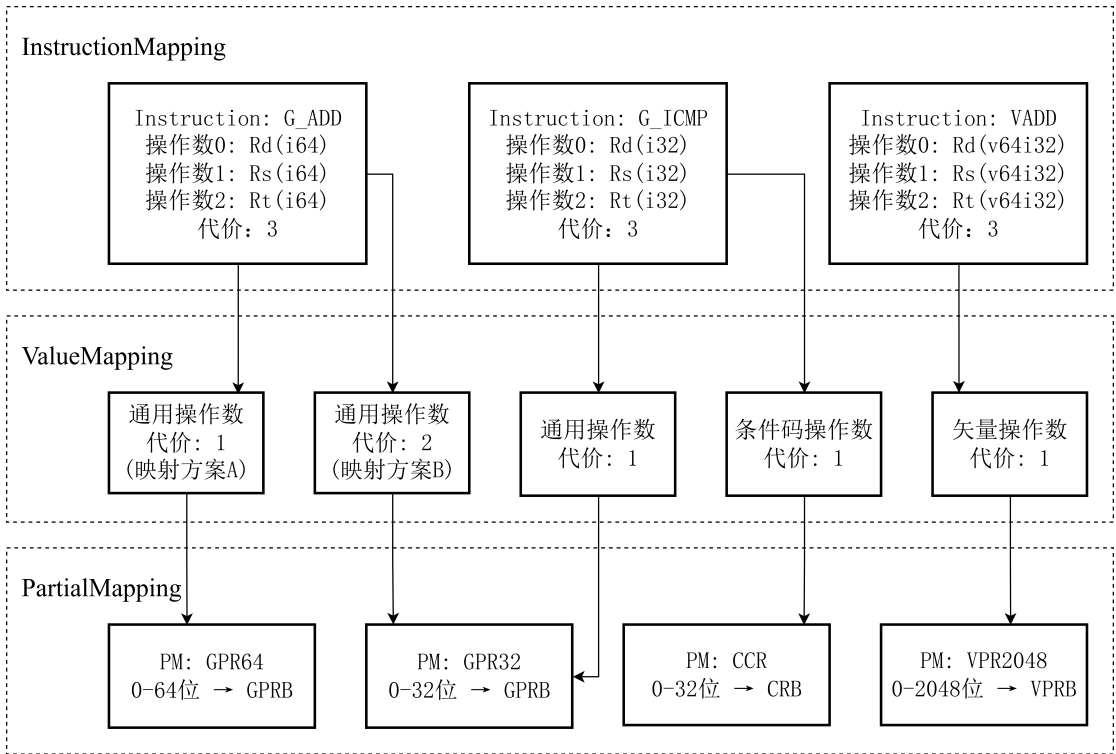


图 2-9 寄存器组分层映射结构图

可通过一个 PartialMapping 将 0 ~ 64 位整体映射至通用寄存器组 GPRB；32 位整型比较指令 G_ICMP 的通用操作数仅需将 0 ~ 32 位映射至 GPRB。

ValueMapping 由一个或多个 PartialMapping 组合而成，用于描述单个操作数到寄存器组的完整映射方案。图中展示了同一类通用操作数可能存在多种 ValueMapping 方案的情况，不同方案对应不同的映射代价，用于反映寄存器使用效率或潜在的跨寄存器组数据复制开销。以图2-9中的 64 位 G_ADD，其操作数既可以用一个 GPR64 来映射，又可以用两个 GPR32 来映射。

InstructionMapping 的作用是对一条指令的所有操作数映射进行统一建模，不仅包含每个操作数对应的 ValueMapping，还关联了该指令整体映射方案的代价值。以图2-9中指令为例，G_ADD、G_ICMP 以及 VADD 等指令均可生成各自的 InstructionMapping，其总代价由各操作数映射代价综合决定。寄存器组选择阶段将基于这些代价信息，在满足指令硬性约束的前提下选择最优映射方案。

在具体实现中，有 Fast 和 Greedy 两种寄存器组分配策略。其中 Fast 策略仅为指令匹配并分配目标架构预定义的默认可用寄存器组组合，无需枚举所有合法方案，因此实现简单、运行开销低，适用于对编译速度敏感的场景；而 Greedy 策略则在默认映射方案的基础上，进一步枚举目标架构允许的所有合法寄存器组组合，逐一计算其映射代价，并最终选择代价最低的方案。该策略能够在更大搜索空间内优化寄存器组分配，减少跨寄存器组的数据复制开销，但相应地增

加了寄存器组选择阶段的计算成本。

在 LLVM 的 RegisterBankInfo 实现中, PartialMapping、ValueMapping 等结构通常以静态表的方式定义, 用于描述不同位宽、不同操作数类型在目标架构下的寄存器组映射规则。以下代码给出了 DSP 目标架构中 PartialMapping 与 ValueMapping 的典型定义方式。

```

1  PartialMapping PartMappings[] = {
2      {0, 32, GPRBRegBank},
3      {0, 64, GPRBRegBank},
4      {0, 32, CRBRegBank},
5      ..... };
6
7  ValueMapping ValueMappings[] = {
8      {nullptr, 0},
9      {&PartMappings[PMI_GPRB32], 1},
10     {&PartMappings[PMI_GPRB32], 1},
11     {&PartMappings[PMI_GPRB32], 1},
12     {&PartMappings[PMI_GPRB64], 1},
13     {&PartMappings[PMI_GPRB64], 1},
14     {&PartMappings[PMI_GPRB64], 1},
15     ..... };

```

通过上述分层映射结构的实现, 寄存器组选择阶段完成了从比特级类型映射到全指令级分配决策的逐层抽象。该方法既避免了在早期阶段过早绑定具体物理寄存器, 又能够在完整的指令粒度上对不同寄存器组分配方案的代价进行评估, 从而为后续的指令选择与寄存器分配阶段提供了明确且优化的寄存器资源使用约束。

3. 映射逻辑的实现

寄存器组选择的核心实现通过重载 RegisterBankInfo 类中的相关接口完成, 其中最核心的是 getInstrMapping 与 getInstrAlternativeMapping 两个函数。前者负责为每条指令生成默认的寄存器组映射方案, 后者则用于在默认映射不适用或存在更优分配时, 提供备选的寄存器组映射方案, 从而支持更灵活、更高效的寄存器资源分配策略。

getInstrMapping 的实现流程如算法 1 所示。其实现采用以规则驱动为核心的寄存器组映射构建流程。该流程首先以指令操作码为入口, 对可直接由 TableGen 规则覆盖的场景进行快速判定。对于特殊指令, 优先尝试使用 TableGen 自动生成的映射结果; 一旦匹配成功, 即可直接返回对应的 InstructionMapping, 从而避免后续不必要的分析与代价评估。

当 TableGen 映射不可用时, 算法进入基于操作数语义的通用映射阶段。此

Algorithm 1: Core Logic of getInstrMapping

Input: MI: Machine Instruction
Output: InstructionMapping

```

1  Opc ← opcode of MI;
2  if MI is non-generic or Opc is G_PHI then
3      Mapping ← getInstrMappingImpl(MI);
4      if Mapping is valid then
5          return Mapping;
6      end
7  end
8  Initialize operand mapping array OpdsMapping;
9  foreach operand op in MI do
10     if op is a register and op.reg is valid then
11         Obtain the low-level type Ty of op.reg;
12         if Ty is invalid or bit width of Ty ≤ 32 then
13             Assign op to a 32-bit GPR container;
14         end
15         else
16             Assign op to a 64-bit GPR container;
17         end
18     end
19 end
20 if Opc consumes a condition value then
21     Override the mapping of the condition operand to the control register
        bank (CRB);
22 end
23 if Opc produces a comparison result then
24     Force the result operand to use a 32-bit GPR container;
25 end
26 Construct and return the final InstructionMapping from OpdsMapping;

```


阶段以操作数为粒度，遍历指令中的所有寄存器操作数，并依据其 LLT 进行映射决策。在完成默认映射后，算法进一步结合指令语义对映射结果进行修正。最终，算法基于上述映射规则构建并返回完整的 `InstructionMapping`。

2.5 机器指令选择

机器指令选择是 `GlobalISel` 框架的第二阶段核心任务，承接第一阶段（`GMIR` 生成、指令合法化、寄存器类型选择）的处理结果，其核心目标是将经过合法化的、附着目标架构信息的通用机器中间表示 `GMIR`，完整转换为目标架构专属的机器中间表示 `MIR`，为后续基于 `MIR` 的寄存器分配、指令调度、窥孔优化等后端流程提供合法输入，是衔接通用中间表示与架构专属表示的关键枢纽。

机器指令选择以单个函数为处理粒度，基于树覆盖的指令选择算法，通过将 `GMIR` 指令序列构建为 `AST`，再通过树模式匹配映射为目标架构指令序列，确保语义等价性与架构兼容性。目前框架实现了两种树覆盖的方式：一种是基于表驱动的自动状态机进行的自动覆盖方式；另一种是基于固定模式的手动覆盖方式。这两种覆盖方式在每次覆盖的时候都只会产生一种成功匹配的树模式，因此可以直接生成对应的 `MIR` 指令序列^[52]。

机器指令选择采用 `PROT+` 自底向上处理的执行逻辑，确保指令依赖关系的正确性与处理效率，具体流程如下：首先以单个函数为处理单元，采用 `RPOT` 算法从函数底部开始依次扫描所有基本块，该遍历顺序可保障父基本块的指令处理滞后于子基本块，避免因分支依赖导致的未定义指令引用问题；进入单个基本块后，按自底向上的顺序逐行处理每条 `GMIR` 指令，处理前先判断当前指令是否已生成对应的 `MIR` 指令，若已生成则直接跳过以避免重复处理，若未生成则以该指令为根节点触发自动匹配模块与手动匹配模块的协同匹配流程；匹配成功后由指令生成模块输出对应的 `MIR` 指令序列，若匹配失败则直接触发编译报错；最后以基本块为单位迭代执行上述流程，直至整个函数的所有 `GMIR` 指令均成功转换为 `MIR` 指令或触发报错，完成该函数的机器指令选择任务。

2.5.1 匹配状态机

在 `LLVM` 的指令选择体系中，匹配状态机是支撑基于表驱动树的树覆盖指令选择的核心组件，匹配状态机本质是一种由 `TableGen` 工具自动生成的 `FSM` (`Finite State Machine`，有限状态机)，其核心作用是将 `GMIR` 指令构成的 `AST` 与目标架构的指令模式进行自动化匹配，从而确定可映射为 `MIR` 的合法目标指令模式。

在 `TableGen` 的描述文件中，目标架构的指令结构以树模式 (`Pattern`) 进行形式化定义。`TableGen` 工具会对所有 `Pattern` 进行解析，并提取其关键特征，包

括操作码、操作数类型、寄存器组约束以及数据位宽等。这些特征随后被系统化地编码为状态机的状态节点与转移条件。TableGen 最终将所有状态节点及其间的转移关系整合，生成完整的状态机实现代码。该状态机覆盖了从初始状态开始，历经若干中间匹配状态，直至到达最终成功匹配状态或失败状态的完整路径。在生成的状态机中，每一个成功匹配状态都唯一对应一条可被选择的目标架构指令模式，从而确保指令匹配过程的确定性与一致性。

在机器指令选择阶段，匹配状态机针对单条 GMIR 指令的执行流程可概括为以下几个步骤：

1. 指令树构建：以当前待处理的 GMIR 指令为根节点，递归遍历其操作数、子指令，构建完整的抽象语法树。
2. 状态机初始化：将状态机置为初始状态，从指令树的叶子节点开始向上遍历。
3. 状态转移匹配：遍历过程中，根据当前 AST 节点的特征触发状态机的转移并进入对应状态；若在某一节点处不存在满足条件的转移路径，则判定该子树匹配失败，状态机回退至最近的分支点并尝试其他可行路径；否则输出对应的目标架构指令树模式。

为保证指令匹配结果的确定性，LLVM 中的指令匹配状态机采用 DFA (Deterministic Finite Automaton, 确定性有限状态机) 模型进行构建。这个设计确保了对于任一给定的指令树，其在整个匹配过程中遵循唯一确定的状态转移路径，并最终导向唯一对应的成功匹配结果，从机制上避免了多模式冲突问题。

基于 DFA 构建的指令匹配状态机，其在指令选择阶段的核心优势主要体现在自动化生成与可扩展性两个方面。开发人员无需为每条目标指令手动编写匹配代码，仅需在 .td 文件中声明相应的指令树模式，TableGen 工具即可自动生成完整、正确的状态机实现。这一方式显著减少了为目标架构开发指令选择器所需的实现工作量。

当目标架构需要新增指令支持时，仅需在描述文件中补充对应的树模式定义并重新生成状态机，无需修改编译器核心代码，从而在保持指令选择框架整体稳定性的同时，极大地提升了系统的可维护性与可扩展性。从运行效率角度分析，该状态机采用确定性的状态转移机制，在匹配过程中无需对所有可能的指令模式进行穷举搜索。相比基于手动遍历或回溯试探的传统匹配方法，这种机制能够有效提升指令选择环节的执行效率。

2.5.2 机器指令选择方法

从功能划分的角度来看，机器指令选择可划分为三个模块：自动匹配模块、手动匹配模块以及指令生成模块。三者共同完成从 **GMIR** 到 **MIR** 的语义映射过程。

1. 自动匹配模块

自动匹配模块的核心机制在于构建一个基于表驱动自动状态机，通过状态转移的方式，实现 **GMIR** 与目标架构指令模式的自动匹配。该设计的主要优势在于能够在不依赖人工干预的情况下，高效适配大多数通用指令的匹配需求，从而显著增强了编译器对于指令集扩展的自适应能力。

这一方法不仅有效减少了为每种指令手动编写、维护匹配逻辑的代码量与开发成本，更重要的是，当目标架构进行指令集扩展或引入新指令时，仅需更新模式定义并重新生成状态机，而无需深度修改编译器核心代码。这种机制在保证代码健壮性的同时，大幅提升了编译器后端的整体可维护性与长期可扩展性。

自动匹配模块的工作流程可进一步划分为两个阶段：第一阶段发生在编译器构建期，由 **TableGen** 工具解析目标架构指令的树模式描述，并将其编码为确定性有限状态机；第二阶段发生在编译期，指令选择器利用已生成的状态机，对单条 **GMIR** 指令构建的抽象语法树进行自底向上的遍历，通过状态转移自动判定其可匹配的目标指令模式。

2. 手动匹配模块

尽管自动匹配模块能够覆盖大多数通用指令的选择需求，但在实际编译器工程实践中，部分目标架构特有的复杂指令的语义或操作数约束难以通过 **TableGen** 的树模式进行完整描述。针对这类场景，需要通过自定义 **C++** 代码实现精细化的指令匹配逻辑。

手动匹配模块通常以内联的 **C++** 代码形式实现，其核心逻辑基于对 **GMIR** 的显式条件判断。该模块会逐一检查指令的操作码、操作数类型及寄存器约束等关键属性，以判定其是否符合特定目标架构专属指令的语义与格式要求。一旦匹配条件得到满足，模块将直接生成对应的目标机器指令。

作为自动匹配机制的补充，手动匹配模块专门用于处理那些高度依赖具体架构、或对执行性能及硬件语义有严格要求的复杂指令场景。它通过提供细粒度的匹配控制，有效弥补了纯表驱动方法在形式化表达能力上的局限，确保了编译器能够全面且精确地支持目标指令集的全部特性。

3. 指令生成模块

指令生成模块处于机器指令选择流程的最终环节，其输入为经过匹配模块（自动或手动）所确定的指令树模式。该模块的核心功能是根据预定义的映射关系，将选定的模式实例化，构造出语义完全等价的机器中间表示（MIR）指令序列。在此过程中，它负责完成具体的操作数绑定、寄存器类别指派以及其他必要的指令属性设置。

通过这一转换流程，GMIR 中的平台无关指令，被最终转化为与特定目标硬件架构紧密绑定、形式精确的机器指令表示。这些生成的 MIR 指令随后便可直接参与后续的寄存器分配、指令调度等低级机器代码优化阶段。

2.5.3 机器指令选择实现

机器指令选择阶段以单条 GMIR 指令为处理粒度，目标是在保证语义等价与约束一致的前提下，将通用中间表示逐条映射为符合 DSP 目标架构语义与资源约束的机器指令。其整体流程可概括为预处理、特殊指令处理、合法性校验、自动匹配以及手动匹配五个阶段，整体流程如算法2所示。这种分段的设计能在保持指令选择流程清晰性的同时，也便于针对不同类别指令引入差异化的处理策略。下面对每个阶段的实现逻辑与设计要点进行详细说明。

1. 预处理

为满足 DSP 目标架构的指令语义，在进入指令选择阶段前，需对部分通用指令执行前置 Lowering，将其转换为更贴近底层硬件的等价形式。一个典型的场景是指针运算的处理：由于 DSP 架构没有专用的指针运算指令，指针通常以整型寄存器保存并参与运算，因此编译器需要将携带指针语义的通用指令转换为语义等价的整型运算指令，并同步更新结果寄存器的 LLT，以确保后续的指令匹配、操作数约束检查等环节能够基于统一的、与硬件相符的类型系统进行。

2. 特殊指令处理

对于语义或结构特殊、难以通过树模式描述的指令，需要在自动匹配之前进行单独处理，主要包括以下几类。

- PHI 指令用于描述控制流汇合处的 SSA 值合并，属于编译器内部语义，不对应具体硬件指令，其处理重点在于保证定义寄存器与各输入操作数满足一致的寄存器约束，以确保后续寄存器分配的合法性。

Algorithm 2: Core Logic of Instruction Selector

Input: MI: Machine Instruction
Output: Boolean: Instruction selection result

```

1 Initialize context objects for MI (MBB, MF, MRI, MIB);
2 Execute pre-selection lowering on MI;
3 if MI is a copy instruction then
4   | return Process copy instruction selection for MI;
5 end
6 Get opcode (Opc) of MI;
7 if Opc is G_PHI then
8   | Extract target register (DefReg) of PHI;
9   | Derive target register class (DefRC) of DefReg;
10  | if DefRC derivation fails then
11    | return false;
12  | end
13  | for each operand in PHI instruction do
14    | if Operand cannot be constrained to DefRC then
15      | return false;
16    | end
17  | end
18  | Update PHI instruction descriptor;
19  | return Constrain DefReg to DefRC;
20 end
21 if MI has implicit operands not declared explicitly then
22   | return false;
23 end
24 if Opc is a condition-related generic instruction then
25   | return Apply condition-specific selection;
26 end
27 if table-driven selectImpl succeeds then
28   | return true;
29 end
30 if Opc is a special-type generic instruction then
31   | return Apply special-type instruction selection;
32 end
33 return false;

```

- COPY 指令仅承担数据搬运语义，不包含可匹配的运算结构，且可能涉及虚拟寄存器与物理寄存器之间或跨寄存器资源的数据复制，通常无法通过通用树模式完成映射，因此需要采用专门的选择逻辑。
- 依赖控制寄存器条件位的指令，如 BRCOND、ICMP 以及 FCMP 等，需在 TableGen 驱动的通用匹配流程之前进行提前处理。通过在指令选择阶段直接识别并生成符合目标架构条件码语义的指令序列，可以避免条件值经由普通数据路径再映射至控制寄存器所带来的冗余指令与数据搬移开销。

3. 合法性校验

在进入模式匹配前，对指令结构进行必要的合法性检查。例如，通用指令不应携带未显式声明的隐式操作数；若指令形态异常，直接返回失败以避免产生不一致的选择结果，并便于定位后端实现问题。

4. 自动匹配

对通过前述阶段筛选的通用指令，调用 selectImpl 执行基于 TableGen 的树覆盖匹配。该路径覆盖面广、维护成本低，能够将绝大多数通用算术、逻辑、访存等指令自动映射为目标指令模式，是指令选择的主要实现方式。

为兼容 SelectionDAGISel，GlobalISel 对通用算术与逻辑指令直接复用既有指令模式，而对依赖 SDNode 的架构专属信息则重新建模，并通过 GINodeEquiv 显式建立 GMIR 操作码与 ISD 操作码的等价关系，实现指令模式的复用。以下代码展示了加法指令的复用。

```

1  Class GINodeEquiv<Instruction i, SDNode node> {
2      Instruction I = i;
3      SDNode Node = node;
4  }
5
6  def : GINodeEquiv<G_ADD, add>;

```

5. 手动匹配

当指令无法被自动匹配覆盖时采用手动匹配。手动匹配通常以专用选择函数的形式存在，对操作数、立即数约束及目标指令序列进行精确构造。例如，针对寄存器内符号扩展的 G_SEXT_INREG，可通过检查扩展位宽并构造等价的目标指令序列完成选择，同时对新生成指令施加寄存器约束，保证与后续流程兼容。

手动匹配机制在处理因芯片型号差异导致的指令集异构问题时发挥着关键作用。以 `UMULH` (无符号高半乘法) 为例, 其实现路径因硬件支持度不同而存在显著差异: 对于原生支持该指令的芯片型号, 可以直接通过 `TableGen` 的 `Pattern` 将 `GMIR` 操作映射为硬件指令; 而对于不支持 `UMULH` 的型号, 则需要在手动路径中将其展开为等价指令序列。

这种设计确保了条件值的生成、传递与消费过程始终被约束在一条最小且确定的指令通路内。这不仅简化了指令选择的逻辑复杂度, 减少了指令序列中不必要的数据复制操作, 同时也有助于降低寄存器分配的压力与整体执行延迟。对于分支密集、条件判断频繁的程序代码而言, 该优化能够显著提升最终生成代码的执行效率。

2.6 本章小结

本章详细阐述了 `GlobalISel` 在 `DSP` 架构上的设计方法与具体实现。首先通过对现有指令选择方案的对比分析, 指出传统 `SelectionDAGISel` 在流程复杂度和维护成本方面的局限, 以及 `FastISel` 在代码质量与指令覆盖能力上的不足的问题, 在此基础上提出将 `GlobalISel` 应用到 `DSP` 架构上的技术方案。

本章随后根据 `GlobalISel` 的核心执行流程, 依次介绍了其在 `DSP` 后端中的具体实现方法: 在 `GMIR` 生成阶段, 通过定制 `ValueHandler` 适配 `DSP` 的 `ABI` 与数据类型约束; 在合法化阶段, 结合 `LegalizerInfo` 规则集与 `Custom` 处理机制确保指令语义落入硬件能力范围; 在寄存器组选择阶段, 针对多寄存器文件架构构建映射与代价模型; 在机器指令选择阶段, 采用 `TableGen` 自动匹配与手动匹配相结合的策略完成指令映射。

第 3 章 面向 DSP 的全局指令选择优化策略

与 SelectionDAGISel 相比，GlobalISel 在性能潜力上存在先天局限。其根源在于两者的设计目标与底层模型存在本质差异。SelectionDAGISel 的核心理念是针对特定硬件进行深度优化。它基于一个全局的、具有丰富语义信息的有向无环图模型，这使得编译器能够在整个函数的范围内进行详尽的指令选择与优化，从而在匹配复杂指令模式、优化调度和执行效率方面具有理论上的优势。换言之，它牺牲了部分编译速度和通用性，以换取为特定架构生成更优代码的潜力。

反观 GlobalISel，其首要设计目标是提升编译速度与增强跨平台可维护性。它采用一种更线性、增量式的处理模型，放弃了全局 DAG 所支持的综合性优化视角。这种通用化、模块化的设计使其在适配新架构时更加便捷，但也意味着它在进行指令选择时，天然缺乏进行全局性、深度窥孔优化的能力。因此，当其面对 DSP 这类对指令级效率和硬件特性匹配要求极高的专用架构时，这种通用性与性能最优之间的内在矛盾便被放大，直接导致了一些性能问题。

为解决这些性能问题，本文基于面向 DSP 的 GlobalISel 框架，创新地提出了一种分阶段协同优化方案，该方案分为合法化前的通用冗余消除与合法化后针对 DSP 架构的特定优化两个主要阶段，通过递进式优化兼顾通用中间表示的冗余消除与 DSP 硬件架构的特性适配。本章后续将详细阐述这种面向 DSP 的全局指令选择优化体系的整体设计思路，拆解分阶段优化策略的具体实现流程，并对其中涉及的关键技术细节进行深入剖析，并明确其相较于现有工作的核心创新与改进之处，为各类专用架构的 GlobalISel 优化落地提供可复用的技术路径。

3.1 优化策略分析与设计

经过 GMIR 生成、指令合法化、寄存器组选择以及机器指令选择 4 个 Pass 之后，原来的 LLVM IR 转换成了 MIR。尽管这一流程已经实现了指令选择的目标，但是转换之后生成的代码质量相比于 SelectionDAGISel 较差。这个差距主要体现在指令数量增加，专用指令利用率低，寄存器选取不够精细高效等。

为了提升 GlobalISel 生成代码的质量并缩小其与 SelectionDAGISel 之间的差距，本文引入了包括 CSE (Common Subexpression Elimination, 公共子表达式消

除)、KnownBits Analysis (已知位分析) 以及 Combiner (合并优化) 等优化策略。这些优化策略通过减少重复子表达式、利用比特级已知信息指导指令选择, 并在保持语义等价的前提下合并和简化指令序列, 从而降低了指令冗余度, 提升了生成代码的性能表现。

3.1.1 优化策略理论依据

程序编译过程 (尤其是从高层 LLVM IR 编译到目标特定低级指令时) 必然引入大量的冗余, 这些冗余由计算冗余 (公共子表达式重复计算)、数据冗余 (冗余拷贝) 和表示冗余 (大量更复杂的指令序列可以用更简单的指令序列来表示) 等引起。优化的任务就是要系统地检测、消除这些冗余, 在程序语义不变的前提下提高程序生成代码执行效率、代码密度、硬件利用率等。

GlobalISel 与 SelectionDAGISel 在代码质量上的差异, 根本原因在于二者的设计目标与实现方式不同。SelectionDAGISel 是针对特定架构定制的设计, 其核心中间表示 SelectionDAG 天然具备对基本块内局部计算图进行重写、合并与规约的能力, 从而能够在指令选择阶段有效消除冗余计算; GlobalISel 为了支持不同架构指令集的特性而采用通用中间表示 + 模块 Pass 的设计, 因此在中间代码产生时会不可避免地产生冗余, 产生这种冗余的原因主要有三点:

1. 通用化转换带来的语义冗余: 为保证跨架构语义的一致性, GlobalISel 在 LLVM IR 向 GMIR 转换及指令合法化阶段, 会将复杂运算拆分为一系列简单的原子指令。例如, 将 DSP 架构的乘加运算 (MAC) 拆分为乘法与加法指令, 将宽类型数据访问拆分为多次窄类型加载/存储指令, 这种拆分虽保证了语义正确性, 却引入了大量指令级冗余。
2. 寄存器组选择的局部性局限: 在寄存器组选择阶段, 寄存器组的分配以基本块为粒度, 没有充分考虑函数级的全局数据流依赖, 容易产生频繁地跨基本块数据复制。例如循环变量在不同基本块中被分配到不同寄存器组, 需通过专用复制指令完成数据迁移, 这显著增加了执行延迟。
3. 硬件特性适配的滞后性: 机器指令选择阶段的主要目的是完成从通用 GMIR 到目标指令的语义映射, 因而默认采取的是局部的、简单的一对一语义映射。复杂指令模式的识别被推迟到了较晚的阶段, 编译器无法及时识别跨越多条 GMIR 指令的复杂语义, 也就无法完整、充分地挖掘硬件中专用功能单元性能潜力。

从理论上讲, 高效优化的前提是要基于精确可靠的程序分析。数据流分析 (如 CSE 所依赖的可用表达式分析) 揭示了值是如何在程序中产生和传播的; 而

信息流分析（如 KnownBits 分析）则是在位级层面推断值的范围。这些分析保证了上述优化变换的可行性及可靠性。

从实际上讲，DSP 处理器指令集中有一些专门针对特定模式的指令设计，如乘加、向量加载/存储及多数据并行运算指令。优化需要具备识别指令模式的能力，能在合适的阶段将通用操作序列转化成高度优化的硬件操作命令。只有在分析与变换机制的支撑下才能使编译器既能保证通用性，又能生成高性能、高效率的目标代码。

3.1.2 关键优化技术分析

1. 公共子表达式消除

公共子表达式消除是一种经典的编译优化技术，其核心目标在于识别并消除程序中重复计算相同表达式的冗余操作。该优化能够通过避免冗余计算与减少不必要的寄存器占用，从而有效精简代码体积并降低执行延迟。在指令选择阶段，如果一个表达式的值已经被计算过，并且其操作数在后续代码中未被重新定义，则可以复用之前的计算结果，避免重复计算，从而减少冗余指令的产生。如下所示，在优化前 $b+c$ 被计算了 3 次，优化后只需要计算 1 次。

1	<code>a = b + c;</code>	1	<code>temp = b + c;</code>
2	<code>d = b + c;</code>	2	<code>a = temp;</code>
3	<code>e = b + c * 2;</code>	3	<code>d = temp;</code>
4	<code>f = (b + c) * 3;</code>	4	<code>e = b + c * 2;</code>
5		5	<code>f = temp * 3;</code>
6		6	

与传统的 CSE 不同，GlobalSel 使用了 Continuous CSE（持续 CSE）策略。Continuous CSE 会在指令创建阶段检查是否已经存在语义等价的表达式，一旦发现可复用的结果便直接进行替换，从而消除不必要的冗余计算指令。与基于全函数或全基本块扫描的传统 CSE 不同，Continuous CSE 主要作用于基本块内部，以增量式方式维护可消除的表达式集合，在不引入额外全局分析开销的前提下实现有效的冗余消除。这种局部的优化策略能够降低指令冗余，在保持优化效果的同时提升编译效率。

CSE 在消除冗余指令、降低寄存器使用压力的同时，也为后续优化创造了更多可能性。在指令合并优化阶段，复用公共子表达式的计算结果常常能带来更多的合并优化机会。例如，当多个计算共享同一子表达式时，合并优化可通过识别并整合这些冗余的计算模式，进一步简化指令序列，从而提升代码的整体执行效率与资源利用率。

2. 已知位分析

已知位分析属于一种静态推断寄存器各位已知状态的方法,通过向其他 Pass 提供已知位信息,以此支持后面各类优化变换,这项分析跟踪每个位的已知状态来帮助优化器做决策,核心目标是在不带来额外运行时开销的情况下,尽量提前拿到寄存器值的准确信息,进而辅助常量传播、指令简化以及后面合并优化决策。该分析借助对寄存器里必然为 0 或必然为 1 的比特位加以建模,为优化器给出细粒度的位级语义信息。

在具体的实现上,已知位分析需要为每个寄存器维护两个位掩码: KnownZeros 和 KnownOnes。其中 KnownZeros 用于标记寄存器中所有必然为 0 的位,相对应的 KnownOnes 则用于标记寄存器中所有必然为 1 的位。在指令流的传播过程中,这两个掩码会随着指令语义不断更新,使寄存器的比特级已知信息逐步得到完善。以下代码展示了已知位分析在逻辑运算中的效果。

```
1  int test(int a) {  
2      int b = a & 0xFFFF;    // 低16位保留,高16位必为0  
3      int c = b | 0x10000;    // 第17位必为1  
4      return c;  
5  }
```

编译器通过上述分析,能够推断出变量 b 的高 16 位恒为 0,变量 c 的第 17 位恒为 1。这种位级确定性信息在后续优化中具有重要价值:一方面可用于消除冗余的掩码、扩展或条件判断指令;另一方面也为指令合并和目标指令的匹配提供了更精确的语义约束,从而支持生成更紧凑、更高效的机器代码。

3. 合并优化

合并优化是一种将特定指令模式转换为更高效形式的编译优化技术。在 GlobalSel 框架中,合并优化是提升生成代码质量的重要手段。合并优化的目的是消除冗余的指令、简化表达式,并通过合并常见的低级操作模式来缩减代码规模、提升执行效率。合并优化可以充分利用目标架构的复杂指令,将多条简单指令合并为语义等价的单一指令,从而减少计算开销、降低寄存器压力,并减少对内存系统的访问次数。

合并优化也可以看作是 Peephole Optimization (窥孔优化)的一种推广,与传统窥孔优化局限于固定长度的相邻指令窗口不同,合并优化的作用范围显著扩大,它能够在更长的指令序列乃至跨基本块的范围内进行模式识别与优化重写,从而支持对更复杂、更分散的指令模式进行整合与简化。该优化策略的理论基础主要来源于以下几个方面的观察:

- 在低级代码生成过程中，为了便于表达中间计算结果，编译器会产生一系列指令结构简单但语义相关的指令序列，这会导致生成的指令存在冗余。目标架构如果有语义等价但更为复杂的单条指令形式，这些冗余的指令序列通常可以被这条高效指令所替代，从而减少了指令数量以及中间结果的产生。
- 指令之间的数据流局部性比较强。相邻的指令会共享操作数或者存在直接的定义和使用关系，这为编译器识别可合并的操作模式提供了天然条件，可以使合并优化在保持程序语义不变的情况下对一系列指令进行整体的重构。
- 随着现代处理器指令集日益丰富，乘累加、融合算术以及向量化等复杂指令日益普及。合并优化在通用中间表示与这些复杂目标指令之间发挥着关键衔接作用，也是影响编译器利用目标机器计算能力的紧要因素。

在 GlobalISel 中，合并优化贯穿于指令选择的各个关键阶段，持续对中间表示进行等价的模式重写和规约。该过程逐步引导中间表示向更符合目标架构特性的形式演进，从而在保持整体编译流程模块化的同时，有效提升最终生成代码的质量。其具体工作流程如图3-1所示。

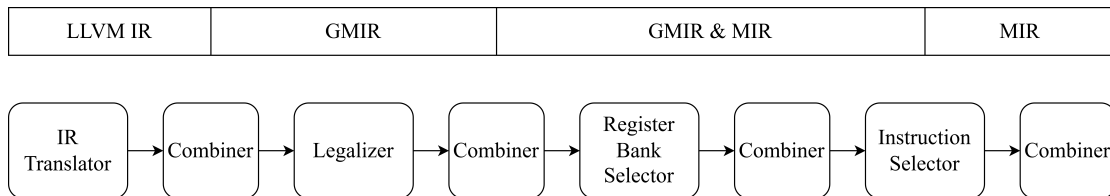


图 3-1 GlobalISel 合并优化流程示意图

3.1.3 面向 DSP 的优化设计

单一的优化策略难以应对复杂的代码生成需求，因此有必要构建一个多策略协同的优化体系。本章以合并优化为核心，结合公共子表达式消除、已知位分析及窥孔优化等多种技术，系统探讨全局指令选择阶段的综合优化方法。由于公共子表达式消除等通用优化在框架中已有成熟的实现，实际工程中仅需调用相关接口即可，因此下文将重点对合并优化的设计与实现过程进行详细论述。

为在 GlobalISel 框架中实现高效且可扩展的 DSP 优化方案，本章采用了一种混合式实现路径：结合 TableGen 声明式规则描述与 C++ 手动回调逻辑。具体而言，基于 TableGen 的 GICombiner 机制主要用于描述可合并指令的结构化模式。通过在 td 文件中定义 GICombineRule，能够以声明式的方式刻画指令之

间的拓扑关系、操作码组合以及操作数约束条件。TableGen 在编译期间会自动将这些规则转换为对应的匹配与执行代码，从而避免手工编写复杂的匹配逻辑，并保证了规则匹配过程的高效性与一致性。

另一方面，C++ 回调机制用于处理 TableGen 规则难以表达的复杂语义判断与非结构化指令变换。在合并优化过程中，许多决策依赖于：

- 操作数的位级属性推理（如 KnownBits 分析）。
- 目标硬件对特定操作或指令序列的实际支持能力。
- 变换结果在数据依赖、类型合法性和副作用方面的精细约束。

这类逻辑通常需要直接访问编译时分析结果并进行灵活的条件判断。因此对于每条合并规则，其核心的匹配验证与指令变换逻辑均由 C++ 代码实现：它依据匹配条件构造新的 GMIR 指令序列，并在此过程中严格维护虚拟寄存器的类型一致性、数据流关系以及指令间的依赖关系，从而确保优化变换的语义正确性与架构合规性。

仅采用基于 TableGen 的 GICombiner 难以准确表达复杂的语义依赖与分析驱动的优化条件；若完全采用 C++ 回调实现，则会导致优化逻辑分散在大量手写代码中，降低规则的可读性与可维护性。基于上述考虑，本文采用 TableGen 负责结构化模式描述、C++ 回调负责语义判断与指令重写的混合设计方案，在保证合并优化表达能力的同时，有效提升了工程可维护性与扩展性。

在实现层面上，合并优化规则首先以 GICombineRule 的形式在 td 文件中声明，并由 TableGen 自动生成对应的匹配执行器代码。随后，这些规则由 Combiner 框架在 MachineFunction 粒度上统一调度与执行，对 GMIR 指令流进行遍历和合并，从而在指令合法化前后逐步消除冗余指令、重构指令序列，并提升最终生成代码的执行效率。

在 GlobalISel 框架中，指令合法化阶段不仅构成代码生成流程中的关键功能边界，同时也是优化策略划分的重要依据。由于指令在合法化前后，其类型信息、操作数形态及目标相关约束的完备程度存在显著差异，编译器在不同阶段所能实施的优化类型与优化空间也随之发生变化。基于这一特点，本文在 DSP 后端中将优化策略按生效阶段进行系统化划分：一类面向通用 GMIR 表达，主要针对语义拆分冗余消除与表达式规范化，统一在 Pre-Legalize Combiner 阶段实施；另一类则依赖合法化后稳定的类型与操作数信息，侧重于目标相关指令匹配与代码质量提升，在 Post-Legalize Combiner 阶段完成。下文将结合具体优化实例，对上述两类优化策略的设计与实现进行详细分析。

3.2 合法化前优化策略

合法化之前，GMIR 仍处于高度通用的表示阶段，指令尚未被约束为 DSP 架构所支持的具体形式。该阶段的优化不涉及目标指令集的具体限制，重点在于语义层面的简化与结构性重写，以减少冗余并规范表达式形态，为后续 Legalizer 与 Instruction Selector 提供更加稳定的匹配条件，同时保留足够的优化自由度，主要包括以下几类：

- 表达式合并与规范化，如消除冗余的类型扩展 ($G_ZEXT(G_ZEXT(x))$)、合并重复的算术或逻辑操作，为后续指令选择创造更简洁的模式。
- 基于语义等价的指令融合，例如将 G_MUL+G_ADD 的通用表示提前重构为更利于匹配硬件乘加指令 MAC 的形式。
- 分析驱动化的优化，如利用 KnownBits、CSE 等分析结果进行常量折叠、冗余计算消除和条件简化。

由于此阶段允许暂时存在非法指令，优化变换不必立即满足目标指令集的约束，从而具备更大的自由度，能够在保持语义不变的前提下，对指令结构进行深度重构。

3.2.1 内存操作优化

内存操作指令的优化是 Pre-Legalize Combiner 阶段中的一种典型优化，主要针对通用 GMIR 中高层内存操作指令所引入的函数调用冗余问题。在 GlobalISel 的默认实现中， G_MEMCPY 、 $G_MEMMOVE$ 等高层内存操作通常会在合法化过程中被直接降级为对标准库函数的调用。这种处理方式在保证语义正确性的同时，也为不同目标架构提供了统一的实现路径，但在实际代码生成过程中并非始终高效。

在小规模内存操作场景下，函数调用本身所带来的调用约定维护、参数准备以及返回处理等额外开销，往往远大于实际内存访问指令的执行成本，从而造成明显的性能浪费。以 DSP 架构为例，在 GlobalISel 框架下以 O0 优化等级编译时，一次 `memcpy` 调用所生成的指令数量可达 83 条，其中函数本体占 80 条指令，额外的调用开销为 3 条指令。对于仅拷贝少量字节的数据而言，这样的实现方式显然并不经济。基于上述观察，本文在合法化之前引入了一种针对高层内存操作的合并优化策略：在满足一定约束条件的前提下，主动将小规模内存拷贝操作内联展开为显式的 `load/store` 指令序列，从而避免不必要的函数调用开销。

将该优化放置于 Pre-Legalize Combiner 阶段具有明确的设计考量。一方面，此阶段的 GMIR 仍保持较高的抽象层次，能够完整保留内存操作的语义信息与

对齐属性，便于判断是否适合进行内联展开；另一方面，在合法化之前进行转换，可以避免在后续阶段引入额外的调用指令及栈帧调整，从源头上降低指令数量与控制流复杂度，同时也有利于后续指令选择与调度优化。

在 DSP 后端的 Pre-Legalize Combiner 中，本文以 lowerMemcpy 为代表实现了该类优化策略，其整体流程如算法3所示。该算法首先综合源地址与目标地址的对齐信息，确定可安全使用的内存访问粒度；随后结合拷贝长度与预设阈值，判断是否存在可行的内联展开方案。当目标地址对应非固定栈对象时，算法还会在必要时对栈对齐进行调整，以提升后续内存访问指令的对齐程度。在满足上述条件后，原始的 G_MEMCPY 指令将被逐步替换为一组等价的 load/store 指令，并最终从指令流中移除。

3.2.2 拓展截断优化

1. 冗余截断与比较指令优化

在合法化前，编译器通常会引入形如截断 + 比较的冗余指令序列。这类模式通常表现为：先对一个宽位宽寄存器执行截断操作，之后再将截断结果与零进行比较。这种现象大多来源于高层 IR 中的类型收缩、布尔表达式转换以及通用指令的拆分策略等因素。

从语义角度来看，如果被截断寄存器的高位仅由符号扩展产生，那么这些高位在逻辑上并不包含有效信息。在这种情况下，对截断后的结果进行零比较，与直接对原始宽位宽值进行零比较在语义上是等价的。基于这一结论，本节在合法化前引入了针对冗余截断 + 比较模式的合并优化。

该优化首先利用 KnownBits 分析对被截断操作数进行位级信息推断，判断其高位是否完全由符号位扩展而来；若满足该条件，则说明截断操作并不会改变数值的符号与零值判定结果。此时，优化过程将原本以截断结果为操作数的比较指令，直接重写为对宽位宽操作数与零进行比较，同时移除多余的 G_TRUNC 指令。一个典型的示例如下所示。

1	%0:_(s64) = COPY \$d0	1	%0:_(s64) = COPY \$d0
2	%1:_(s32) = G_CONSTANT i32 0	2	%1:_(s32) = G_CONSTANT i32 0
3	%2:_(s32) = G_TRUNC %0:_	3	%3:_(s1) = G_ICMP slt, %0:_, %1:_
4	%3:_(s1) = G_ICMP slt, %1:_, %2:_	4	
5		5	

在经过这一优化后，编译器不仅减少了一条通用中间指令和对应的虚拟寄存器定义，还简化了数据流依赖关系，使比较操作更贴近硬件语义。在 DSP 架构下，该优化有助于后续指令选择阶段更直接地匹配硬件比较指令与条件跳转指令，从而提升代码密度并降低执行延迟。

Algorithm 3: lowerMemcpy: inline memcpy as loads/stores

Input: $MI, Dst, Src, Len, Limit$
Output: Legalized or UnableToLegalize

```

1  $Align \leftarrow \min(DstAlign, SrcAlign);$ 
2  $CanRealign \leftarrow \text{IsNonFixedFrameIndex}(Dst);$ 
3  $(DstMMO, SrcMMO) \leftarrow \text{ReadMemOperands}(MI);$ 
4  $MemOps \leftarrow \text{FindMemOpTypes}(Len, Limit, Align, CanRealign);$ 
5 if  $MemOps = \emptyset$  then
6   | return UnableToLegalize;
7 end
8 if  $CanRealign$  then
9   |  $NewAlign \leftarrow \text{ABIAlign}(MemOps[0]);$ 
10  |  $NewAlign \leftarrow \text{ClampToStack}(NewAlign);$ 
11  | if  $NewAlign > Align$  then
12  |   |  $\text{UpdateStackAlign}(Dst, NewAlign);$ 
13  |   |  $Align \leftarrow NewAlign;$ 
14  | end
15 end
16  $\text{InitBuilderAt}(MI);$ 
17  $Off \leftarrow 0;$ 
18  $Rem \leftarrow Len;$ 
19 foreach  $Ty \in MemOps$  do
20   | if  $Ty.bytes > Rem$  then
21   |   |  $Off \leftarrow Off - (Ty.bytes - Rem);$ 
22   | end
23   |  $LoadMMO \leftarrow \text{MakeMMO}(SrcMMO, Off, Ty.bytes);$ 
24   |  $StoreMMO \leftarrow \text{MakeMMO}(DstMMO, Off, Ty.bytes);$ 
25   |  $P_s \leftarrow \text{PtrAt}(Src, Off);$ 
26   |  $P_d \leftarrow \text{PtrAt}(Dst, Off);$ 
27   |  $V \leftarrow \text{EmitLoad}(Ty, P_s, LoadMMO);$ 
28   |  $\text{EmitStore}(V, P_d, StoreMMO);$ 
29   |  $Off \leftarrow Off + Ty.bytes;$ 
30   |  $Rem \leftarrow Rem - Ty.bytes;$ 
31 end
32  $\text{EraseInstr}(MI);$ 
33 return Legalized;

```

2. 拓展下推优化

在 GMIR 中, 算术指令往往作用于经过符号扩展 G_SEXT 或零扩展 G_ZEXT 后的操作数。这种形式虽然在语义上是正确的, 但在实际工程中会引入多余的扩展指令, 导致指令数量的增加及后续目标指令匹配与融合的成功率的降低。为此, 本文引入一种 EPO (Extension Pushing Optimization, 扩展下推优化), 优化通过重写算术运算与扩展指令的组合形式, 将算术操作下推至扩展之前, 从而获得更加规范、紧凑的中间表示。

考虑如下典型模式:

$$\text{add}(\text{zext } x, \text{zext } y)$$

该形式在 GMIR 中表现为两个扩展指令后接一条算术指令。由于 DSP 指令集中包含带扩展的算术指令, 允许在较小位宽上直接完成算术运算, 再对结果进行一次统一扩展。因此, 将算术操作推到扩展之前能够减少冗余的扩展指令数量, 降低指令条数。除此之外, 还能形成更规范的 IR 结构, 提升指令选择阶段的模式匹配成功率。优化针对 ADD 和 SUB 指令, 适用于零扩展与符号扩展两种情形, 其核心重写规则可概括为:

$$\text{op}(\text{ext } x, \text{ext } y) \Rightarrow \text{ext}(\text{op}(x, y))$$

其中, op 表示算术操作 ADD 或 SUB, ext 表示扩展操作 ZEXT 或 SEXT。值得注意的是, 在变换过程中必须保证扩展类型的一致性, 并在符号扩展场景下正确维护符号语义。在 Combiner 中, 该优化通过参数化的 TableGen 规则进行描述。其核心定义如下所示。

```

1  class epo_base<Instruction opcode, Instruction extOpcode>
2      : GICombineRule <
3      (defs root:$root),
4      (match (extOpcode $ext1, $src1):$ExtMI, (extOpcode $ext2, $src2),
5          (opcode $dst, $ext1, $ext2):$root,
6          [{ return matchEPO(*${root}, MRI, $dst, $src1, $src2); }]),
7      (apply [{
8          applyEPO(*${root}, MRI, B,
9              ${ExtMI}->getOpcode() == TargetOpcode::G_SEXT, $dst, $src1, $src2);
10         }])
11  >;

```

该规则通过模板参数抽象了算术操作类型 ADD/SUB 以及扩展类型 ZEXT/-SEXT, 从而避免重复定义多条相似规则。在匹配阶段, 规则要求两个操作数均来源于相同类型的扩展指令, 并由指定的算术指令使用; 在应用阶段, 根据扩展

类型生成新的算术指令，并在必要时重新插入对应的扩展操作。该优化在语义上是安全的，其正确性基于如下事实：

- 对于零扩展 ZEXT，扩展前后的值在无符号算术意义下保持一致。
- 对于符号扩展 SEXT，在保证算术运算不发生未定义行为的前提下，先运算后扩展与先扩展后运算在数学语义上等价。
- 优化过程中不改变数据依赖关系，仅调整指令组合方式。

EPO 属于一种 IR 规范化类优化，并不以直接降低程序的运行时间为目标，而是通过统一和规整中间表示结构，为后续目标相关优化与指令选择提供更优的输入形式。在 DSP 架构上，该优化提升了复合指令匹配的成功率，从而在代码生成阶段间接减少最终生成指令的数量。

3.3 合法化后优化策略

合法化之后，所有 GMIR 指令均已被转换为目标架构支持的合法形式，指令的类型、操作数位宽及内存访问方式均受到严格约束。此时优化的重点转向目标相关的性能优化与代码质量提升，主要包括以下几类：

- 目标指令匹配与替换优化，如将合法化后形成的指令序列进一步合并为 DSP 架构提供的伪指令或复合指令，以提升执行效率。
- 指令调度友好的局部合并，如通过消除多余拷贝、简化寄存器间的数据搬移来降低寄存器压力，为后续寄存器分配与指令调度创造更优条件。
- 面向代码密度和执行效率的微优化，如立即数折叠、访存地址计算合并以及针对硬件流水线特性的局部重写。

由于合法化后指令已经紧密贴合目标架构，这一阶段的优化虽在语义变换自由度上受限，但能够更直接地转化为实际的性能收益。

3.3.1 立即数装载优化

在指令选择阶段，G_CONSTANT 和 G_FCONSTANT 需要被转化为目标指令集可执行的装载立即数序列。对于 DSP 后端而言，指令采用固定 32 位编码格式。由于指令由操作码与操作数组成，且立即数字段的编码宽度通常不超过 16 位，单条指令中可用于表示立即数的位数受到严格限制，因此 32 位常量一般无法通过单条指令直接完成装载。因此，对于 32 位立即数，若不加区分地统一采用长立即数装载伪指令 MOVI，则会引入额外的指令开销。

针对上述问题，本文结合 DSP 平台的 ISA 特性，在全局指令选择阶段对立即数装载过程进行定制化优化。核心思想是根据常量的位型特征对立即数进行分类，并优先选择编码长度更短、执行开销更低的装载形式，从而在保证语义正确性的前提下减少冗余指令。以 32 位立即数为例，可将其划分为以下三类情况。

1. 立即数的高 16 位全 0

当立即数的高 16 位全为 0 时，立即数可视为一个零扩展的 16 位无符号数，只需要装载低 16 位即可。由于硬件指令集的限制，即使立即数的高 16 位全为 0，也无法省略对高位赋值的操作。为此本节提出一种基于零寄存器的装载策略，通过设置一个零寄存器并将零寄存器作为源操作数，结合立即数加法指令来完成立即数的加载，从而避免了使用通用的长立即数装载伪指令，对应汇编指令如下所示。

1	MOVIGH GR2 0x0;	1	ADDI ZERO 0x1;
2	MOVIGL GR2 0x1;	2	
3		3	

2. 立即数的高 17 位全 1

当立即数的高 17 位全为 1，即为可用 16 位有符号立即数表示的常量时，可以利用 MOVIGLX 指令来实现将一个 16 位的有符号立即数进行符号位扩展至 32bit 后复制到 rd 中。于是当常量满足 $\text{Imm} \in [-2^{15}, 2^{15} - 1]$ 时，可将 imm16 符号扩展到 32 位写入 Rd，对应汇编指令如下所示。

1	MOVIGH GR2 0xFFFF;	1	MOVIGLX GR2 0xF123;
2	MOVIGL GR2 0xF123;	2	
3		3	

3. 其他情况

对于不满足上述情况的立即数，由于无法通过单条指令完成装载，只能使用伪指令拓展的长立即数装载方式，伪指令会在后续阶段展开为 MOVIGH 和 MOVIGL 两条机器指令。其中 MOVIGH 装载立即数的高 16 位到寄存器高半区，而 MOVIGL 装载立即数的低 16 位到寄存器低半区。

上述优化策略在指令选择阶段完成对立即数形式的判定与映射，避免了在后续低层次优化中对立即数序列进行二次修正。通过对立即数位型的精细化分

类，该优化能在不增加指令选择复杂度的前提下，有效减少冗余指令的生成，从而为后续寄存器分配与指令调度提供了更紧凑的输入形式。该优化能够在常量使用频繁的程序中显著降低代码体积并提高整体执行效率。

3.3.2 乘法优化

乘法的硬件实现通常具有较高的延迟与开销，因此对于乘数为编译期常量的整数乘法，直接生成乘法指令往往不是最优选择。事实上，许多常量乘法在算术上可以等价地分解为移位与加减操作，从而以更低的执行成本完成相同计算。基于这一考虑，本文在合法化后阶段对 `G_MUL` 指令引入常量乘法强度削减优化。当乘数为满足特定形式的编译期常量时，编译器将原始乘法操作替换为由移位 `G_SHL` 与加法 `G_ADD` 或减法 `G_SUB` 组成的等价指令序列。

1. 理论基础

从算术等价关系来看，若常量 C 满足以下形式之一：

$$C = 2^n \pm 1 \quad (3.1)$$

或

$$C = (2^n \pm 1) \times 2^m \quad (3.2)$$

则乘法运算可以通过移位与加减运算等价实现：

$$x \times (2^n + 1) = (x \ll n) + x \quad (3.3)$$

$$x \times (2^n - 1) = (x \ll n) - x \quad (3.4)$$

$$x \times ((2^n \pm 1) \times 2^m) = ((x \ll n) \pm x) \ll m \quad (3.5)$$

这些变换在保持程序语义完全一致的前提下，能够显著降低指令执行代价，并减少对乘法硬件单元的依赖。

2. 匹配阶段

在匹配阶段，优化逻辑以 `G_MUL` 指令为目标，对其操作数及常量特征进行系统分析，整体匹配流程如算法4所示。具体而言，该阶段主要包括以下三个步骤：

1. 常量识别：通过常量传播与等价查询机制，判断右操作数是否为编译期常量，若满足条件，则获取乘法右操作数的常量值。

2. 位型分析：利用位运算分析常量的二进制结构，判断其是否满足上述形式。
3. 约束检查：为避免干扰后续指令融合或破坏已存在的扩展语义，匹配阶段还会检查源操作数是否仅被当前指令使用以及排除可能影响符号扩展、零扩展或寄存器重用的场景。

Algorithm 4: Constant-Multiply Strength Reduction

Input: MachineInstr MI
Output: InstructionMapping or fail

```

1 if  $opcode(MI) \neq G\_MUL$  then
2   | return fail;
3 end
4 if  $RHS$  is not an integer constant then
5   | return fail;
6 end
7  $C \leftarrow sext(const(RHS));$ 
8  $TZ \leftarrow countr\_zero(C);$ 
9  $C' \leftarrow ashr(C, TZ);$ 
10 if  $C' > 0$  then
11   | if  $(C' - 1)$  is power of two then
12     | Rewrite as  $(x \ll \log_2(C' - 1)) + x$ ;
13   | else
14     | if  $(C' + 1)$  is power of two then
15       | Rewrite as  $(x \ll \log_2(C' + 1)) - x$ ;
16     | end
17     | else
18       | return fail;
19     | end
20   | end
21 else
22   | if  $(-C' + 1)$  is power of two then
23     | Rewrite as  $x - (x \ll \log_2(-C' + 1))$ ;
24   | else
25     | if  $(-C' - 1)$  is power of two then
26       | Rewrite as  $-((x \ll \log_2(-C' - 1)) + x)$ ;
27     | end
28     | else
29       | return fail;
30     | end
31   | end
32 end
33 return success;

```


3. 重写阶段

在重写阶段，编译器根据匹配到的常量形式，生成对应的移位与加减指令序列，并替换原有的 `G_MUL` 指令。重写操作后，原本需多个周期完成的乘法操作被转换为仅由低延迟移位与加减指令组成的序列，在保持计算结果精确不变的前提下，显著降低了指令的执行开销，并减少了对专用乘法硬件单元的占用。

3.4 本章小结

本章针对 DSP 架构下 GlobalISel 的性能问题，系统分析并设计了多种全局指令选择优化策略。其中针对指令选择过程中产生的冗余指令与效率问题，本章针对 DSP 架构引入并实现了 CSE、KnownBits Analysis 以及 Combiner 等优化技术，用于减少重复计算、简化指令序列并提升硬件资源利用率，并对其理论基础与在不同阶段的应用方式进行了详细的分析。

本章提出的优化策略依据其对目标架构的依赖程度，可划分为合法化前与合法化后两个阶段。在具体技术手段上，通过公共子表达式消除减少冗余计算与寄存器占用，借助已知位分析提供比特级信息以支持常量传播与指令简化，并利用指令合并优化重构代码序列以增强执行密度。进一步地，结合 DSP 架构特性，本章引入了针对性的立即数装载优化与乘法相关优化，有效增强了指令生成过程对硬件约束的适配能力，从而在保障语义正确性的同时，提升了整体执行性能。

第 4 章 测试评估平台设计与实现

随着 DSP 编译器后端功能的不断扩展，尤其是全局指令选择框架及其相关优化方法的引入，编译器在指令匹配、寄存器组的选择、合法化等阶段的行为对输入越加敏感，以及组合关系也越加复杂。尽管全局指令选择在实现上比基于 DAG 的指令选择方法更加模块化，但其指令选择结果由多个 Pass 的协同决策共同决定，这导致整个过程对输入的程序结构更加敏感。

DSP 工具链已经建立了较为完整的 CI (Continuous Integration, 持续集成) 测试流程，能够较好地保证主分支代码的功能正确性。但是该 CI 流程主要面向传统编译流程构建，其设计目的是保证功能的可用性，没有针对全局指令选择及其对应优化在整个复杂流程中的正确性和性能稳定性做到有针对性的保证。随着 GlobalISel 在 DSP 工具链中的落地，现有 CI 测试在测试结果可分析性、性能稳定性判断、复杂输入场景覆盖上暴露出问题，需要对相关问题进行总结分析。

针对上述问题，本文设计并实现了一套面向 DSP 架构的随机测试评估平台。其核心创新体现在以下两方面：

1. 突破通用随机测试工具对专用架构的适配局限，通过定制扩展 YARPGen 实现 DSP 内建函数的自动化生成，强化对 DSP 专用指令与硬件特性的测试覆盖。
2. 构建了性能数据生成 - 采集 - 存储 - 分析 - 展示的全链路管理体系，实现对编译器性能指标的系统化采集与可视化分析。

最终形成覆盖生成 - 编译 - 执行 - 评估的全流程自动化闭环，为 GlobalISel 场景下 DSP 编译器的优化验证与迭代维护提供核心支撑。

本章将围绕现有测试流程缺陷分析、随机测试工具定制适配、平台架构设计与核心模块实现三方面展开论述，构建适配 GlobalISel 场景的 DSP 编译器专项测试评估体系，为后端优化策略的有效性验证与工具链迭代维护提供可靠支撑。

4.1 DSP 编译器测试现状与改进

4.1.1 测试流程现状

在当前开发流程中，开发者提交代码后，CI 系统会自动触发预设的测试流程，对最新版本的编译器进行编译与功能验证。所有测试项通过且经项目负责

人审核批准后，相应代码方可合并至主分支。该测试流程在当前开发阶段具有关键作用：一方面，它能在代码提交早期发现编译错误、链接问题及显著的功能缺陷，从而保障主分支代码的基本正确性；另一方面，标准化的 CI 流水线实现了测试过程的自动化与统一化，减少了人工介入，有效降低了开发与维护成本。

总体而言，现有的测试体系已具备验证功能正确性的基本能力，能够有效发现明显错误和功能缺陷，阻止重大问题进入主分支。然而，该体系主要以测试是否通过作为判断标准，缺乏对编译器优化效果、性能变化以及复杂场景下系统鲁棒性的评估能力，难以满足 DSP 工具链对高性能和高可靠性长期发展的需求。具体体现在测试用例覆盖率不足以及测试结果的可读性和分析能力较弱这两个问题上。

1. 测试集来源单一，场景覆盖局限于常见路径，缺乏随机化测试机制

测试用例的覆盖完整性直接决定了 DSP 工具链发现潜在缺陷的能力。然而，现有 DSP 工具链在测试集构成与场景覆盖方面仍存在明显不足，难以对核心编译流程的鲁棒性进行充分验证。从测试来源来看，当前测试集主要包括以下三类：

- 基于库函数的手写测试用例：围绕基础功能验证设计，重点覆盖 C 标准库、数学库以及运行时支持库等核心库函数的调用场景。
- 移植的开源 Benchmark 程序：以通用计算或嵌入式系统性能评估为目标，用于衡量工具链在典型应用负载下的性能表现。
- 已知缺陷的最小复现样例：开发过程中针对已暴露的问题提取出的最小测试用例，用于回归测试，防止同类缺陷再次引入。

上述测试用例在验证工具链基础功能正确性以及复现已知缺陷方面具有一定价值，但其整体覆盖范围仍然较为有限。主要原因在于测试用例的设计高度依赖人工经验，多集中于常规的库函数调用路径与标准输入输出场景，对于极端数据类型、异常输入组合、复杂控制流结构以及并发与实时性等边界条件缺乏系统性的覆盖。

这种覆盖不足会直接引发两类工程风险。第一，工具链在典型应用环境中表现正常，但在特定的应用场景或极端条件下可能出现逻辑错误或功能异常。第二，编译器内部条件引发的缺陷难以提前检测，这类缺陷通常仅出现在特定的指令组合或数据模式下，例如指令选择阶段的模式识别错误、寄存器分配中的资源冲突，或是优化转换过程中产生的语义偏差。如果这些问题在测试阶段未能被及时发现，就可能在实际运行环境中引发崩溃或性能异常，造成严重后果。

对于采用 GlobalISel 的 DSP 编译流程而言,上述问题进一步被放大。由于指令选择结果往往由多个 Pass 的协同决定,例如寄存器组选择与指令合法化之间的相互影响、不同合法化路径对后续指令匹配结果的约束关系等,这类行为通常只会在特定输入结构和数据特征组合下才会显现。依赖人工经验构造的测试样例难以覆盖这些复杂交互场景,导致部分潜在缺陷在早期测试阶段无法暴露出来,直至在实际的 DSP 应用中才被触发,从而引发较高的工程风险和后期维护成本。

2. 测试产物可读性与分析能力不足

现有测试体系在测试结果的表达和分析能力方面同样存在明显不足。目前的 CI 流程只是把测试用例是否通过作为判定标准,产出的测试结果缺乏对编译器后端行为的量化描述,未系统记录程序执行周期、代码体积、指令打包效率等关键性能指标,也未对生成的汇编代码进行结构化管理。这使得测试结果只能用于验证功能正确性,而无法反映全局指令选择及相关优化策略对编译器性能造成的实际影响。

除此之外,测试结果既没有做历史化存储,也没有做跨版本管理,这导致开发人员难以对不同提交之间的性能变化进行对比分析,也无法从整体上观察工具链性能随时间演进的趋势。随着 CI 测试记录不断积累,大量中间提交和实验性版本产生的结果进一步加剧了测试数据杂乱的问题,显著提高了有效信息筛选和问题定位的难度和成本。

综上所述,现有测试体系在输入覆盖能力和结果分析深度方面存在明显局限,难以支撑全局指令选择框架下编译器后端行为的系统性验证,已无法满足 DSP 工具链在高性能和高可靠性方向持续演进的需求。

4.1.2 测试体系优化需求与技术路径

基于 4.1.1 节的分析可知,随着全局指令选择框架及相关优化策略在 DSP 工具链中的实现,单纯依赖人工构造测试样例,以样例通过与否这一单一判定标准,难以支撑对编译器后端行为的系统性验证。为了确保全局指令选择在复杂场景下的功能与性能的稳定,现有测试体系需从测试输入覆盖与测试结果分析深度两方面进行针对性优化。

现有测试体系需提升测试用例的覆盖广度与复杂场景的模拟能力,以充分验证全局指令选择框架中基于条件触发的各类行为。由于指令选择、寄存器组绑定和合法化等阶段的决策往往依赖于输入程序的控制流结构、数据分布以及运算模式,仅仅依靠人工设计的测试样例难以系统覆盖所有潜在组合路径。因此,有必要引入随机测试生成机制,通过自动化方式生成大量具有结构多样性和数

据随机性的测试程序，从而覆盖人工测试难以触及的边缘场景。这种随机测试方式能够有效暴露隐藏在特定输入条件下的潜在缺陷，提升编译器后端对复杂输入场景的整体鲁棒性。

全局指令选择及其相关优化策略的引入，往往不会直接导致功能错误，而是可能以性能退化、代码体积膨胀或指令级行为异常等形式体现出来。为准确评估其对 DSP 架构性能的影响，有必要构建一套能够自动采集、存储和分析性能指标的评估平台，对程序执行周期、代码体积及指令级行为进行量化记录，并支持跨版本的对比分析和趋势观察。通过将测试结果结构化存储并以可视化方式呈现，可有效降低性能分析门槛，辅助开发者快速定位性能回退和异常变化。

综上所述，面向全局指令选择框架的 DSP 工具链测试体系，应当以随机测试驱动的高覆盖输入验证为基础，以性能指标采集与可视化分析为核心支撑，构建一套覆盖广、可量化、可追溯的测试评估机制。针对这一目标，本文在后续章节中设计并实现了一套集随机测试生成与性能评估于一体的测试评估平台，该平台可有效支撑全局指令选择及其优化策略的功能验证与性能量化分析，为技术方案的落地提供系统化保障。

4.1.3 随机测试生成技术与生成器选型

随机测试生成技术是自动化测试的核心技术之一，其核心原理是使用随机或伪随机算法，按照预设的语法和语义规则生成大规模的测试输入，以此验证软件系统在不同输入场景下功能的正确性、运行的稳定性以及整体的鲁棒性。和传统的人工编写测试用例的方式相比，随机测试能够在更短时间内覆盖更广的输入空间，有利于发现隐藏较深、难以通过人工经验判断的边界错误和组合缺陷。

在编译器测试中，随机测试技术通常借助专用的随机测试生成器来实现。这类工具会随机组合表达式、控制流结构、数据类型以及运算符等语言构造要素，来生成大量结构各异的测试程序，并将其作为输入交由编译器处理，从而验证编译器前端解析、优化过程以及后端代码生成等多个阶段的正确性与健壮性。

在编译器随机测试工具的发展进程中，CSmith^[53]是较早在业界得到广泛应用的 C 语言随机程序生成器，其目标是发现编译器在处理常规 C 程序时可能存在的逻辑错误。CSmith 的特点是生成的随机程序不仅严格遵循 C99 标准，还会主动规避 UB（Undefined Behavior, 未定义行为，代码执行结果在编程语言标准中未被明确规定的行为），从而保证测试结果可复现。

CSmith 在验证编译器的基础语义一致性、防范功能性错误等方面起到了关键作用，但受限于设计目标，它的测试重心集中于功能正确性的校验，对于编译器在高性能优化场景下的行为，覆盖能力存在明显局限。

随着现代处理器架构日趋复杂，编译器的优化策略也在持续迭代升级，仅依

靠功能正确性验证已难以满足编译器工具链的测试需求。在这一背景下，YARPGen (Yet Another Random Program Generator)^[54-55]应运而生，它的设计初衷正是针对 CSmith 在性能相关测试方面覆盖不足的问题，重点面向编译器中后端的性能优化阶段开展随机化测试。它可以在不依靠动态检查的情况下，使用生成规则来生成没有未定义行为的表达式，同时提升了生成代码的多样性，触发更多编译器优化的可能。在代码生成策略上，YARPGen 侧重构造密集型循环、数组连续访问、数据并行计算与向量运算等典型代码模式，从而有针对性地覆盖循环向量化、指令级并行、缓存优化以及指令调度等编译器后端的关键优化阶段。

从语言兼容和架构适配上来看，YARPGen 不仅原生支持 C 语言，还支持 C++ 的部分核心特性，并能够生成面向 SIMD、向量指令等现代处理器架构扩展的针对性代码。这一特性使其在测试面向高性能计算的嵌入式 DSP 场景时具有一定的实用价值。

基于上述随机测试生成器的特点，本文在测试体系的设计中，选择了 YARPGen 并结合 DSP 架构特性对其进行定制化扩展。通过生成含 DSP 内建函数集的随机测试程序，测试体系能更高效地验证编译器行为，为后续性能评估与回归分析提供可靠的测试基础。

4.2 平台设计

4.2.1 平台整体设计

为解决 4.1.1 节提到的测试覆盖不足、结果分析能力弱等问题，本文设计并实现了一套面向 DSP 编译器的性能测试评估平台。平台整体分为测试子系统与评估子系统两大模块，前者在原有 CI 测试平台上进行功能新增与架构重构，后者则为全新搭建的独立模块。

测试子系统的核心改进在于移植并定制随机测试生成器 YARPGen，拓展其功能，添加适配 DSP 指令集架构的 Intrinsic（内建函数）集，同时完成适配现有 CI 流程的环境配置。该子系统在每次测试触发时可自动生成一批随机样例，并与原有固定测试集并行执行，同时将测试过程中检出错误的样例纳入固定测试集，形成迭代优化的测试用例库。

评估子系统对接线上 CI 测试平台，通过脚本实现编译流程的自动化触发。在开发人员提交 PR 后，脚本会自动触发测试任务，同时在部署在 Runner 上的仿真器和芯片上执行；测试完成后，结果会同步至 Gitlab 的 Perf 仓库；后端系统定时拉取 Perf 仓库并更新数据库，当前端发起请求时，系统会返回查询结果并在前端进行可视化展示，以此直观地呈现新变更引入的性能提升或退步。

系统整体设计如图 4-1 所示，整体架构分为测试子系统和评估子系统两部分。

测试子系统完成从测试样例生成、编译到执行的全自动化流程；评估子系统则通过前端、后端与数据库的协作，实现测试结果的结构化存储、查询、对比分析与可视化展示。两个子系统功能上解耦，通过 Perf 仓库实现数据交互。这种设计既保证了测试流程的高效运行，又提升了测试结果的可分析性与可追溯性。

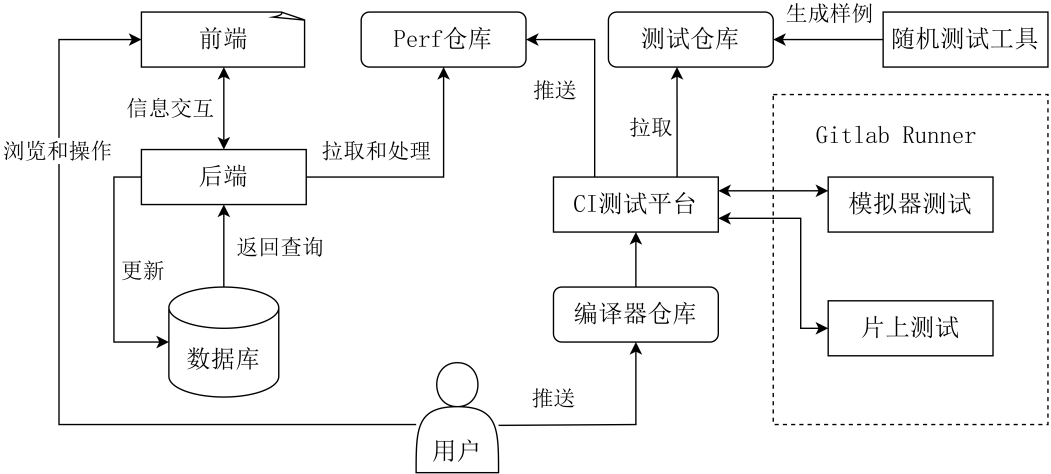


图 4-1 系统整体设计图

4.2.2 测试子系统设计

测试子系统的核心目标是在 CI 环境下构建一套全自动化的测试流水线，覆盖从测试用例生成、编译执行到结果归档与分析的完整流程。该子系统不仅用于验证编译器功能正确性，还承担着对编译器性能变化进行长期、稳定跟踪的任务，为后续的性能评估与回归分析提供可靠数据基础。

从功能上看，测试子系统主要由随机测试样例生成模块、编译与测试模块以及结果处理模块三部分构成，如图4-2所示：

- 随机测试样例生成模块：随机测试工具用于自动生成适配 DSP 架构的随机测试样例，重点覆盖密集循环、向量运算等性能相关场景。随机测试工具在执行测试时再生成，并将错误的样例推送至测试仓库，与其他测试样例一起被统一管理。二者共同为 CI 测试提供稳定的用例来源。
- 编译与测试模块：CI 测试平台作为调度核心，从编译器代码仓库中拉取开发者提交的最新 DSP 工具链代码，并从测试仓库中获取待执行的测试脚本、测试样例以及需要的测试工具。首先进行编译，编译成功后将测试任务分发至 Gitlab Runner，触发两类测试流程：一类是在 DSP 架构模拟器上执行的测试，用于快速验证功能正确性及基础性能表现；另一类是在实际 DSP 硬件芯片上执行的测试，用于发现硬件上的 Bug，以及获取更加准确的性能指标。

- 结果处理模块：测试完成后，CI 测试平台会将所有测试结果统一归档，并推送至 Perf 仓库。归档内容不仅包括测试通过或失败状态，还涵盖性能指标数据以及生成的汇编代码，为后续评估分析提供完整、可靠的数据基础。除此之外，为了解决 4.1.2 中提到的冗余问题，设置过滤器来保证只有主分支的代码才会传到 Perf 仓库中。

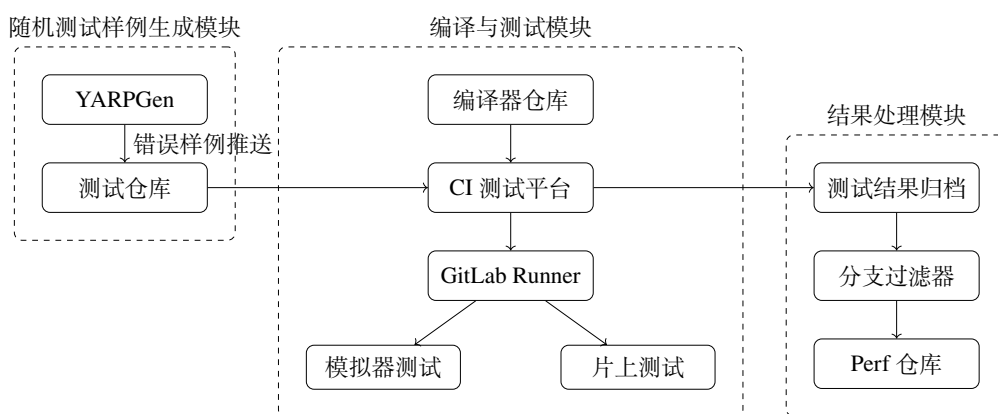


图 4-2 测试子系统整体结构与功能模块设计图

4.2.3 评估子系统设计

评估子系统的任务是实现测试结果的高效利用与直观呈现，采用前端、后端与数据库协同工作的三层架构，如图 4-3 所示，实现对 CI 流水线输出性能数据的统一接入、持久化存储与多维度分析。其设计目标主要包括以下三点：第一，将分散的测试产出进行结构化处理，构建可高效检索的存储体系；第二，提供标准化、稳定可靠的数据查询接口，支持跨编译器版本的性能历史对比与长期趋势分析；第三，将分析结果以交互式可视化形式呈现，从而提升性能回归问题的定位精度与诊断效率。系统的具体模块与流程如下：

- 后端模块负责从 Perf 仓库拉取性能报告，对原始数据进行结构化解析并将数据存储至数据库；同时，后端对外提供统一的数据接口，支持测试结果查询、跨版本对比以及性能趋势分析等功能，为前端的可视化展示提供服务支撑。
- 数据库模块作为测试结果的持久化存储中心，统一保存所有历史测试数据，支持按照芯片型号、Commit 号、优化等级、测试集以及测试样例等多维条件进行快速检索。这一设计为性能回归分析、版本对比以及长期趋势观察提供了坚实的数据基础。

- 前端模块则为用户提供直观、交互友好的可视化界面。用户可以通过前端查看单个提交版本的测试结果，对比不同提交之间的性能差异，或以时间序列方式观察性能变化趋势等。同时，前端支持灵活的数据筛选操作，例如仅查看片上测试的性能指标，相关请求由后端实时处理并从数据库中返回结果进行展示。

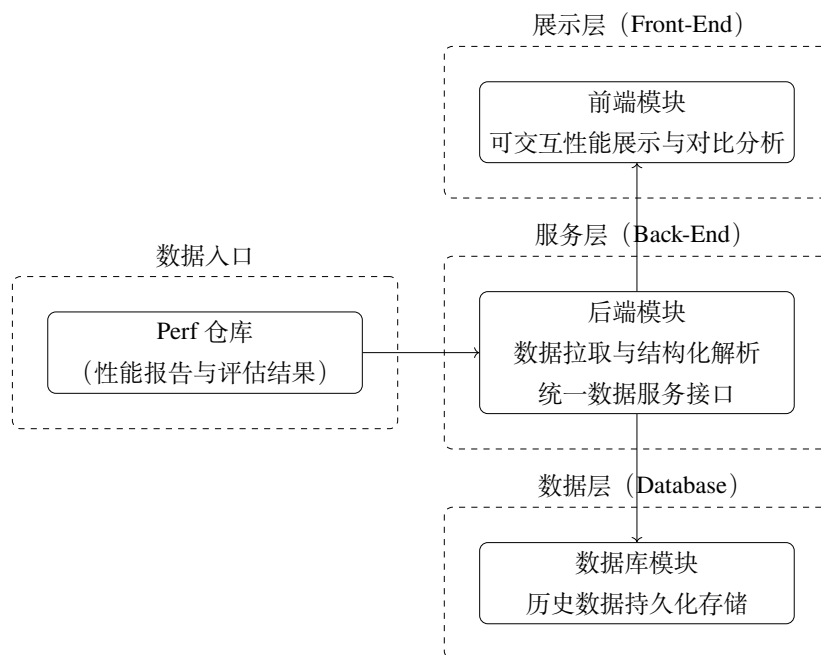


图 4-3 评估子系统整体结构与三层架构设计图

4.3 平台实现

4.3.1 面向 DSP 的 YARPGen 实现

YARPGen 是通用的编译器测试用例生成工具，其默认设计面向通用 CPU 架构及具备标准运行时环境的 C/C++ 程序。它假定目标平台具备标准库的支持，并可依赖 stdio 等通用接口完成程序结果的输出与验证。但 DSP 平台在硬件特性、调试机制以及运行环境方面均与通用处理器存在显著差异，这使得原生 YARPGen 难以直接应用于 DSP 编译器流水线的自动化验证。

为了使随机测试生成机制能够适配到 DSP 编译器后端，本节对 YARPGen 进行了面向 DSP 架构的定制化扩展。整体修改遵循可复用和可扩展的设计原则，在保留 YARPGen 原有随机生成能力的基础上，新增了对 DSP 平台特有硬件约束与运行时环境的支持。具体改动体现在以下几个方面。

1. 引入 DSP 专用调试头文件

DSP 编译器的测试与验证依赖架构专用的调试与输出接口，而非标准的 stdio 输出机制。为保证生成的随机测试程序能够在 DSP 环境中正确输出运行结果，需在测试用例中引入特定头文件以支持调试输出和结果对比。在 YARPGen 的 emitCheckFunc 阶段，通过条件编译区分通用平台和 DSP 平台，为 DSP 环境添加专用调试头文件，并声明调试输出相关的函数接口，对应函数如下所示。

```

1 void emitCheckFunc(std::ostream &stream) {
2     std::ostream &out_file = stream;
3     out_file << "#ifdef DSP_VALIDATION\n";
4     out_file << "#include <swift_debug.h>\n";
5     out_file << "#else\n";
6     out_file << "#include <stdio.h>\n\n";
7     out_file << "#endif\n";
8     ..... }

```

其中，DSP_VALIDATION 用于标识当前测试是否运行于 DSP 验证环境。该宏由测试流水线在编译阶段统一注入，从而保证同一套测试生成逻辑可同时服务于通用平台与 DSP 平台，避免代码分叉维护。通过该方式，YARPGen 在 DSP 模式下能够调用调试头文件中提供的调试输出接口，实现与 DSP 仿真器或硬件调试环境的对接。

2. 添加 DSP 专用调试输出逻辑

在通用平台中，YARPGen 通过标准输出对程序执行结果进行校验；而在 DSP 平台上，测试结果需要通过专用调试通道输出，以便测试框架进行采集与比对。为此，在 YARPGen 的 emitMain 阶段中，通过条件编译为 DSP 环境下添加调试函数调用，用于输出校验结果。

3. 引入目标架构选项与条件编译参数

YARPGen 的原生版本仅区分生成语言，而不会区分目标后端架构。为了支持 DSP 架构的相关逻辑，本文在 YARPGen 中引入了目标架构选项，并通过条件编译参数来控制测试生成与代码发射阶段的具体行为。这一改动为后续引入 DSP 架构的 Intrinsic 集提供了基础设施支持。

4. 添加 DSP 架构的 Intrinsic

为高效测试 DSP 编译器的行为,尤其是指令选择阶段对 Intrinsic 的处理,本文在 YARPGen 中引入了 DSP 架构的 Intrinsic 集,并将其纳入随机测试生成范围。由于 YARPGen 原生不支持 Intrinsic,于是需要拓展其功能,在 IRNode 中引入新的 Intrinsic 选项,并根据操作数数量进行分类,如图4-4所示。以 SIN 函数为例,对 YARPGen 的 IRNode 生成逻辑进行了扩展。

- **createHelper**: 核心作用是创建 SIN 函数调用表达式,具体包括生成算术表达式参数、处理布尔类型转换,最终返回封装后的 IntrinsicExpr 指针。该函数是 YARPGen 中表达式生成逻辑的核心组件,主要用于随机生成包含 SIN 函数调用的测试用例。
- **propagateType**: 核心作用是对 SIN 函数的两个参数表达式进行类型推导、类型提升和类型统一,最终确定 SIN 函数调用表达式的返回值类型。
- **evaluate**: 在测试生成或运行时阶段计算 SIN 函数调用的结果,具体包括处理参数的类型转换、未定义行为判断,以及最终返回计算后的标量值。该函数是 YARPGen 中表达式求值逻辑的核心组件,主要用于确定 SIN 函数调用的实际结果值。
- **emitCDefinitionImpl**: 核心作用是为 C 语言环境生成 SIN 函数的宏定义实现。因为 C 语言标准库中没有通用的 SIN 函数,该方法通过输出 C 语言的语句表达式宏,为 SIN 提供兼容 C 语言的实现,保证生成的测试用例在 C 环境下能正常编译运行。通过上述的定制化扩展,YARPGen 被成功适配到 DSP 编译器的流水线测试场景中。基于条件编译的设计使得:同一套随机测试生成逻辑可同时服务于通用平台与 DSP 平台;DSP 专用 Intrinsic 能够被系统性纳入随机测试覆盖范围。

对应的类层次结构如图4-5所示。

4.3.2 测试子系统实现

测试子系统在保留原有功能的基础上,主要实现了两方面的扩展:一是引入对编译器性能指标的自动化采集能力;二是集成面向 DSP 架构定制的 YARPGen,并将完整的测试流程纳入 CI。相应地,系统实现主要包括性能指标测量支持的引入以及测试脚本逻辑的重构两个方面。

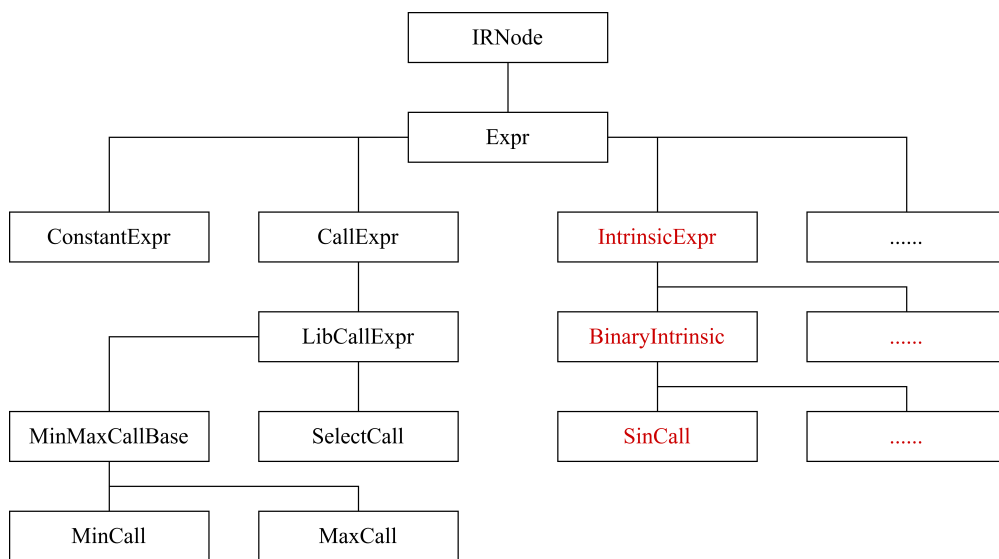


图 4-4 拓展后的 IRNode 结构图

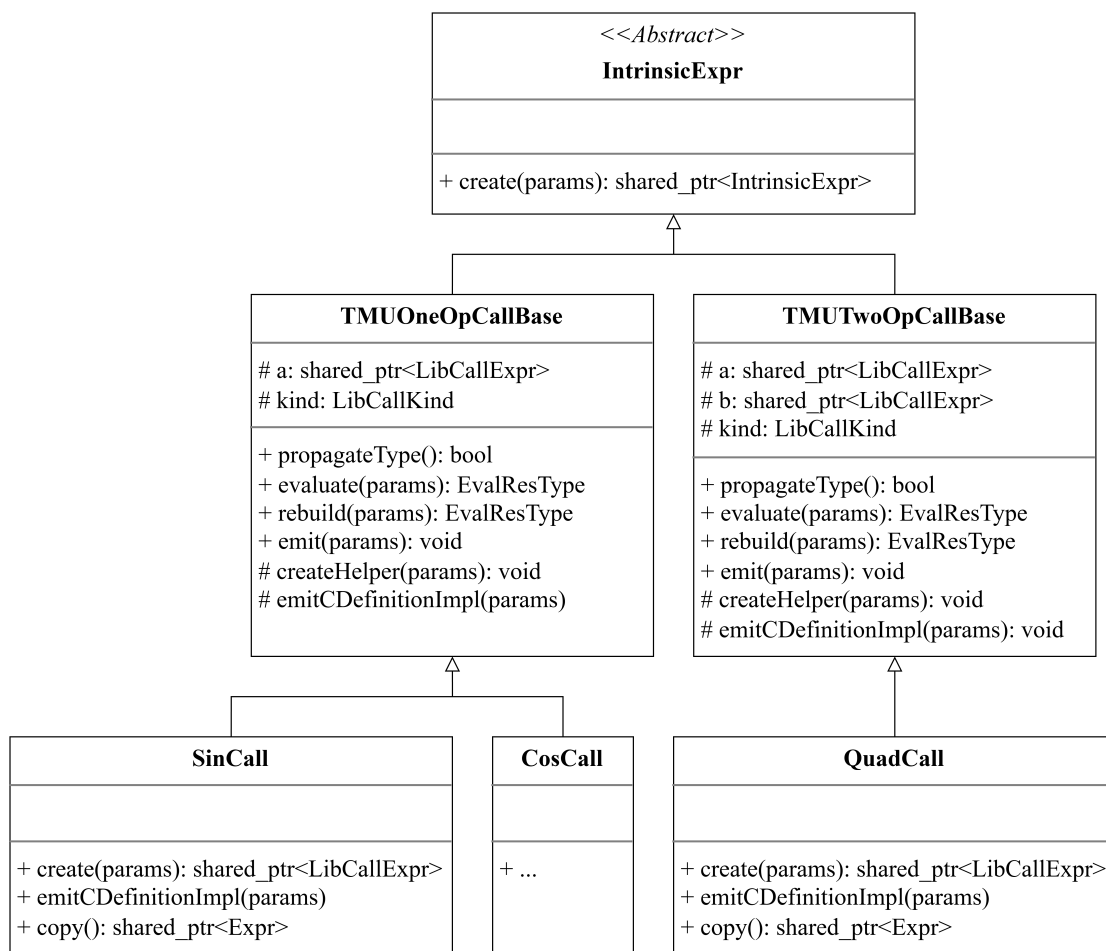


图 4-5 扩展后的类结构

1. 添加测量性能指标的支持

在对 VLIW 架构 DSP 编译器进行性能评估时，通常关注以下几项核心指标：程序运行周期数、生成代码体积、汇编代码质量以及指令打包效率。这些指标分别从执行效率、存储占用、代码可读性以及指令级并行优化能力等维度，综合反映了编译器后端在目标平台上的性能表现。为准确获取上述不同维度的性能数据，系统采用了差异化的采集策略。

- 代码体积的测量通过 DSP 工具链中的 `llvm-size` 完成，工具通过对编译完成后的目标文件进行分析，来自动统计生成代码所占用的存储空间。
- 汇编代码的获取则依赖于 DSP 工具链中的反汇编器，对编译完成后的目标文件进行反汇编，生成对应的汇编输出，为后续的指令级分析提供基础数据。
- 指令打包效率由 DSP 工具链的模拟器进行统计，模拟器能够在程序执行过程中收集指令发射和打包相关信息，从而反映编译器在指令调度与并行化方面的优化效果。
- 运行周期数的获取在实现上具有一定复杂性。在模拟器环境中，程序运行周期数可直接由模拟器在程序执行过程中提供；而在实际 DSP 芯片上，则需依赖底层驱动库所暴露的硬件计数器接口进行测量。为将周期数测量无缝集成至自动化测试流水线，平台对原有驱动库进行了重构与精简，将其提取为独立、可重定向的模块，仅保留初始化与时间戳读取两个核心函数。在测试程序的链接阶段，该模块被统一链接至主函数中，从而实现对程序执行前后时间戳的自动采集与周期数计算。通过这一方式，芯片上的运行周期数测量能够与编译、执行流程紧密结合，实现流水线化和自动化。

2. 重构测试脚本并添加 YARPGen

原有的测试脚本逻辑仅关注测试样例的功能正确性验证，无法满足性能评估平台对多维性能数据采集的需求，为此，需要对测试脚本的整体逻辑进行重写和扩展。在新的脚本中，测试流程在完成编译与执行后，会自动触发性能数据采集模块，分别获取运行周期数、代码大小、汇编输出以及指令打包效率等指标，并将其统一整理为结构化的性能报告。

性能报告以标准化 `Json` 格式生成，便于后续自动解析和存储。在报告生成完成后，测试脚本会将测试结果进行打包，并自动上传至 `Perf` 仓库，作为后续评估和可视化分析的数据源。

此外为了提升测试覆盖度，CI 脚本中还集成了 YARPGen 的随机样例生成逻辑，在每次测试触发时可自动生成一批新的随机测试程序，并与已有测试样例共同参与测试执行。通过这一机制，测试流水线不仅能够验证固定测试集上的功能和性能表现，还能够持续引入新的输入场景，提升对潜在缺陷的发现能力。

4.3.3 评估子系统实现

评估子系统用于对测试阶段产生的大量性能数据进行集中管理、分析与展示，其实现采用典型的三层架构，包括前端展示层、后端服务层以及数据库存储层。各层在功能上相互解耦，在数据流转上通过标准化接口进行协作，从而保证系统的可扩展性与维护性。

1. 数据展示层的实现

数据展示层（下文简称前端）面向编译器性能分析与回归验证的实际需求，用于解决性能数据可视化难、跨版本对比成本高的问题。前端部分基于 Vue 框架实现交互界面，并结合 Apache ECharts 实现性能数据的图形化展示。通过前端界面，用户可以直观地查看单个提交版本的测试结果，对比不同提交之间的性能差异，并以时间序列形式观察性能指标的变化趋势。可视化展示方式有效降低了性能分析门槛，使开发者能够快速定位性能回退或异常波动问题。在实现过程中，前端通过组件化方式对不同功能模块进行拆分，具体模块如下：

- **Dashboard**：从宏观上展示 DSP 芯片与其他芯片在典型测试集上的性能对比结果，用于快速评估工具链性能水平。
- **Summary**：基于完整测试集统计不同提交版本下的平均代码大小与执行周期，用于观察性能随时间的整体演进趋势。
- **History**：针对单个测试样例，以时间序列方式展示其在不同提交下的性能变化，用于精确定位性能回退首次引入的位置。
- **Compare**：对任意两个提交版本进行逐样例性能对比，并支持汇编代码对照分析，用于解释性能差异的底层原因。
- **Custom**：支持上传本地测试结果并与主分支数据进行对比，便于在本地优化验证阶段提前发现潜在性能问题。

前端通过调用后端提供的 REST 接口获取性能数据，并在本地对数据进行初步处理和筛选，以减少不必要的数据传输，前端部分核心 API 如表4-1所示。性能数据的可视化展示采用 Apache ECharts 实现，针对不同分析场景选用合适的

图表类型：例如使用柱状图展示与其他芯片在相同测试集下执行周期数或代码大小上的对比结果，使用折线图展示执行周期数及代码大小随提交版本变化的趋势情况。通过使用这种方式，前端能够直观反映性能变化特征，帮助开发者快速识别性能回退或异常波动。

表 4-1 前端部分核心 API 列表

接口名称	请求方式	功能描述
getOptLevelOptions	GET	获取编译优化级别选项
getChipOptions	GET	获取目标芯片选项列表
getSuiteOptions	GET	获取测试集选项列表
getCaseOptions	GET	获取测试用例名称选项列表
getAllCommits	GET	获取所有代码提交版本信息
getCommitByHash	GET	根据哈希值查询单个提交的详细信息
getHistoricalStatisticalInfo	POST	查询指定条件下测试集的历史统计信息
getLatestPerfOfSuite	POST	查询指定条件下测试集的最新性能数据
getHistoricalPerf	POST	查询单个测试用例的历史性能趋势数据
getTwoCommitsComparison	POST	对比两个提交版本的测试用例性能差异
getAssemblyComparison	POST	对比两个提交版本的汇编代码差异
uploadLocalReport	POST	上传本地性能测试报告并解析
removeAllCustomReports	GET	清空所有已上传的本地自定义测试报告

2. 业务逻辑层的实现

业务逻辑层（下文简称后端）是评估子系统的核心控制单元，负责性能数据的自动获取、解析建模以及对外接口服务。后端部分基于 **Spring Boot** 框架实现，负责性能数据解析、业务逻辑处理以及接口服务等任务。后端在启动或触发定时任务时，会从 **Perf** 仓库中拉取测试结果，并按照预定义的数据格式对原始 **JSON** 数据进行解析。解析过程中，后端会将测试环境信息（如目标处理器、优化等级、测试平台）与单个测试样例的性能指标进行关联建模，从而形成结构化的数据对象。这种建模方式使得不同维度的性能数据能够被统一管理，为后续的多条件查询和对比分析提供基础。

随后，这些数据通过 **MyBatis** 持久层框架映射到数据库表结构中，实现性能数据的持久化存储。为保证在多用户查询或批量数据写入场景下系统的稳定性，后端引入 **Druid** 数据库连接池，对数据库连接进行统一管理与复用，有效降低了连接创建和释放的开销。

在接口设计方面，后端向前端提供了一组面向性能分析任务的 **REST** 接口，

包括按提交版本查询、跨版本对比以及历史趋势获取等功能。复杂的数据筛选与聚合逻辑均在后端完成，从而降低前端实现复杂度，提高系统整体的模块化程度。后端业务逻辑层的实际实现架构及核心组件的调用关系如图4-6所示。

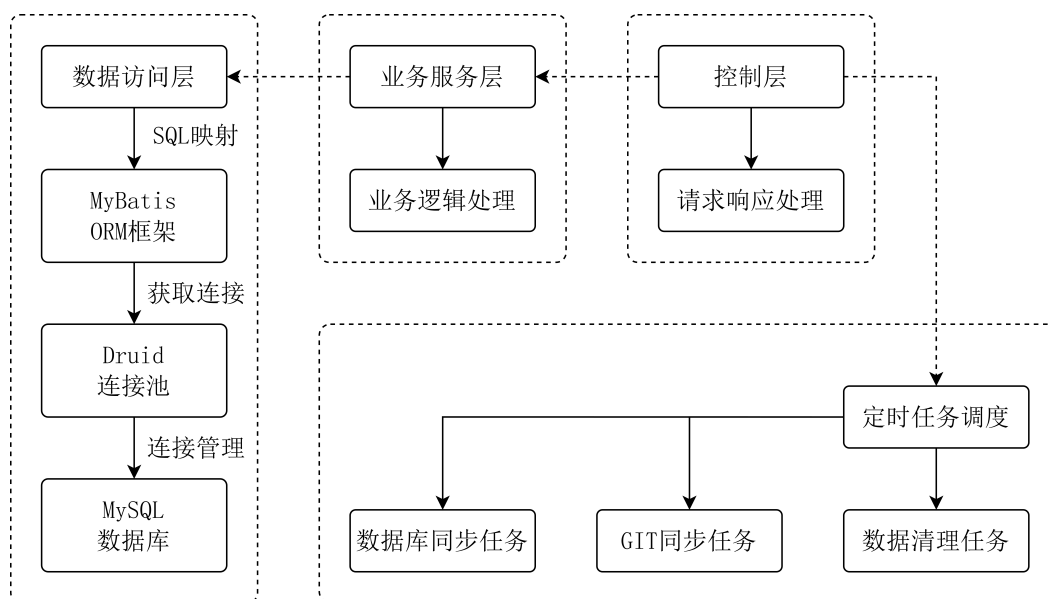


图 4-6 后端业务逻辑层实现架构图

3. 数据存储层的实现

数据存储层的作用是对业务逻辑层处理后的性能报告进行持久化存储，为了实现更高效的查询，本文采用 Mysql 关系型数据库来作为核心存储引擎。同时考虑到性能测试报告会随着工具链的持续迭代不断积累，在设计数据库时，对常用查询维度建立索引，以提升历史数据查询和对比分析的效率。该数据存储机制为评估子系统提供了稳定、可扩展的数据基础，有效支撑了后续章节中针对全局指令选择性能影响的实验分析。以 record 表为例，表4-2对 record 表的核心字段、数据类型、约束规则及索引设计进行了详细说明。

4.4 本章小结

本章分析了 DSP 编译器在引入 GlobalISel 及相关优化后，现有 CI 测试在输入覆盖与结果分析方面存在的不足。针对现存的不足，本章设计并实现了一套面向 DSP 工具链的测试评估平台，通过引入随机测试生成工具、完善性能数据自动采集，并结合前后端与数据库的三层架构，实现了测试结果的统一管理 with 可视化分析。最终，形成了一套覆盖测试生成、流水线执行、性能采集与对比分析的完整流程，为后续 GlobalISel 优化效果的定量评估与回归分析提供了可靠支撑。

表 4-2 record 表结构与索引设计

字段名	数据类型	约束	说明
id	varchar(64)	NOT NULL, PK	测试记录主键
case_name	varchar(128)	NOT NULL	测试样例名称
status	varchar(16)	NOT NULL	测试执行状态
target_cpu	varchar(32)	NOT NULL	目标 CPU 架构
opt_level	int	NOT NULL	编译优化级别
platform	varchar(16)	NOT NULL	测试平台
testsuite	varchar(64)	NOT NULL	测试集名称
hash	varchar(64)	NOT NULL, FK	关联 commit 表的哈希值
cycle_num	int	NOT NULL	测试用例执行周期数
code_size	int	NOT NULL	编译后代码大小
bundle_x	int	NOT NULL	指令打包情况

索引说明：
1. PRIMARY (id): 主键索引，加速单条测试记录的查询与修改；
2. fk_record_commit_hash (hash): 外键索引，加速提交与测试记录的关联查询；
3. idx_record_for_assembly_fk (target_cpu, opt_level, testsuite, case_name): 复合索引，优化汇编关联查询；
4. idx_record_for_stats (target_cpu, platform, opt_level, testsuite, hash): 复合索引，加速性能统计类查询。

第 5 章 实验结果与性能分析

本章通过一系列系统性实验对本文研究内容展开全面验证与分析，依次验证 GlobalISel 在 DSP 后端生成代码的正确性、本文提出的 GlobalISel 分阶段优化策略的有效性，并开展其与传统指令选择方案的综合性能对比，最后展示测试评估平台的功能及应用效果。

5.1 GlobalISel 代码生成正确性验证

5.1.1 基本指令正确性验证

本节旨在验证 DSP 后端基本指令的正确性，重点关注算术运算指令和内存访问指令的匹配和选择结果。为此本节设计了一个示例程序一，这个示例程序包含函数参数传递、内存读写操作和整数加法与乘法运算等操作，能够覆盖了 DSP 后端常见的指令选择路径。由于 LLVM IR 具有良好的目标无关性，且从 C 语言到 LLVM IR 的转换过程并非本文关注重点，因此本节及后续章节在分析过程中直接展示从 LLVM IR 到目标机器码的转换流程。

1.LLVM IR 输入

示例程序一的 LLVM IR 如下所示，该示例程序包含了基本的算术与访存操作，后文将以此为基础分析指令选择过程中中间表示的演化。

```
1  define dso_local i32 @testArith(i32 noundef %a, i32 noundef %b) {  
2      entry:  
3      %a.addr = alloca i32  
4      %b.addr = alloca i32  
5      store i32 %a, ptr %a.addr  
6      store i32 %b, ptr %b.addr  
7      %0 = load i32, ptr %a.addr  
8      %1 = load i32, ptr %b.addr  
9      %add = add i32 %0, %1  
10     %2 = load i32, ptr %a.addr  
11     %mul = mul i32 %add, %2  
12     ret i32 %mul  
13 }
```

2.GMIR 生成阶段

如图5-1所示，在 GMIR 生成阶段，LLVM IR 被转换为与目标架构无关的 GMIR。从中间结果可以观察到：

- alloca 被映射为 COPY 指令，函数参数通过 COPY 指令从物理寄存器拷贝到虚拟寄存器。
- load 与 store 分别被映射为 G_LOAD 与 G_STORE。
- add 与 mul 分别被映射为 G_ADD 与 G_MUL。
- ret 被映射为 RET。

```

*** IR Dump After IRTranslator (irtranslator) ***:
# Machine code for function testArith: IsSSA, TracksLiveness
Frame Objects:
  fi#0: size=4, align=4, at location [SP]
  fi#1: size=4, align=4, at location [SP]
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  %0:_(s32) = COPY $a0
  %1:_(s32) = COPY $a1
  %2:_(p0) = G_FRAME_INDEX %stack.0.a.addr
  %3:_(p0) = G_FRAME_INDEX %stack.1.b.addr
  G_STORE %0:_(s32), %2:_(p0) :: (store (s32) into %ir.a.addr)
  G_STORE %1:_(s32), %3:_(p0) :: (store (s32) into %ir.b.addr)
  %4:_(s32) = G_LOAD %2:_(p0), debug-location !17 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:13
  %5:_(s32) = G_LOAD %3:_(p0), debug-location !18 :: (dereferenceable load (s32) from %ir.b.addr); ./arith.c:2:17
  %6:_(s32) = nsw G_ADD %4:_, %5:_, debug-location !19; ./arith.c:2:15
  %7:_(s32) = G_LOAD %2:_(p0), debug-location !20 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:22
  %8:_(s32) = nsw G_MUL %6:_, %7:_, debug-location !21; ./arith.c:2:20
  $v0 = COPY %8:_(s32), debug-location !22; ./arith.c:2:5
  RET implicit $lr, implicit $v0, debug-location !22; ./arith.c:2:5

# End machine code for function testArith.

```

图 5-1 示例程序一的 GMIR 生成阶段 Dump 图

该结果表明，LLVM IR 到 GMIR 的转换过程保持了原有运算语义与数据依赖关系，为后续后端处理提供了正确的输入。

3. 合法化阶段

指令合法化阶段的核心任务，在于将 GMIR 中目标架构不直接支持的操作，转化为目标架构可以直接处理的合法形式。在本示例程序中，32 位的 G_ADD、G_MUL、G_LOAD 以及 G_STORE 等指令均为 DSP 架构原生支持的操作，因此并未触发类型扩展及指令拆解等复杂的合法化过程。如图5-2所示，合法化前后的指令序列在结构和语义上保持一致。这一结果表明，当前 DSP 合法化规则能够正确处理基本的算术运算与内存访问指令。

```
# *** IR Dump After Legalizer (legalizer) ***:
# Machine code for function testArith: IsSSA, TracksLiveness, Legalized
Frame Objects:
  fi#0: size=4, align=4, at location [SP]
  fi#1: size=4, align=4, at location [SP]
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  %0:_(s32) = COPY $a0
  %1:_(s32) = COPY $a1
  %2:_(p0) = G_FRAME_INDEX %stack.0.a.addr
  G_STORE %0:_(s32), %2:_(p0) :: (store (s32) into %ir.a.addr)
  %3:_(p0) = G_FRAME_INDEX %stack.1.b.addr
  G_STORE %1:_(s32), %3:_(p0) :: (store (s32) into %ir.b.addr)
  %4:_(s32) = G_LOAD %2:_(p0), debug-location !17 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:13
  %5:_(s32) = G_LOAD %3:_(p0), debug-location !18 :: (dereferenceable load (s32) from %ir.b.addr); ./arith.c:2:17
  %6:_(s32) = nsw G_ADD %4:_, %5:_, debug-location !19; ./arith.c:2:15
  %7:_(s32) = G_LOAD %2:_(p0), debug-location !20 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:22
  %8:_(s32) = nsw G_MUL %6:_, %7:_, debug-location !21; ./arith.c:2:20
  $v0 = COPY %8:_(s32), debug-location !22; ./arith.c:2:5
  RET implicit $lr, implicit $v0, debug-location !22; ./arith.c:2:5

# End machine code for function testArith.
```

图 5-2 示例程序一的合法化阶段 Dump 图

4. 寄存器组选择阶段

在寄存器组选择阶段，编译器需要为每个虚拟寄存器分配合适的寄存器组。在当前的 DSP 后端实现中，所有参与整数算术运算的虚拟寄存器都被统一映射至 GPRB 中。如图5-3所示，从寄存器组选择后的中间结果可以观察到，所有算术指令的操作数及结果均被分配到 GPRB 寄存器组，并且寄存器类型与对应操作数的位宽保持一致，符合 DSP 架构对整数运算的寄存器使用约束。

```
# *** IR Dump After RegBankSelect (regbankselect) ***:
# Machine code for function testArith: IsSSA, TracksLiveness, Legalized, RegBankSelected
Frame Objects:
  fi#0: size=4, align=4, at location [SP]
  fi#1: size=4, align=4, at location [SP]
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  %0:gprb(s32) = COPY $a0
  %1:gprb(s32) = COPY $a1
  %2:gprb(p0) = G_FRAME_INDEX %stack.0.a.addr
  G_STORE %0:gprb(s32), %2:gprb(p0) :: (store (s32) into %ir.a.addr)
  %3:gprb(p0) = G_FRAME_INDEX %stack.1.b.addr
  G_STORE %1:gprb(s32), %3:gprb(p0) :: (store (s32) into %ir.b.addr)
  %4:gprb(s32) = G_LOAD %2:gprb(p0), debug-location !17 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:13
  %5:gprb(s32) = G_LOAD %3:gprb(p0), debug-location !18 :: (dereferenceable load (s32) from %ir.b.addr); ./arith.c:2:17
  %6:gprb(s32) = nsw G_ADD %4:gprb, %5:gprb, debug-location !19; ./arith.c:2:15
  %7:gprb(s32) = G_LOAD %2:gprb(p0), debug-location !20 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:22
  %8:gprb(s32) = nsw G_MUL %6:gprb, %7:gprb, debug-location !21; ./arith.c:2:20
  $v0 = COPY %8:gprb(s32), debug-location !22; ./arith.c:2:5
  RET implicit $lr, implicit $v0, debug-location !22; ./arith.c:2:5

# End machine code for function testArith.
```

图 5-3 示例程序一的寄存器组选择阶段 Dump 图

该结果表明所有参与算术运算的虚拟寄存器均被正确分配至通用寄存器组，同时未引入额外的跨寄存器组数据复制或冗余中间指令。

5. 机器指令选择阶段

在机器指令选择阶段，GMIR 被转换为 DSP 架构的目标机器指令。如 5-4 所示，从中间结果可以观察到，除了 COPY 指令外的所有指令都被正确的映射到 DSP 后端机器指令，COPY 指令需要在后续阶段中单独处理。

```
# *** IR Dump After InstructionSelect (instruction-select) ***:
# Machine code for function testArith: IsSSA, TracksLiveness, Legalized, RegBankSelected, Selected
Frame Objects:
  fi#0: size=4, align=4, at location [SP]
  fi#1: size=4, align=4, at location [SP]
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  %0:gpr32 = COPY $a0
  %1:gpr32 = COPY $a1
  STORE32 %0:gpr32, %stack.0.a.addr, 0 :: (store (s32) into %ir.a.addr)
  STORE32 %1:gpr32, %stack.1.b.addr, 0 :: (store (s32) into %ir.b.addr)
  %4:gpr32 = LOAD32 %stack.0.a.addr, 0, debug-location !17 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2
:13
  %5:gpr32 = LOAD32 %stack.1.b.addr, 0, debug-location !18 :: (dereferenceable load (s32) from %ir.b.addr); ./arith.c:2
:17
  %6:gpr32 = nsw ADD %4:gpr32, %5:gpr32, implicit-def dead $carry, debug-location !19; ./arith.c:2:15
  %7:gpr32 = LOAD32 %stack.0.a.addr, 0, debug-location !20 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2
:22
  %8:gpr32 = nsw MUL32 %6:gpr32, %7:gpr32, debug-location !21; ./arith.c:2:20
  $v0 = COPY %8:gpr32, debug-location !22; ./arith.c:2:5
  RET implicit $lr, implicit $v0, debug-location !22; ./arith.c:2:5

# End machine code for function testArith.
```

图 5-4 示例程序一的机器指令选择阶段 Dump 图

6. 最终代码生成阶段

在完成寄存器分配以及指令调度等后续后端处理流程后，编译器最终生成了符合 DSP 指令集规范的目标代码。如图 5-5 所示，生成的目标代码在指令顺序、操作数选择以及控制流结构等方面均与原始程序语义保持一致。这表明在该示例程序中，基于 GlobalISel 框架的代码生成流程能够在 DSP 后端上正确运行，实现了从高级 LLVM IR 到目标平台可执行机器指令的精准映射。

5.1.2 控制流指令正确性验证

除基本算术与内存访问指令这些基本指令外，控制流指令的正确生成和映射同样是验证编译器后端正确性的重要体现。本节针对对控制流相关指令的处理过程进行验证，重点覆盖比较与条件分支与函数调用两类典型控制流场景。

1. 比较 + 分支 + 跳转指令

比较与条件分支是程序控制流构建的基础。在 LLVM IR 中，比较与条件分支通过 icmp 与 br 组合表达。为验证 DSP 后端对该类控制流指令的处理正确性，

```

# *** IR Dump After DSP NOP Rewriter (enableNOPRewriter) ***:
# Machine code for function testArith: NoPHIs, TracksLiveness, NoVRegs, Legalized, RegBankSelected, Selected, TiedOpsRe
written
Frame Objects:
  fi#0: size=4, align=4, at location [SP-4]
  fi#1: size=4, align=4, at location [SP-8]
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  $sp = ADDI $sp, -8, implicit-def $carry
  CFI_INSTRUCTION def_cfa_offset 8
  STORE32 killed $a0, $sp, 4 :: (store (s32) into %ir.a.addr)
  STORE32 killed $a1, $sp, 0 :: (store (s32) into %ir.b.addr)
  $v0 = LOAD32 $sp, 4, debug-location !17 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:13
  NOP debug-location !18; ./arith.c:2:17
  $v1 = LOAD32 $sp, 0, debug-location !18 :: (dereferenceable load (s32) from %ir.b.addr); ./arith.c:2:17
  NOP debug-location !19; ./arith.c:2:15
  $v0 = nsw ADD killed $v0, killed $v1, implicit-def dead $carry, debug-location !19; ./arith.c:2:15
  $v1 = LOAD32 $sp, 4, debug-location !20 :: (dereferenceable load (s32) from %ir.a.addr); ./arith.c:2:22
  $v0 = nsw MUL32 killed $v0, killed $v1, debug-location !21; ./arith.c:2:20
  $sp = ADDI $sp, 8, implicit-def $carry
  RET implicit $lr, implicit killed $v0, debug-location !22; ./arith.c:2:5
  JNOP debug-location !22; ./arith.c:2:5
  JNOP debug-location !22; ./arith.c:2:5

# End machine code for function testArith.

```

图 5-5 示例程序一的最终阶段 Dump 图

本节构建了一个同时包含比较、条件分支及无条件跳转的示例程序二，其 LLVM IR 如下所示。

```

1 define dso_local i32 @testBranch() {
2   entry:
3     %retval = alloca i32
4     %a = alloca i32
5     %b = alloca i32
6     store i32 10, ptr %a
7     store i32 20, ptr %b
8     %0 = load i32, ptr %a
9     %1 = load i32, ptr %b
10    %cmp = icmp slt i32 %0, %1
11    br i1 %cmp, label %if.then, label %if.else
12
13    if.then:
14      store i32 0, ptr %retval
15      br label %return
16
17    if.else:
18      store i32 1, ptr %retval
19      br label %return
20
21    return: %2 = load i32, ptr %retval
22    ret i32 %2
23 }

```

针对该示例程序，在 GlobalISel 各关键阶段中的处理过程如下：

- 在 GMIR 生成阶段，LLVM IR 中的比较指令被正确转换为 G_ICMP，其比较谓词与操作数保持一致；条件跳转指令被转换为 G_BRCOND，并以比较结果作为分支条件。
- 在合法化阶段，由于 DSP 架构支持整数比较操作，无需进一步拆分，相关指令保持原有结构。
- 在寄存器组选择阶段，编译器为比较操作及其结果所涉及的虚拟寄存器分配了合适的寄存器组，确保比较结果能够以符合 DSP 架构约定的方式参与后续控制流指令。
- 在机器指令选择阶段，G_ICMP 与后续的 G_BRCOND 会被联合处理，映射为 DSP 架构下的比较指令与条件跳转指令，比较结果通过条件码寄存器传递，从而完成分支控制。

如图5-6所示，在函数入口基本块中，比较指令首先生成条件码寄存器状态，随后通过条件跳转指令根据比较结果在两个后继基本块之间进行分支选择，验证了比较与条件分支指令的正确映射。

```
# *** IR Dump After DSP NOP Rewriter (enableNOPRewriter) ***:
# Machine code for function testBranch: NoPHIs, TracksLiveness, NoVRegs, Legalized, RegBankSelected, Selected, TiedOpsR
ewritten
Frame Objects:
  fi#0: size=4, align=4, at location [SP-4]
  fi#1: size=4, align=4, at location [SP-8]
  fi#2: size=4, align=4, at location [SP-12]

bb.1.entry:
  successors: %bb.2, %bb.3

  $sp = ADDI $sp, -16, implicit-def $carry
  CFI_INSTRUCTION def_cfa_offset 16
  $v0 = MOVLZ 10, debug-location !14; ./branch.c:2:9
  STORE32 killed $v0, $sp, 8, debug-location !14 :: (store (s32) into %ir.a); ./branch.c:2:9
  $v0 = MOVLZ 20, debug-location !16; ./branch.c:2:17
  STORE32 killed $v0, $sp, 4, debug-location !16 :: (store (s32) into %ir.b); ./branch.c:2:17
  NOP debug-location !17; ./branch.c:3:9
  $v0 = LOAD32 $sp, 8, debug-location !17 :: (dereferenceable load (s32) from %ir.a); ./branch.c:3:9
  NOP debug-location !19; ./branch.c:3:13
  $v1 = LOAD32 $sp, 4, debug-location !19 :: (dereferenceable load (s32) from %ir.b); ./branch.c:3:13
  GE killed $v0, killed $v1, implicit-def $con, implicit-def $con, debug-location !20; ./branch.c:3:11
  JC %bb.3, implicit $con, implicit $con, debug-location !20; ./branch.c:3:11
  JNOP debug-location !20; ./branch.c:3:11
  JNOP debug-location !20; ./branch.c:3:11
  JMP %bb.2, debug-location !20; ./branch.c:3:11
  JNOP debug-location !20; ./branch.c:3:11
  JNOP debug-location !20; ./branch.c:3:11
```

图 5-6 示例程序二入口基本块的最终阶段 Dump 图

如图5-7所示，条件判断成立时进入 if.then 基本块，不成立则进入 if.else 基本块，两个基本块各自执行相应的逻辑操作，且在块末尾均通过无条件跳转指令，汇合到同一个返回基本块中。从最终生成的机器码结果来看，各分支基本块间的跳转关系，以及这些分支块与返回块的连接方式，均与 LLVM IR 所定义的控制流结构一致，未产生任何额外的控制流变动。

```

bb.2.if.then:
; predecessors: %bb.1
successors: %bb.4(0x80000000); %bb.4(100.00%)

$v0 = MOVLZ 0, debug-location !21; ./branch.c:4:9
STORE32 killed $v0, $sp, 12, debug-location !21 :: (store (s32) into %ir.retval); ./branch.c:4:9
JMP %bb.4, debug-location !21; ./branch.c:4:9
JNOP debug-location !21; ./branch.c:4:9
JNOP debug-location !21; ./branch.c:4:9

bb.3.if.else:
; predecessors: %bb.1
successors: %bb.4(0x80000000); %bb.4(100.00%)

$v0 = MOVLZ 1, debug-location !23; ./branch.c:6:9
STORE32 killed $v0, $sp, 12, debug-location !23 :: (store (s32) into %ir.retval); ./branch.c:6:9
JMP %bb.4, debug-location !23; ./branch.c:6:9
JNOP debug-location !23; ./branch.c:6:9
JNOP debug-location !23; ./branch.c:6:9

bb.4.return:
; predecessors: %bb.3, %bb.2

$v0 = LOAD32 $sp, 12, debug-location !25 :: (dereferenceable load (s32) from %ir.retval); ./branch.c:8:1
$sp = ADDI $sp, 16, implicit-def $carry
RET implicit $lr, implicit killed $v0, debug-location !25; ./branch.c:8:1
JNOP debug-location !25; ./branch.c:8:1
JNOP debug-location !25; ./branch.c:8:1

```

图 5-7 示例程序二条件分支基本块的最终阶段 Dump 图

从实验结果可以观察到，在 LLVM IR 向目标代码的转换过程中，比较条件及其对应的分支目标均保持一致，程序控制流未发生偏移或错误跳转，实验结果验证了比较、分支与跳转指令选择的正确性。

2. 函数调用指令

函数调用是一类重要的控制流操作，其正确性依赖于调用约定、参数传递机制以及返回值处理等多个环节。为验证 DSP 后端在 GlobalISel 框架下对函数调用指令的处理正确性，构造了一个简单的示例程序四，其中 main 函数调用前文定义的 testBranch 函数，其 LLVM IR 如下所示。

```

1 define dso_local i32 @main() {
2   entry:
3   %retval = alloca i32
4   store i32 0, ptr %retval
5   %call = call i32 @testBranch()
6   ret i32 %call
7 }

```

针对该示例程序，函数调用在各阶段的处理过程如下：

- 在 GMIR 生成阶段，call 指令被转换为与调用约定相关的一系列包括参数准备、调用指令以及返回值接收等操作的 GMIR 指令。

- 在合法化阶段，合法化前后指令结构保持一致，表明 DSP 后端的合法化规则能够覆盖该类函数调用场景的基本操作需求。
- 在寄存器组选择阶段，函数参数与返回值在符合 DSP ABI 的规则下被分配至 DSP 架构规定的寄存器或栈位置。
- 在机器指令选择阶段，与调用相关的 GMIR 指令会被映射为 DSP 架构支持的跳转与返回指令。

如图5-8所示，实验结果表明 DSP 后端在 GlobalISel 框架下能够正确完成函数调用相关控制流的指令选择，从而保证了跨函数控制流的正确传递。

```
# *** IR Dump After DSP NOP Rewriter (enableNOPRewriter) ***:
# Machine code for function main: NoPHIs, TracksLiveness, NoRegs, Legalized, RegBankSelected, Selected, TiedOpsRewritten
Frame Objects:
  fi#0: size=4, align=4, at location [SP-8]
  fi#1: size=4, align=4, at location [SP-4]

bb.1.entry:
  $sp = ADDI $sp, -16, implicit-def $carry
  CFI_INSTRUCTION def_cfa_offset 16
  STORE32 killed $lr, $sp, 12 :: (store (s32) into %stack.1)
  CFI_INSTRUCTION offset $lr, -4
  $v0 = MOVLZ 0
  STORE32 killed $v0, $sp, 8 :: (store (s32) into %ir.retval)
  CALL @testBranch, <regmask $fp $lr $d8 $d9 $d10 $d11 $s0 $s1 $s2 $s3 $s4 $s5 $s6 $s7>, implicit-def $lr, implicit $lr,
  implicit-def $sp, implicit-def $v0, debug-location !27; ./branch.c:12:12
  NOP debug-location !27; ./branch.c:12:12
  NOP debug-location !27; ./branch.c:12:12
  $lr = LOAD32 $sp, 12, debug-location !28 :: (load (s32) from %stack.1); ./branch.c:12:5
  $sp = ADDI $sp, 16, implicit-def $carry
  RET implicit $lr, implicit killed $v0, debug-location !28; ./branch.c:12:5
  JNOP debug-location !28; ./branch.c:12:5
  JNOP debug-location !28; ./branch.c:12:5
```

图 5-8 示例程序四的最终阶段 Dump 图

5.2 GlobalISel 优化策略有效性验证

为验证本文所提出优化策略的有效性，本节通过一系列实验分别对两个阶段的优化效果进行验证。首先进行基于典型样例的针对性验证，构造一个符合优化触发条件的典型测试程序，通过观察优化前后的中间表示变化，判断优化是否正确触发并产生预期结果。然后在 Embench 基准测试集进行系统性量化评估。实验通过对比优化开启与优化关闭两种配置下生成的代码，从代码体积与执行周期两个维度进行量化分析，从而系统评估优化对性能产生的实际影响。

5.2.1 合法化前优化策略验证

1. 典型样例验证

为验证合法化前优化是否有效，本节以内存操作优化为例设计了示例程序五，程序包含一个内存拷贝函数，用于从源地址拷贝 16 字节的 32 位整型数据到

目标地址，其 LLVM IR 如下所示。

```

1 define void @main(ptr %dst, ptr %src) {
2   call void @llvm.memcpy.p0.p0.i32(ptr %dst, ptr %src, i32 16, i1 0)
3   ret void
4 }
5
6 declare void @llvm.memcpy.p0.p0.i32(ptr, ptr, i32, i1)

```

在未启用优化的编译流程中，内存拷贝以函数调用形式实现。在启用了本文实现的合法化前优化后，编译器在 GMIR 生成阶段将此内存拷贝调用识别并转换为等价的基础内存操作指令序列。图5-9与图5-10分别展示了优化前后的中间表示。

```

# *** IR Dump After IRTranslator (irtranslator) ***:
# Machine code for function main: IsSSA, TracksLiveness
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  %0:_(p0) = COPY $a0
  %1:_(p0) = COPY $a1
  %2:_(s32) = G_CONSTANT i32 16
  G_MEMCPY %0:_(p0), %1:_(p0), %2:_(s32), 0 :: (store (s8) into %ir.dst, align 8), (load (s8) from %ir.src, align 8)
  RET implicit $lr

# End machine code for function main.

```

图 5-9 示例程序五优化前 Dump 图

```

# *** IR Dump After DSP00PreLegalizerCombiner (dsp-00-prelegalizer-combiner) ***:
# Machine code for function main: IsSSA, TracksLiveness
Function Live Ins: $a0, $a1

bb.1.entry:
  liveins: $a0, $a1
  %0:_(p0) = COPY $a0
  %1:_(p0) = COPY $a1
  %3:_(s64) = G_LOAD %1:_(p0) :: (load (s64) from %ir.src)
  G_STORE %3:_(s64), %0:_(p0) :: (store (s64) into %ir.dst)
  %4:_(s32) = G_CONSTANT i32 8
  %5:_(p0) = G_PTR_ADD %1:_, %4:_(s32)
  %6:_(s64) = G_LOAD %5:_(p0) :: (load (s64) from %ir.src + 8)
  %7:_(p0) = G_PTR_ADD %0:_, %4:_(s32)
  G_STORE %6:_(s64), %7:_(p0) :: (store (s64) into %ir.dst + 8)
  RET implicit $lr

# End machine code for function main.

```

图 5-10 示例程序五优化后 Dump 图

对比两图可以看出，memcpy 指令在优化后被成功替换为一组 G_LOAD 与 G_STORE 指令序列，完成了从内置函数到基础内存操作的转换，证明了该优化策略的有效性。

2. 系统性量化评估

以优化等级 O2 为例，选取 Embench 测试集中的 10 个样例进行测试。在开优化和不开优化两种条件下，对这些样例的代码体积和执行周期进行统计并通过可视化方式呈现优化前后的量化对比结果。

代码体积是衡量编译器后端代码生成质量的重要指标之一，尤其在 DSP 这类嵌入式场景中，受限的程序存储空间使得指令密度成为影响系统规模与功耗水平的关键因素。因此，分析不同指令选择方案对最终生成代码体积的影响，对于评估 GlobalISel 框架下的优化效果具有重要意义。

本节及后续章节都基于 `llvm-size` 工具来完成代码体积的量化，并以目标文件中 `text` 段大小作为代码体积的度量指标。`text` 段大小能够直接反映指令选择与代码生成阶段所产生的指令数量与编码密度，同时有效规避调试信息、符号表等非执行内容对统计结果的干扰。

从图5-11可以观察到，除了 `crc32` 样例的缩减率是负值之外，其余样例的缩减率均为正值，这表明大多样例的代码体积都有减小，其中 `edn` 样例的效果最明显。

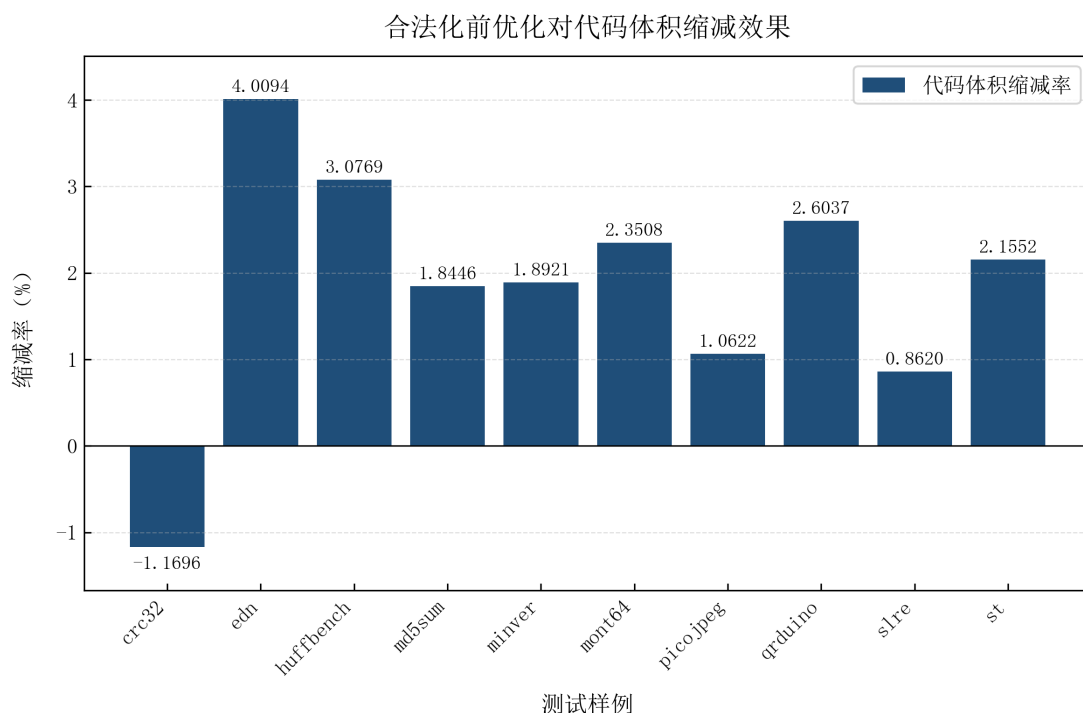


图 5-11 合法化前优化对代码体积提升效果图

执行周期是衡量编译器后端优化效果最直接、也是最具代表性的性能指标之一。对于 DSP 这种以吞吐率和实时性为核心设计目标的处理器架构而言，程序执行周期的变化能够直观反映编译器后端各阶段的决策对实际运行性能的影响。

响。因此，本节围绕执行周期这一维度，对不同指令选择方案在 DSP 平台上的运行表现进行对比分析。

本节及后续章节的执行周期测量基于 DSP 工具链配套的仿真器环境完成。在仿真环境中，仿真器能够在程序执行过程中精确统计指令发射与执行情况，并直接给出程序完成所消耗的总周期数。

为保证不同指令选择方案在执行周期统计上的可比性，本节在测量时统一采用相同的测试程序、输入数据以及运行环境配置。除此之外，本节关注程序主体计算阶段的执行周期，避免初始化、调试输出等非核心逻辑对统计结果产生干扰。实验最终得到的执行周期能够真实地反映编译器后端生成的代码在 DSP 架构上的运行效率。

从图5-12可以观察到，所有样例的执行周期缩减率都为正值，这表明所有的样例的执行效率均提高了。

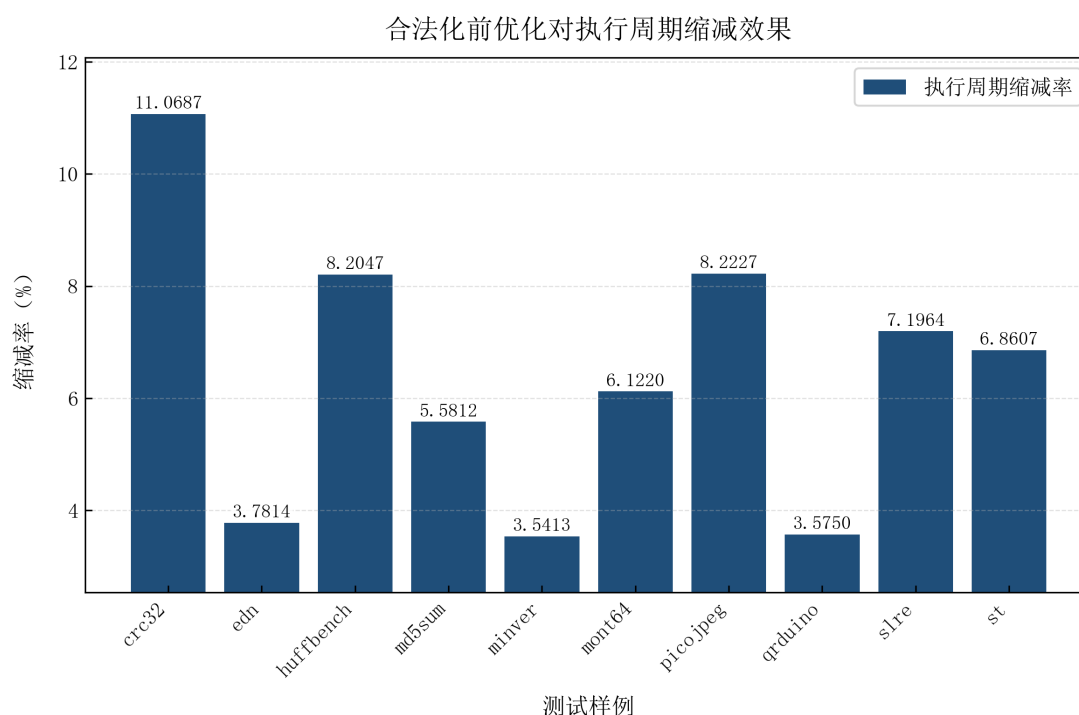


图 5-12 合法化前优化对执行周期提升效果图

以上实验结果表明：本文实现的面向 DSP 后端的合法化前优化在大多数场景下都能有效减少代码体积并提高执行效率。

5.2.2 合法化后优化策略验证

1. 典型样例验证

为验证合法化后优化是否有效，本节以常量乘法强度削弱为例设计了示例程序六，程序包含一个 32 位整型乘法指令，其 LLVM IR 如下所示。

```

1 define dso_local i32 @main() #0 align 16 {
2   entry:
3     %retval = alloca i32, align 4
4     %a = alloca i32, align 4
5     store i32 0, ptr %retval, align 4
6     store i32 17, ptr %a, align 4
7     %0 = load i32, ptr %a, align 4
8     %mul = mul nsw i32 %0, 33
9     ret i32 %mul
10 }

```

在未启用优化的编译流程中，乘法操作以一条硬件乘法指令的形式实现。在启用了本文实现的合法后优化后，编译器在机器指令选择阶段前将乘法指令替换为更优的指令组合。图5-9与图5-10分别展示了优化前后的中间表示。

```

# *** IR Dump After Legalizer (legalizer) ***:
# Machine code for function main: IsSSA, TracksLiveness, Legalized
Frame Objects:
  fi#0: size=4, align=4, at location [SP]
  fi#1: size=4, align=4, at location [SP]

bb.1.entry:
  %0:_(p0) = G_FRAME_INDEX %stack.0.retval
  %2:_(s32) = G_CONSTANT i32 0
  G_STORE %2:_(s32), %0:_(p0) :: (store (s32) into %ir.retval)
  %1:_(p0) = G_FRAME_INDEX %stack.1.a
  %3:_(s32) = G_CONSTANT i32 17
  G_STORE %3:_(s32), %1:_(p0) :: (store (s32) into %ir.a)
  %4:_(s32) = G_LOAD %1:_(p0) :: (dereferenceable load (s32) from %ir.a)
  %5:_(s32) = G_CONSTANT i32 33
  %6:_(s32) = nsw G_MUL %4:_, %5:_
  $v0 = COPY %6:_(s32)
  RET implicit $lr, implicit $v0

# End machine code for function main.

```

图 5-13 示例程序六优化前 Dump 图

```

# *** IR Dump After DSPPostLegalizerCombiner (dsp-postlegalizer-combiner) ***:
# Machine code for function main: IsSSA, TracksLiveness, Legalized
Frame Objects:
  fi#0: size=4, align=4, at location [SP]
  fi#1: size=4, align=4, at location [SP]

bb.1.entry:
  %0:_(p0) = G_FRAME_INDEX %stack.0.retval
  %2:_(s32) = G_CONSTANT i32 0
  G_STORE %2:_(s32), %0:_(p0) :: (store (s32) into %ir.retval)
  %1:_(p0) = G_FRAME_INDEX %stack.1.a
  %3:_(s32) = G_CONSTANT i32 17
  G_STORE %3:_(s32), %1:_(p0) :: (store (s32) into %ir.a)
  %4:_(s32) = G_LOAD %1:_(p0) :: (dereferenceable load (s32) from %ir.a)
  %7:_(s32) = G_CONSTANT i32 5
  %8:_(s32) = G_SHL %4:_, %7:_(s32)
  %9:_(s32) = G_ADD %8:_, %4:_
  $v0 = COPY %9:_(s32)
  RET implicit $lr, implicit $v0

# End machine code for function main.

```

图 5-14 示例程序六优化后 Dump 图

图5-13与图5-14展示了优化前后的 GMIR 对比，从图中可以看出优化前的 G_MUL 指令在优化后被成功替换为 G_SHL 与 G_ADD 指令的组合。此转换证明了常量乘法强度削弱优化在合法化后阶段被正确触发并应用，达到了以低成本操作替代高成本运算的优化目标。

2. 系统性量化评估

以优化等级 O2 为例并选取与 5.2.1 节 A 相同的 10 个样例，在开优化和不开优化两种条件下，对这些样例的代码体积和执行周期进行统计并通过可视化方式呈现优化前后的量化对比结果。

从图5-15可以观察到，mont64 样例的代码体积缩减效果最为显著，缩减率高达 16.2749%；qrduino、slre、picojpeg 等样例也实现了 2% 以上的正向缩减；仅 st 样例出现小幅体积增加。

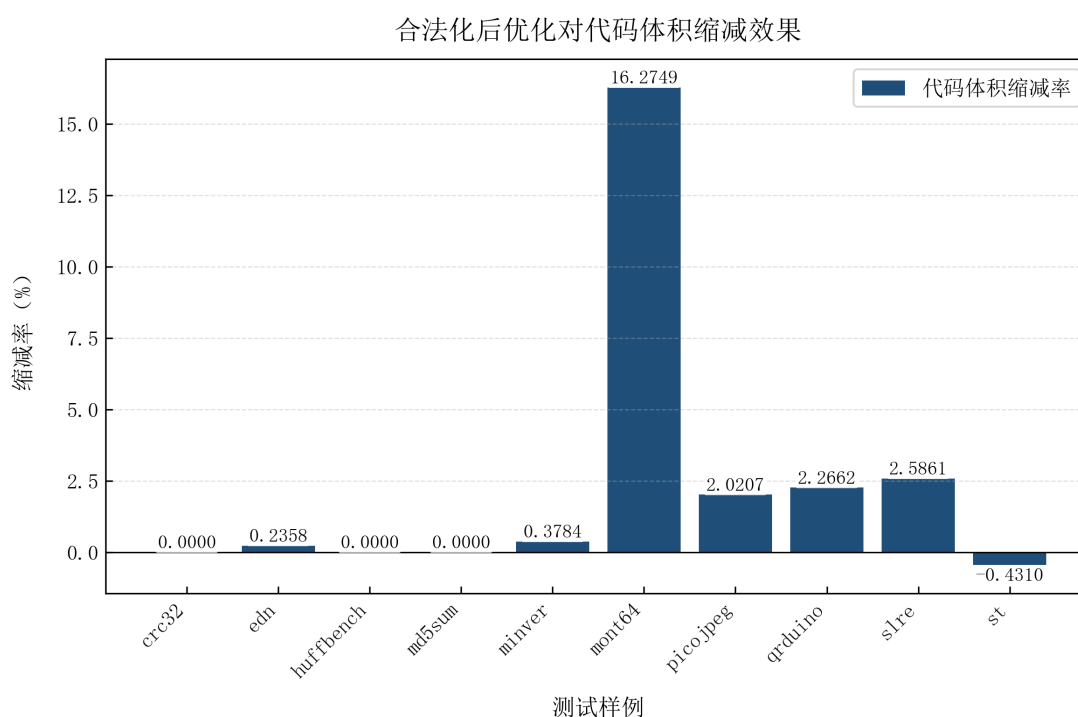


图 5-15 合法化后优化对代码体积提升效果图

从图5-16可以观察到，大部分样例的执行周期均获得正向缩减，其中 mont64 样例的缩减效果最为突出，执行周期缩减率高达 18.3536%；edn、md5sum、picojpeg 等样例也实现了 1% 至 3% 的性能提升；仅 huffbench 与 st 样例的执行周期数出现了小幅增加。

以上实验结果表明：本文实现的面向 DSP 后端的合法化后优化在大多数场景下都能有效减少执行周期并提高执行效率。从优化效果上来看，合法化后优

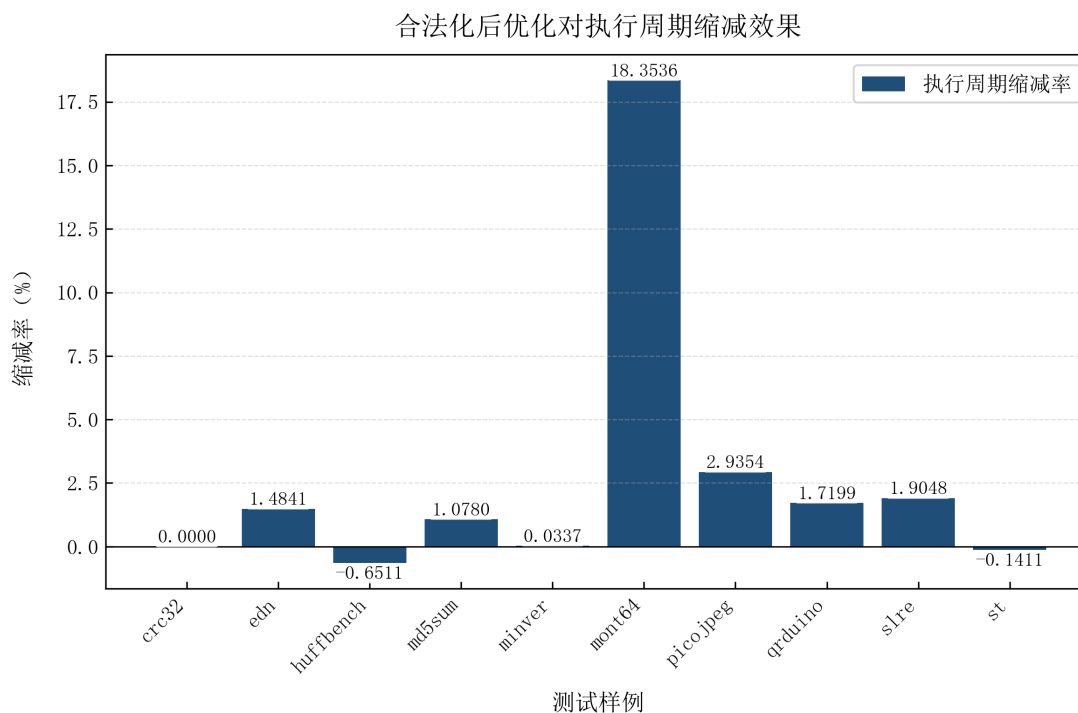


图 5-16 合法化后优化对执行周期提升效果图

化并不像合法化前优化那样普遍提升效率，这是因为合法化后优化针对性更强，仅在特定指令组合模式下才能发挥作用。

5.3 GlobalISel 综合性能对比分析

上一节已通过对比实验分别验证了合法化前、后优化策略的有效性，为全面评估 GlobalISel 框架相较于传统指令选择方案的综合性能优势，本节从代码大小、执行周期以及编译时间三个维度出发，对基于 DAG 的指令选择、未优化的全局指令选择以及优化后的全局指令选择来进行全方面的对比。

5.3.1 代码体积对比

1. 实验设置与统计方法

本节选取 Embench 作为测试集，对每个基准程序分别在四个优化等级下进行编译，并统计生成的目标代码 text 段大小。对于同一优化等级下的同一测试样例，取单次编译结果作为该样例的代码体积。随后，把测试集中所有基准程序的结果进行平均计算，得到各个优化等级下的平均代码段体积，用于不同指令选择方案之间的对比分析。

2. 实验结果分析

图5-17给出了在 Embench 测试集上，未引入优化的 GlobalISel 与引入优化后的 GlobalISel 在不同优化等级下的平均代码段体积对比结果。

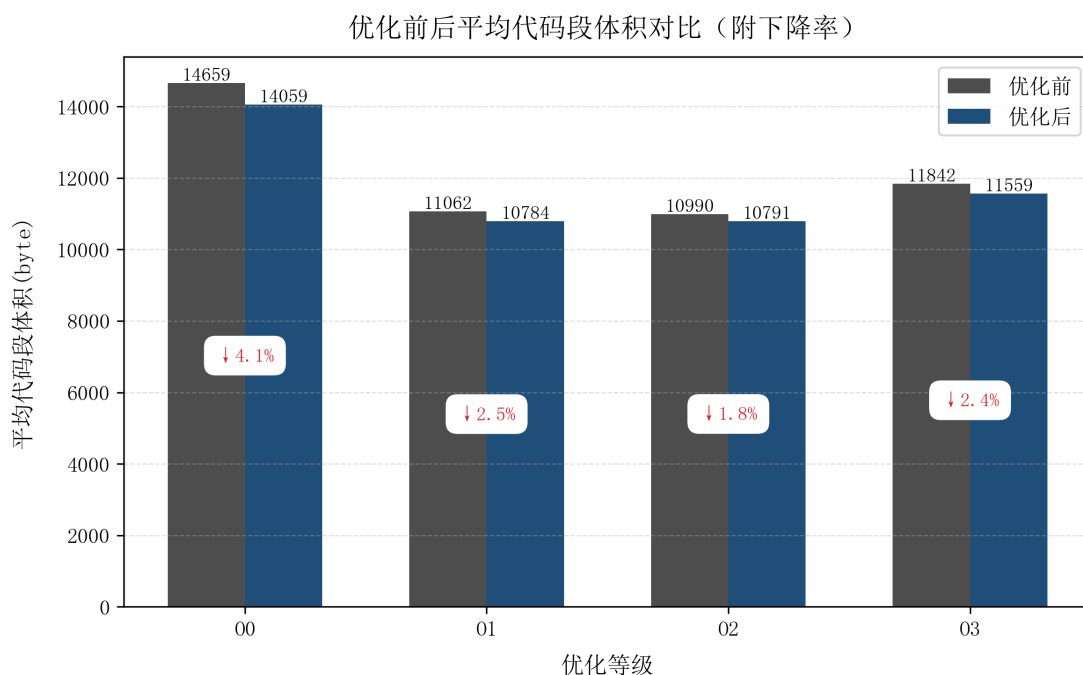


图 5-17 优化前后平均代码段体积对比图

从整体上看，在 4 个优化等级下的优化后 GlobalISel 方案最终生成的平均代码体积更小，而且在 O0 性能等级下体现更加明显，表明即使是不进行高层 LLVM IR 的优化，在后端指令选择和指令组合方面提高的优化手段亦可以减小指令冗余、提高指令密度。高优化等级下，代码体积下降更加收敛，因为高层 LLVM IR 已经展开得很充分，有些冗余已经在前端或中端去掉，后端指令选择优化对代码体积的影响更多是精细化的指令密度改进，而非数量级的减少。

3. 实验结论

实验结果表明，相较于未优化的 GlobalISel 实现，引入针对 DSP 架构特点的指令选择与指令组合优化后，编译器在各优化等级下均生成了更紧凑的目标代码。这表明本文提出的优化策略不仅在性能层面有效，还在代码体积这一关键指标上取得效果，为后续嵌入式场景的实际部署提供了有力支撑。

5.3.2 执行周期对比

1. 实验设置与统计方法

实验同样基于 Embench 测试集展开。为确保数据的稳定性和统计意义，每个基准程序均在相同的硬件平台和编译配置下独立运行多次，以消除单次测量中随机扰动的影响。具体流程为：每个样例执行 10 次，去掉最高和最低各两次结果，取剩余 6 次的平均值作为该样例的执行周期。最后，对所有样例的结果取总平均值，得到该优化等级下的整体执行周期。该统计方式能够有效平衡测量精度与实验成本，在保证数据稳定性的同时，避免个别异常运行结果对整体趋势判断产生干扰。

2. 实验结果与分析

图5-18给出了在 Embench 测试集上，不同指令选择方案在各优化等级下的平均执行周期对比结果。

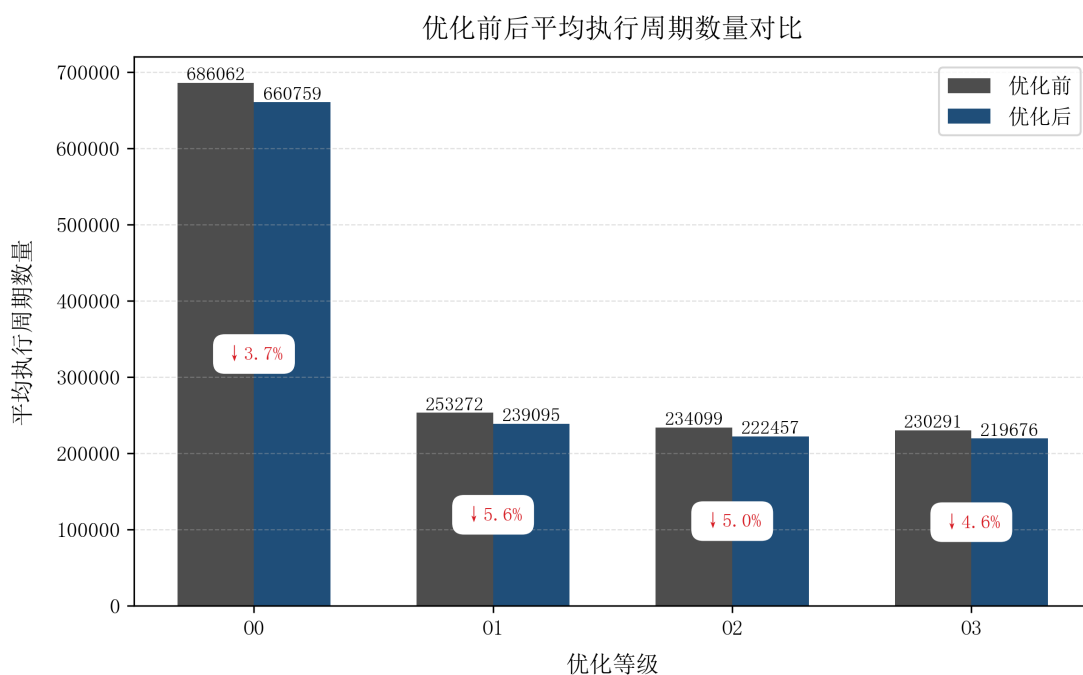


图 5-18 不同指令选择方案平均执行周期对比图

从实验结果可以观察到，相较于未优化的 GlobalISel 实现，引入针对 DSP 架构的指令选择与指令组合优化后，程序在所有优化等级下的平均执行周期均呈现出下降趋势。这表明，在高层 LLVM IR 优化与后端优化协同作用的场景中，GlobalISel 所引入的目标相关指令匹配、冗余指令消除以及寄存器组绑定优化，能够进一步挖掘 DSP 架构的指令级并行潜力，从而在整体执行效率上获得持续收益。

3. 实验结论

从实验结果来看，DSP 后端在引入针对其目标架构特性的 GlobalISel 优化后，编译器生成代码在执行周期维度上取得了稳定且可观的性能提升。并且该提升在不同优化等级下均具有一致性，说明其并非依赖特定编译配置，而是源于指令选择框架与目标相关优化策略在整体设计上的协同改进。这一结果进一步验证了前文提出的优化方法在实际运行性能上的有效性。

5.3.3 编译时间对比

除了代码体积和执行周期这两个性能指标外，编译时间作为一项关键的工程指标，直接反映了编译器的可用性与开发迭代效率，尤其在需要持续进行性能优化与回归测试的编译器开发实践中，后端编译阶段的耗时变化将对整体开发效率产生显著影响。为此，本节将从编译时间维度出发，系统性地对比分析不同指令选择方案在 DSP 后端上的性能表现。

1. 度量方法与相关指标说明

本节基于 Clang 自带的编译时间追踪机制来对编译时间进行量化分析，该机制能够对编译过程中各阶段的执行时间进行精细化统计，从而为分析编译器性能瓶颈与优化效果提供可靠的数据支撑。图5-19中展示了部分关键时间指标及其在不同编译阶段的分布情况。

Clang time report				
Total Execution Time: 0.3594 seconds (0.3535 wall clock)				
---User Time---	--System Time--	--User+System--	---Wall Time---	--- Name ---
0.1562 (50.0%)	0.0312 (66.7%)	0.1875 (52.2%)	0.1848 (52.3%)	Front end
0.0781 (25.0%)	0.0156 (33.3%)	0.0938 (26.1%)	0.0911 (25.8%)	Machine code generation
0.0625 (20.0%)	0.0000 (0.0%)	0.0625 (17.4%)	0.0653 (18.5%)	Optimizer
0.0156 (5.0%)	0.0000 (0.0%)	0.0156 (4.3%)	0.0124 (3.5%)	LLVM IR generation
0.3125 (100.0%)	0.0469 (100.0%)	0.3594 (100.0%)	0.3535 (100.0%)	Total

图 5-19 不同编译阶段时间指标图

其中，各时间指标的含义如下：

- User Time（用户态时间）：表示 CPU 在用户态执行编译器自身代码所消耗的时间，仅反映编译器逻辑计算的开销，不包含操作系统内核相关操作。
- System Time（系统态时间）：表示 CPU 在内核态执行系统调用（如文件读写、内存管理、进程调度等）所消耗的时间，属于编译过程中的系统辅助开销。

- **User+System Time** (用户态加系统态时间): 反映 CPU 在该阶段用于计算的总耗时, 是用户态和系统态时间的累加, 但不包含 I/O 等待和 CPU 空闲时间。
- **Wall Time** (壁钟时间): 指从编译阶段开始到结束所消耗的实际物理时间。在实际场景中, 这一指标最能直观反映用户实际感知到的编译等待时长, 因此也是本文关注的核心量化指标。

LLVM 编译流程可划分为以下几个阶段:

- **Front end** (前端): 负责源代码解析、语法分析、语义分析等核心操作, 核心作用是将高级语言代码转换为中间表示形式。
- **LLVM IR generation** (LLVM IR 生成): 负责将前端分析的结果转换为 LLVM IR, 为后续优化和代码生成奠定基础。
- **Optimizer** (优化器): 负责对 LLVM IR 进行多级优化, 包括控制流优化、数据流优化及目标无关优化等。
- **Machine code generation** (机器码生成): 作为编译器后端的核心环节, 该阶段负责指令选择、寄存器分配、指令调度以及最终目标机器码的生成。

由于指令选择阶段在 LLVM 后端中与其他流程的耦合度较高, 难以单独抽离出来进行精确量化, 因此本节在编译时间的分析中, 将关注点放在后端整体执行时间上。具体而言, 统计 **Machine Code Generation** 阶段的 **Wall Time**, 以此作为后端编译开销的统一度量标准。这种统计方式在保证分析结果可比性和稳定性的同时, 能够有效反映后端相关优化对实际编译时间的影响, 为后续性能评估提供依据。

2. 实验设置与统计方法

为降低单次测量中环境噪声与随机波动对实验结果的影响, 实验采用多次重复实验的统计方法。具体而言, 在保持实验环境与配置完全一致的条件下, 对每个测试样例分别独立执行 10 次完整的编译流程。在结果统计阶段, 剔除两个最大值和两个最小值后再对剩余数据取平均值, 以此作为该样例在对应优化等级下的编译时间。最后对 **Embench** 测试集中 22 个基准程序的结果汇总并进一步取平均值, 得到不同优化等级下的平均编译时间, 以此用于不同指令选择方案之间的对比分析。

3. 实验结果与分析

图5-20给出了在 Embench 测试集上两种指令选择方案在不同优化等级下的平均编译时间对比结果。

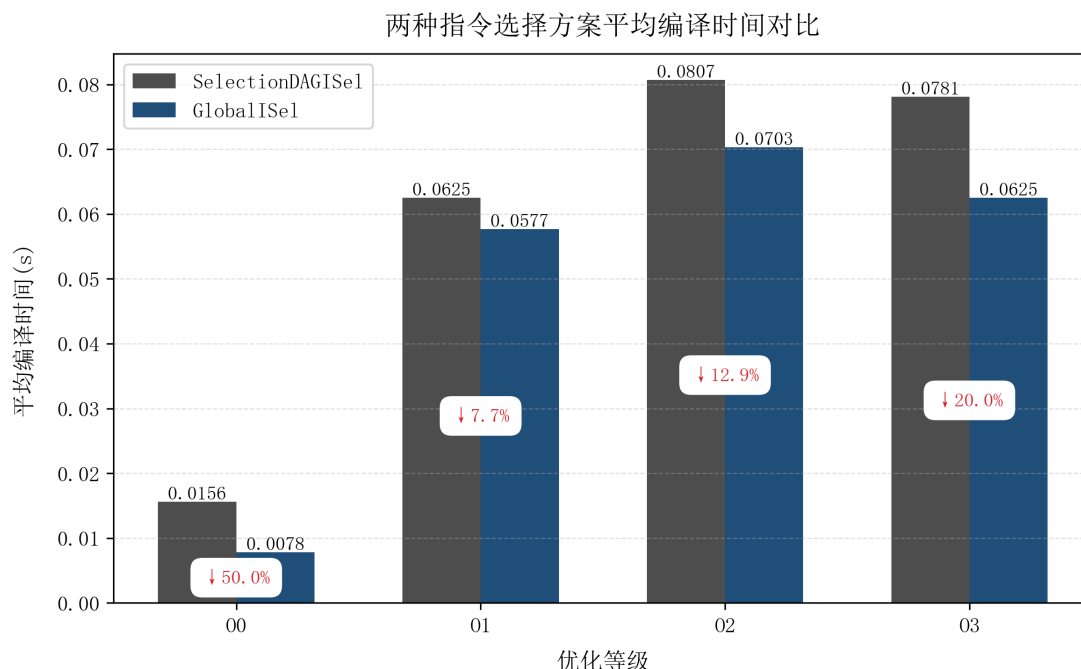


图 5-20 两种指令选择方案平均编译时间对比图

从图5-20中可以观察到，相比于 SelectionDAGISel，GlobalISel 在 4 个优化等级下的平均编译时间均更短，尤其是在 O0 优化等级下，平均编译时间只有 SelectionDAGISel 的一半。这个现象表明在中端优化尚未充分展开的情况下，指令选择框架自身的设计差异就已经对后端编译效率产生了影响。

4. 实验结论

从实验结果可以观察到，在编译时间上 GlobalISel 相比 SelectionDAG 具备明显优势，且该优势在各优化等级下均保持一致。这说明该优势并非源于特定的优化配置，而是来自指令选择框架本身在流程设计和中间表示处理上的效率优化，这一特性使 GlobalISel 更适配频繁编译、快速迭代的开发场景。

5.4 测试评估平台功能展示

测试评估平台围绕编译器工具链的持续迭代需求来进行设计。平台的评估子系统集成了性能数据采集、历史回归分析、版本对比以及汇编级诊断等功能。平台通过系统化地记录与分析不同 Commit 在统一测试集上的性能表现，已在实

际开发过程中协助作者发现并定位多个编译器缺陷，为编译器优化与回归验证提供了重要支撑。下文结合平台主要功能模块，详细展示其核心功能。

1. 全局性能趋势分析模块

如图5-21所示，该模块用于展示历史 Commit 的整体性能概况。模块基于完整测试集，对不同 Commit 下的平均代码体积和平均执行周期进行统计与可视化展示，便于快速识别性能回退或异常波动，为后续的分析提供依据。除此之外，模块还支持按测试集、测试平台以及芯片版本区分数据，同时展示测试集在不同优化等级及版本下的平均代码体积与平均执行周期。

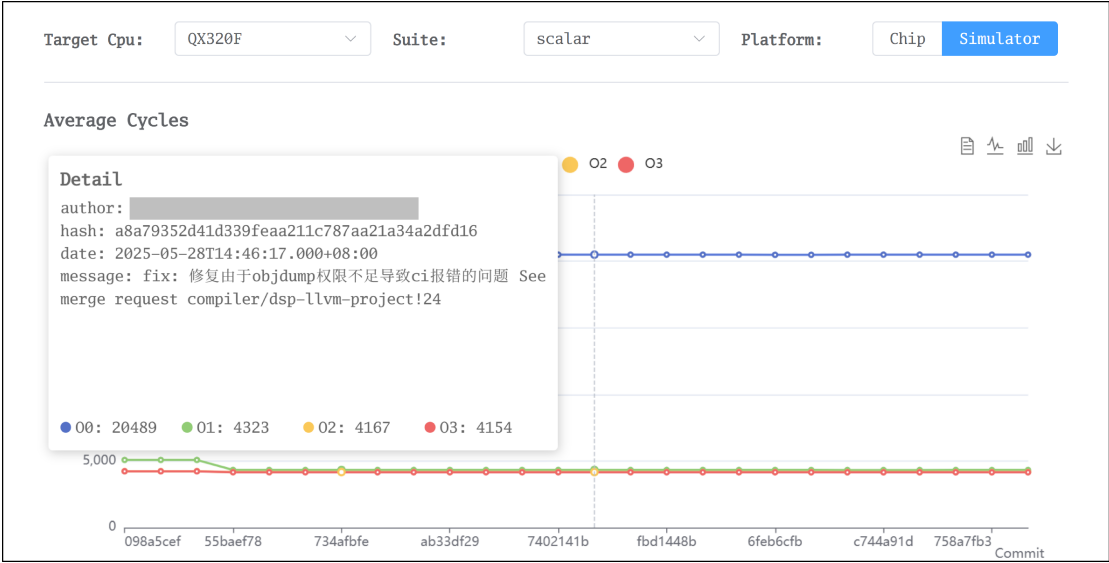


图 5-21 全局性能趋势分析界面展示图

2. 单测试用例性能演化分析模块

如图5-22所示，该模块用于展示单个测试用例在不同 Commit 下的性能变化情况，并支持按时间序列展示其执行周期、代码体积以及指令打包效率等关键指标。通过该模块，开发者可以精确定位某一性能变化首次引入的 Commit，从而显著提升性能回归分析的效率与准确性。

3. 跨版本性能差异对比分析模块

如图5-23所示，该模块用于对任意两个 Commit 之间的性能指标进行对比分析。模块不仅提供测试用例级别的执行周期与代码体积对比结果，还提供样例汇编代码对比的功能，以此帮助开发者从底层代码生成角度分析性能差异的根本原因，适用于优化效果验证与回归问题定位的情景。



图 5-22 单测试用例性能演化分析界面展示图

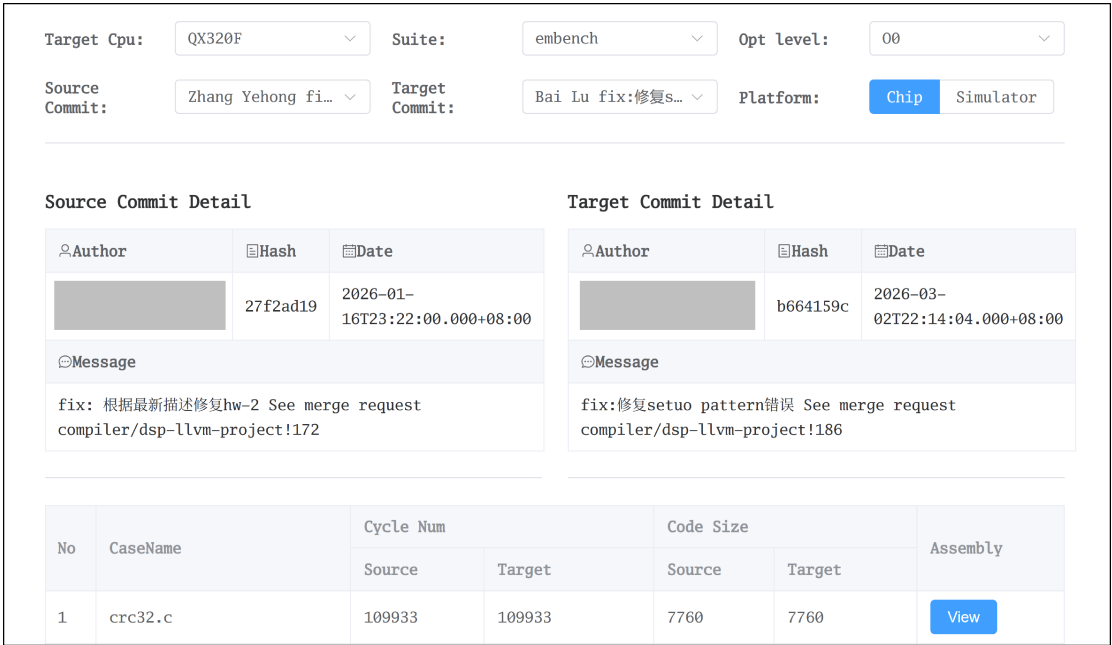


图 5-23 跨版本性能差异对比分析界面展示图

4. 本地性能测试报告上传模块

如图5-24所示，该模块用于上传本地生成的性能测试结果，并与 Master 分支的结果进行对比分析。该模块为开发者在本地修改编译器或新增优化策略后的快速验证提供了便利。

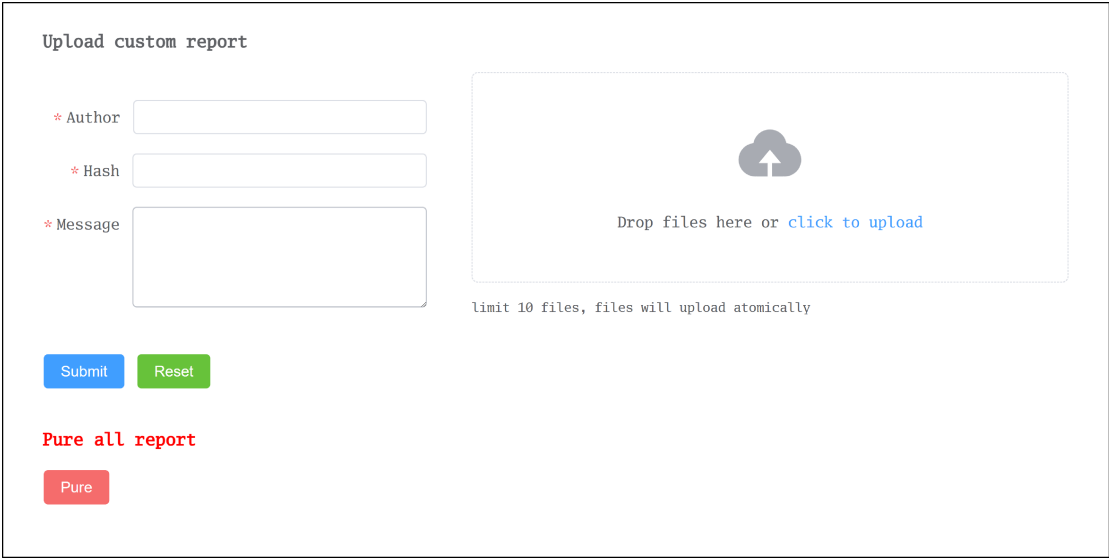


图 5-24 本地性能测试报告上传界面展示图

5.5 本章小结

本章首先用一系列有代表性的示例程序来验证面向 DSP 后端的 GlobalISel 框架代码生成的正确性。示例程序覆盖了指令合法化、寄存器组选择以及机器指令选择等关键阶段，从而确保了 GlobalISel 在 DSP 平台上的功能正确性。

随后针对本文提出的两阶段优化策略，编写典型样例来进行有效性的验证，之后基于 Embench 测试集开展系统性量化评估，实验结果表明两阶段优化策略在多数场景都能有效减少代码体积和执行周期。

本章接着对代码体积、执行周期以及编译时间等关键性能指标进行了全面的对比分析。实验结果表明，优化后的 GlobalISel 在各关键环节均能保持原始程序语义与数据依赖关系，其生成的目标代码符合 DSP 指令集规范与控制流逻辑，并且在性能上有明显的提升。

最后本章展示了测试评估平台的核心功能界面，其中包括全局性能趋势分析、单测试用例性能演化追踪、跨版本性能对比和本地测试报告上传等界面。这些功能为实验的正确性验证和性能分析提供了高效的数据支持与可视化手段，并有助于 DSP 工具链快速定位性能回退与代码生成缺陷。

第 6 章 总结与展望

6.1 论文总结

本文以 LLVM 编译器框架为平台，针对 VLIW DSP 架构在指令选择阶段面临的性能与可扩展性挑战，从框架适配、优化策略到评估体系三个层面展开了系统研究，形成了一套面向 DSP 后端的全局指令选择实现与测试评估方案。全文的主要研究内容与成果可以总结为以下几个方面。

在框架适配层面，本文探索并实现了 GlobalISel 这一新型指令选择技术在 DSP 后端的完整适配路径。通过解耦架构相关的调用约定、合法化规则与寄存器信息，验证了 GlobalISel 模块化设计在处理复杂指令集和异构寄存器资源方面的优势，为其他非主流架构快速接入 LLVM 生态提供了实践参考。

在优化技术层面，本文深入分析了 GlobalISel 流程中指令冗余的来源，并据此提出了“合法化前通用化简”与“合法化后架构专属优化”的分阶段策略。该策略不仅有效缩减了代码体积和执行周期，更重要的是，它展示了如何在保持编译器后端模块化、可维护性的同时，渐进式地注入高性能优化，为平衡编译速度与代码质量提供了新思路。

在评估体系层面，本文构建了一个面向编译器后端的自动化、可量化的性能测试评估平台。该平台通过引入随机测试和持续集成思想，解决了传统测试方法对后端优化效果评估不足的痛点。除此之外，本文创新性地结合目标架构特性来对 YARPGen 进行定制化扩展，提高了测试的覆盖率。该平台的设计与实现为编译器优化提供了可靠的数据支撑。

最后，本文从编译时间、代码体积和执行周期等多个维度，对基于 DAG 的指令选择方案、未优化的 GlobalISel 以及优化后的 GlobalISel 进行了对比分析。实验结果表明，在 DSP 架构下，GlobalISel 在编译效率方面具有一定优势；在引入本文提出的优化策略后，其生成代码在体积和执行效率等方面均表现出较为稳定的改进效果，从而验证了所提出方法在工程实践中的有效性。

6.2 论文展望

本文在 DSP 架构下对 GlobalISel 的实现和优化进行了系统的研究，并针对 DSP 现有测试体系中存在的问题，设计并实现了一套测试评估平台。尽管这些工作在实践中取得了初步成效，但在优化深度与覆盖范围等方面仍存在进一步

提升的空间，后续工作可从以下几个方向进行展开。

首先，本文当前实现的优化策略主要集中在局部模式合并和指令级重写，并没有充分利用到更高层次的全局信息。未来可以将数据流分析、控制流分析与 GlobalISel 的合并优化机制相结合，在函数级层面挖掘更多的优化机会，用于进一步提升代码质量和执行效率。

其次，本文的优化设计主要围绕通用算术指令和部分专用指令展开，未来可以进一步针对流水线结构、指令并行执行能力以及专用加速单元，设计更细粒度的目标相关优化策略，使 GlobalISel 在 DSP 平台上的优势得到更充分发挥。

最后，当前平台的重心在于对性能指标进行离线统计分析与可视化呈现。为增强其在持续集成与日常开发流程中的实用价值，后续可考虑引入更为自动化的性能回归检测机制。例如，可建立基于预设性能阈值的自动判定策略，或采用对性能数据趋势进行智能分析的告警模型，从而实现对性能退化的早期发现与主动预警。

参考文献

- [1] PROAKIS J G. Digital signal processing: principles, algorithms, and applications, 4/e[M]. Pearson Education India, 2007.
- [2] WANG Y, LI C, LIU C, et al. Advancing dsp into hpc, ai, and beyond: challenges, mechanisms, and future directions[J]. CCF Transactions on High Performance Computing, 2021, 3(1): 114-125.
- [3] SONG W, XU D, CHEN L. Overview of dsp architecture development[J]. Microelectronics & Computer, 2023, 40(4): 1-7.
- [4] FLYNN M J. Some computer organizations and their effectiveness[J]. IEEE transactions on computers, 2009, 100(9): 948-960.
- [5] FISHER J A. Very long instruction word architectures and the eli-512[C]//25 years of the International symposia on Computer architecture (selected papers). 1998: 263-273.
- [6] RAU B R, FISHER J A. Instruction-level parallel processing: History, overview, and perspective[M]//Instruction-Level Parallelism: A Special Issue of The Journal of Supercomputing. Springer, 2011: 9-50.
- [7] GIORDANI M, POLESE M, MEZZAVILLA M, et al. Toward 6g networks: Use cases and technologies[J]. IEEE communications magazine, 2020, 58(3): 55-61.
- [8] SATYANARAYANAN M, BAHL P, CACERES R, et al. The case for vm-based cloudlets in mobile computing[J]. IEEE pervasive Computing, 2009, 8(4): 14-23.
- [9] MUCHNICK S. Advanced compiler design implementation[M]. Morgan kaufmann, 1997.
- [10] BACKUS J W, BEEBER R J, BEST S, et al. The fortran automatic coding system [C]//Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability. 1957: 188-198.

- [11] CONWAY M E. Design of a separable transition-diagram compiler[J]. Communications of the ACM, 1963, 6(7): 396-408.
- [12] AHO A V, ULLMAN J D. The theory of parsing, translation, and compiling: Vol. 1[M]. Prentice-Hall Englewood Cliffs, NJ, 1972.
- [13] JOHNSON S C, RITCHIE D M. Unix time-sharing system: Portability of c programs and the unix system[J]. The Bell System Technical Journal, 1978, 57(6): 2021-2048.
- [14] STALLMAN R M, MCGRATH R, SMITH P. Gnu make[J]. Free Software Foundation, Boston, 1988.
- [15] LATTNER C, ADVE V. Llvm: A compilation framework for lifelong program analysis & transformation[C]//International symposium on code generation and optimization, 2004. CGO 2004. IEEE, 2004: 75-86.
- [16] 刘颖, 吕方, 王蕾, 等. 异构并行编程模型研究与进展[J]. 软件学报, 2014, 25(7): 17.
- [17] GUIDE D. Cuda c programming guide[J]. NVIDIA, July, 2013, 29(31): 6.
- [18] NICKOLLS J. Scalable parallel programming with cuda introduction[C]//2008 IEEE Hot Chips 20 Symposium (HCS). IEEE Computer Society, 2008: 1-9.
- [19] BEZBARUAH M, DHAKULKAR S, PANDEY P, et al. Comparative analysis of gcc and llvm for performance optimization on aarch64[C]//2024 IEEE High Performance Extreme Computing Conference (HPEC). IEEE, 2024: 1-6.
- [20] HJORT BLINDELL G. Tree covering[M]//Instruction Selection: Principles, Methods, and Applications. Springer, 2016: 31-76.
- [21] SMOTHERMAN M, KRISHNAMURTHY S, ARAVIND P, et al. Efficient dag construction and heuristic calculation for instruction scheduling[C]//Proceedings of the 24th annual international symposium on Microarchitecture. 1991: 93-102.
- [22] GRIFFITH A. Gcc: the complete reference[M]. McGraw-Hill, Inc., 2002.
- [23] JOHNSON S C. A portable compiler: Theory and practice[C]//Proceedings of the 5th ACM SIGACT-SIGPLAN symposium on Principles of programming languages. 1978: 97-104.

-
- [24] BILARDI G, PINGALI K. The static single assignment form and its computation [J]. Cornell University Technical Report, 1999.
- [25] LEIDEL J, KABRICK R, DONOFRIO D. Toward an automated hardware pipelining llvm pass infrastructure[C]//2021 IEEE/ACM 7th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC). IEEE, 2021: 39-49.
- [26] HUBER J, WEI W, GEORGAKOUDIS G, et al. A case study of llvm-based analysis for optimizing simd code generation[C]//International Workshop on OpenMP. Springer, 2021: 142-155.
- [27] FANG S, ZHENG W. Retrofitting control flow graphs in llvm ir for auto vectorization[A]. 2025.
- [28] FARVARDIN K, REPPY J. A new backend for standard ml of new jersey[C]//Proceedings of the 32nd Symposium on Implementation and Application of Functional Languages. 2020: 55-66.
- [29] APPEL A W, MACQUEEN D B. Standard ml of new jersey[C]//International Symposium on Programming Language Implementation and Logic Programming. Springer, 1991: 1-13.
- [30] MPEIS P, PETOUMENOS P, HAZELWOOD K, et al. Developer and user-transparent compiler optimization for interactive applications[C]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 268-281.
- [31] LAMMICH P. Refinement of parallel algorithms down to llvm[C]//13th International Conference on Interactive Theorem Proving (ITP 2022). Dagstuhl, 2022: 24-1.
- [32] BALASUBRAMANIAN K K, DI SALVO M, ROCCHIA W, et al. Designing risc-v instruction set extensions for artificial neural networks: An llvm compiler-driven perspective[J]. IEEE Access, 2024, 12: 55925-55944.
- [33] DE LA TORRE J C, JAREÑO J, ARAGÓN-JURADO J M, et al. Source code obfuscation with genetic algorithms using llvm code optimizations[J]. Logic Journal of the IGPL, 2025, 33(5): jzae069.

- [34] FAN Y, REGEHR J. High-throughput, formal-methods-assisted fuzzing for llvm [C]//2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). IEEE, 2024: 349-358.
- [35] 刘玉, 刘谷, 耿锐. 基于 LLVM 实现的国产 DSP 优化编译器[J]. 中国集成电路, 2020, 29(7): 24-28.
- [36] 沈莉, 周文浩, 王飞, 等. swLLVM: Optimized Compiler for New Generation Sunway Supercomputer[J]. Journal of Software, 2023, 35(5): 2359-2378.
- [37] 吴忱. 基于 LLVM 编译器的自动调优算法研究与实现[D]. 西安电子科技大学, 2024.
- [38] EBNER D, KRALL A, SCHOLZ B. Instruction code selection[C]//Lecture Notes in Computer Science: Vol. 12608 Compiler Construction. Springer, 2021: 17-32.
- [39] EBNER D, BRANDNER F, SCHOLZ B, et al. Generalized instruction selection using ssa-graphs[C]//Proceedings of the 2008 ACM SIGPLAN-SIGBED conference on Languages, compilers, and tools for embedded systems. 2008: 31-40.
- [40] COLOMBET Q. Global instruction selection[C]//LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2015.
- [41] RONG Y, YU Z, WENG Z, et al. Irfuzzer: Specialized fuzzing for llvm backend code generation[A]. 2024.
- [42] COLOMBET Q, LARIN S, et al. Globalisel: Design, implementation, and future directions[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2016.
- [43] BOGNER J. Fuzzing and testing instruction selection in llvm[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2017.
- [44] SANDERS D. The globalisel combiner[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2019.
- [45] ZHUFENG H, JIANDONG S. Optimization based on llvm global instruction selection[C]//Journal of Physics: Conference Series: Vol. 1856. IOP Publishing, 2021: 012004.

-
- [46] NACKE K, et al. Bringing globalisel to powerpc[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2022.
- [47] HOUTRYVE P. Extending the globalisel combiner with mir pattern support[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2024.
- [48] STADLER T. Directly generating gmir to speed up llvm compilation[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2024.
- [49] XIE J. Supporting scalable vectors in globalisel: A risc-v case study[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2024.
- [50] AMILKANTHWAR M. Eliminating globalisel fallback on aarch64 simd[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2024.
- [51] HICKEY N. Tracking and eliminating globalisel fallbacks with continuous integration[C]//Proceedings of the LLVM Developers' Meeting. San Jose, CA, USA: LLVM Foundation, 2025.
- [52] 彭成寒, 李灵, 戴贤泽, 等. 深入理解 LLVM 代码生成[M]. 北京: 机械工业出版社, 2024.
- [53] TAMKUN K. Using csmith to find compiler bugs[Z]. 2021.
- [54] LIVINSKII V, BABOKIN D, REGEHR J. Random testing for c and c++ compilers with yarpgen[J]. Proceedings of the ACM on Programming Languages, 2020, 4(OOPSLA): 1-25.
- [55] LIVINSKII V, BABOKIN D, REGEHR J. Fuzzing loop optimizations in compilers for c++ and data-parallel languages[J]. Proceedings of the ACM on Programming Languages, 2023, 7(PLDI): 1826-1847.

致 谢

不知不觉三年的时光已经过去，临别之际，借此机会感谢在我前行路上给予我指引、支持与陪伴的每一位师长、同窗与亲人。

首先，我由衷地感谢我的导师吴俊教授，吴老师不仅在科研上给予我系统而耐心的指导，也在生活中给予我真诚的关心与鼓励。从论文的选题，再到论文的反复修改与打磨，吴老师都给了我很大的帮助。

其次，我要感谢业鸿师兄。在项目推进的过程中，每当我遇到无法解决的问题时，师兄总能一针见血地指出问题关键，并耐心分享解决思路与实践经验，帮助我逐步厘清方向、走出迷雾。这份毫无保留的帮助与信任，使我在科研道路上少走了许多弯路。同时，我也很感谢陈航师兄、晟宁师兄以及泽鹏师兄在学习和生活中给予的支持与鼓励，与各位师兄的交流探讨，不仅拓宽了我的技术思路，也让我的科研时光充满了温暖与乐趣。

此外，我也要感谢在研究生阶段并肩同行的同窗好友。与你们在实验室中共同讨论问题、相互启发，在生活中彼此陪伴、互相扶持，是这段求学时光中不可或缺的珍贵记忆。正是这些点滴的交流与陪伴，让科研之路不再孤单，也让三年的学习生活更加充实而难忘。

最后，我要感谢我的家人。三年来，他们始终是最坚实的后盾，无论顺境还是逆境，都给予我无条件的支持。正是家人的默默付出与殷切期盼，才让我能够心无旁骛地投入到学术研究中，勇敢追逐自己的目标。

复旦大学 学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。论文中除特别标注的内容外，不包含任何其他个人或机构已经发表或撰写过的研究成果。对本研究做出重要贡献的个人和集体，均已在论文中作了明确的声明并表示了谢意。本声明的法律结果由本人承担。

作者签名：_____ 日期：_____

复旦大学 学位论文使用授权声明

本人完全了解复旦大学有关收藏和利用博士、硕士学位论文的规定，即：学校有权收藏、使用并向国家有关部门或机构送交论文的印刷本和电子版本；允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其它复制手段保存论文。涉密学位论文在解密后遵守此规定。

作者签名：_____ 导师签名：_____ 日期：_____